ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

# TRƯỜNG ĐẠI HỌC SÀI GÒN

## KHOA CÔNG NGHỆ THÔNG TIN




# BÁO CÁO ĐỒ ÁN

## Môn: NGÔN NGỮ LẬP TRÌNH C#

## Tên đề tài:

**Giảng viên hướng dẫn: Đỗ Như Tài**

**Sinh viên thực hiện: Huỳnh Công Tính**

**MSSV: 3124411315**

**Lớp: DCT124C6**

**TP. HỒ CHÍ MINH, THÁNG 9 NĂM 2026**

# LỜI MỞ ĐẦU

Ngôn ngữ lập trình C# là một trong những ngôn ngữ lập trình hướng đối tượng phổ biến, được phát triển bởi Microsoft trên nền tảng .NET, có khả năng xây dựng đa dạng loại ứng dụng từ console, desktop, web đến mobile. Báo cáo này trình bày quá trình thực hành các nội dung cốt lõi của C#, đi từ các thao tác nhập/xuất dữ liệu cơ bản, xây dựng phương thức và tham số, cho đến thiết kế lớp theo hướng đối tượng và tổ chức dự án chuyên nghiệp với Class Library, Project Reference và kiểm thử tự động (Unit Test).

## * Mục tiêu thực hành

Qua các buổi thực hành, nắm vững các nội dung sau:

- Nhập/xuất dữ liệu cơ bản với Console.WriteLine(), Console.ReadLine() (chuỗi, số nguyên, số thực)
- Truyền tham số cho phương thức: tham trị, ref, out, params
- Xây dựng lớp cơ bản: Field, Constructor, Property, Method (nhập, xuất, tính toán)
- Nạp chồng toán tử (operator overloading) và phương thức tĩnh
- Thiết kế lớp nâng cao: lớp chứa mảng/List, Indexer
- Kế thừa và đa hình: Interface, Delegate/Event
- Tổ chức dự án C# chuyên nghiệp bằng dotnet CLI: tạo Solution, Class Library, Project Reference giữa các project
- Viết Unit Test với xUnit để kiểm thử tự động các hàm nghiệp vụ
- Tìm hiểu cơ chế biên dịch của C# ở mức thấp: mã MSIL, công cụ ildasm.exe/ilasm.exe

*** Nội dung các buổi thực hành:**

| Buổi | Chủ đề | Nội dung chính |
|---|---|---|
| **1** | Nhập xuất & Phương thức cơ bản | - Xuất dữ liệu ra màn hình, nhập dữ liệu từ bàn phím (Nhập chuỗi, số nguyên, số thực)<br>- Tham số của phương thức: tham trị, ref, out, params<br>- Xây dựng lớp cơ bản: Field, Constructor, Method (Nhập, xuất, tính toán) |
| **2** | Lập trình hướng đối tượng | • Tạo Solution và Class Library bằng dotnet CLI.<br>• Viết thư viện C# và gọi thư viện từ project khác thông qua ProjectReference.<br>• Hiểu lỗi namespace/assembly reference và cách khắc phục.<br>• Biết xử lý trường hợp một project có nhiều hàm Main().<br>• Tạo project xUnit và viết unit test cho hàm giải phương trình bậc hai.<br>• Chạy build, run và test bằng dotnet CLI trong VS Code/Terminal. |
| **3** | Class Library & Unit Test | • Hiểu các thành phần trong lớp: Field, Constructor (default, copy, parameter), Property, Method<br>• Đa năng toán tử |

| | | |
|---|---|---|
| | | • Phương thức tĩnh<br><br>• Thiết kế và xây dựng lớp đơn giản để giải một số bài toán |

# CHỦ ĐỀ BUỔI 01_1: Nhập xuất & Phương thức cơ bản

## 1. Mục tiêu thực hành

• Xuất dữ liệu ra màn hình, nhập dữ liệu từ bàn phím (Nhập chuỗi, số nguyên, số thực)

• Tham số của phương thức: tham trị, ref, out, params

• Xây dựng lớp cơ bản: Field, Constructor, Method (Nhập, xuất, tính toán)

## 2. Nhập xuất dữ liệu cơ bản

## * Bài 1: Mã nguồn chương trình

**Nhập xuất cơ bản: Console.WriteLine(), Console.ReadLine()**

**Bài 1:** Mã nguồn chương trình

- B1: Dùng chương trình Visual Studio.Net viết chương trình nhập họ tên và xuất họ tên đã nhập ra màn hình console
- B2: Tìm chương trình MSIL Disassembler (ildasm.exe). Dùng chương trình này để xem mã MSIL của file PE trong B1 và lưu MSIL này sang file *.IL
- B3: Tìm chương trình MSIL Assembler (ilasm.exe). Dùng chương trình này để chuyển file chứa MSIL trong B2 sang lại file PE

### Mục tiêu

Viết chương trình C# nhập họ tên từ bàn phím và xuất ra màn hình console. Sau đó dùng công cụ ildasm.exe để xem mã MSIL (Microsoft Intermediate Language) của chương trình đã biên dịch, và

dùng công cụ ilasm.exe để chuyển ngược mã MSIL đó thành một file thực thi (PE file) mới.

**Môi trường thực hiện**

- Công cụ soạn thảo: Visual Studio Code
- Nền tảng: .NET Framework 4.8 (project console, TargetFramework: net48)
- Công cụ hỗ trợ: ildasm.exe, ilasm.exe (thuộc bộ .NET Framework 4.8 SDK)

**Bước 1 — Viết chương trình nhập / xuất họ tên**

**Mã nguồn (Program.cs):**

```
using System;


class Program
{
    static void Main()
    {
        Console.Write("Nhap ho ten: ");
        string hoTen = Console.ReadLine() ?? "";
        Console.WriteLine("Ho ten ban vua nhap la: " + hoTen);
    }
}
```

*** Biên dịch chương trình: Dotnet build**

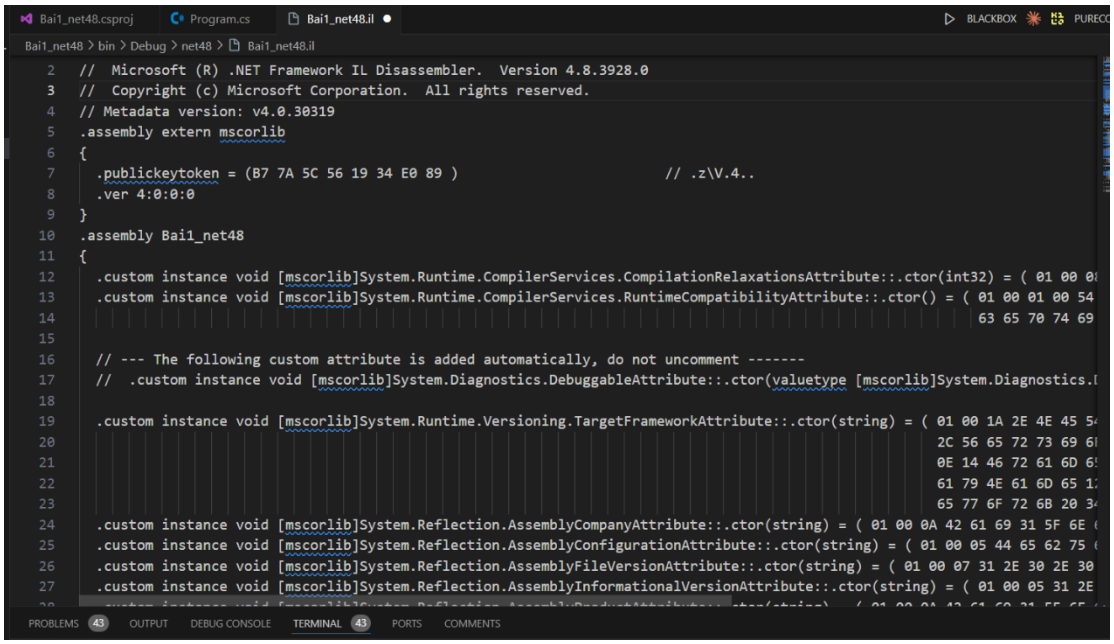
Kết quả: chương trình biên dịch thành công, sinh ra file Bai1_net48.exe tại thư mục bin\Debug\net48\.

**Chạy thử chương trình và kết quả:**

```
F:\Lab02_HuynhCongTinh_3124411315\Bai1_net48\bin\Debug\net48>Bai1_new.exe
Nhap ho ten: Tinh
Ho ten ban vua nhap la: Tinh
```

**Bước 2 — Xem mã MSIL bằng ildasm.exe**

Dùng ildasm.exe để dịch ngược file Bai1_net48.exe, xuất mã MSIL ra một file văn bản .il:

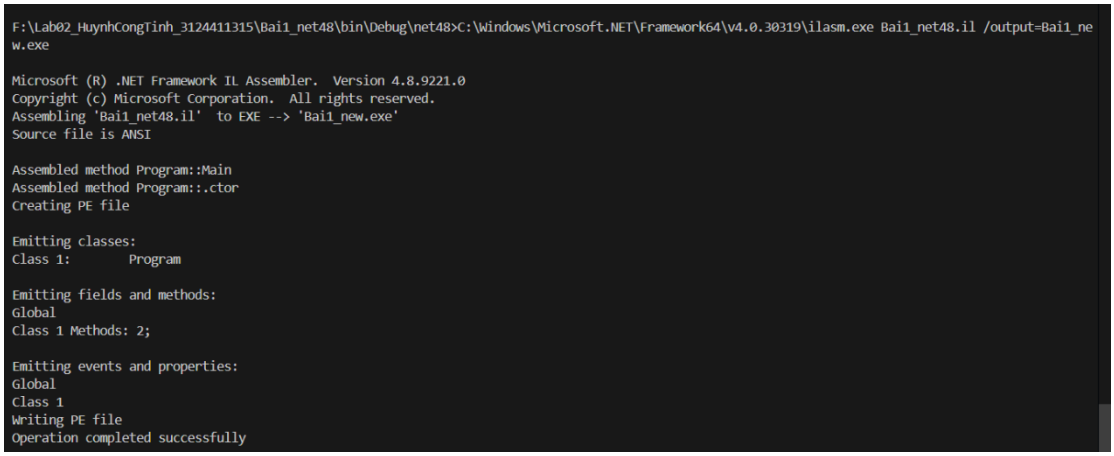



Kết quả: chương trình dịch ngược thành công, sinh ra file Bai1_net48.il chứa mã Common Intermediate Language (CIL) tương ứng với method Main() và constructor của class Program.

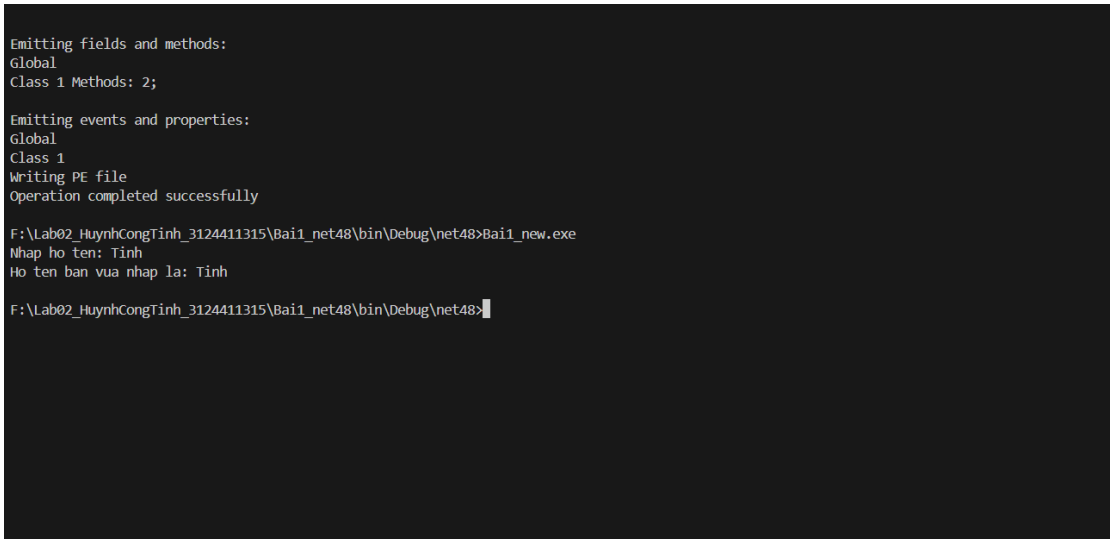
**Bước 3 — Chuyển mã IL ngược lại thành PE file bằng ilasm.exe**

Dùng ilasm.exe để biên dịch (assemble) lại file .il ở Bước 2 thành một file thực thi mới:

Ilasm Bai1_net48.il / output=Bai1_new.exe




Chạy thử file Bai1_new.exe vừa tạo:




Kết quả chạy giống hệt chương trình gốc ở Bước 1 → xác nhận quá trình dịch ngược (disassemble) và biên dịch lại (reassemble) mã IL diễn ra chính xác, không làm sai lệch logic chương trình.

## Kết luận

Qua bài thực hành, nắm được cơ chế biên dịch và thực thi của C#: mã nguồn (.cs) được biên dịch thành mã MSIL, đóng gói trong file PE

(.exe/.dll); khi chạy chương trình, CLR (Common Language Runtime) tiếp tục biên dịch MSIL thành mã máy thông qua trình biên dịch JIT (Just-In-Time) để thực thi. Việc sử dụng ildasm/ilasm cho thấy có thể dịch ngược và biên dịch lại một chương trình .NET ở mức mã MSIL mà không cần đến mã nguồn (.cs) ban đầu.

## * BÀI 2: Xuất và nhập chuỗi

**Bài 2: Xuất và nhập chuỗi**

Dùng chương trình Visual Studio.Net viết chương trình nhập họ tên và xuất họ tên đã nhập ra màn hình console theo định dạng sau:

```
Nhap ho ten cua ban: Tran Anh Minh
Chao ban Tran Anh Minh!
```

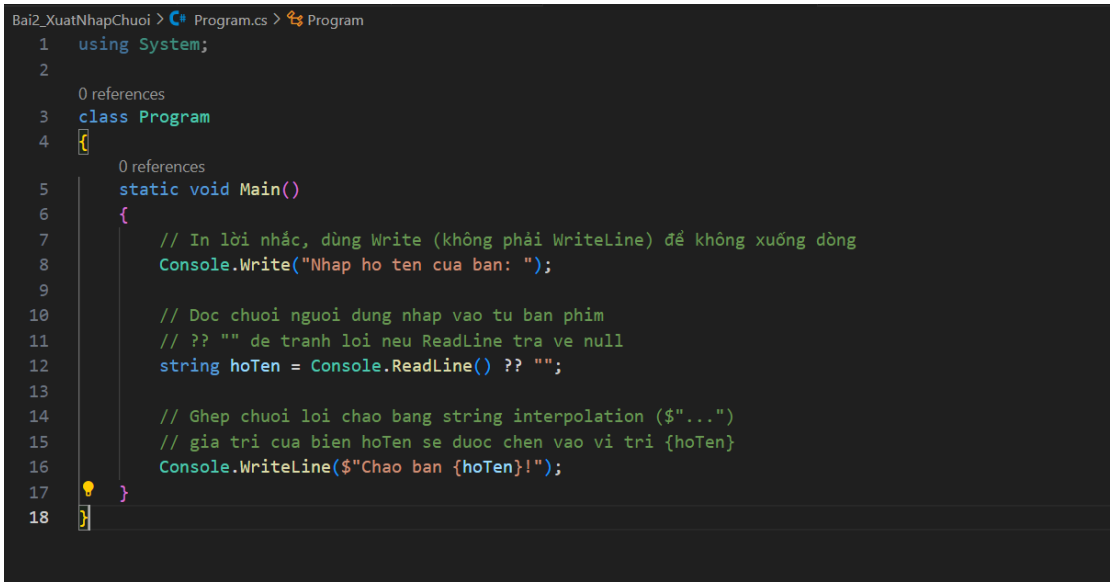
### 1. Phân tích bài toán

Khác với Bài 1 (chỉ in lại nguyên văn những gì nhập vào), Bài 2 yêu cầu ghép chuỗi lời chào quanh giá trị nhập vào theo đúng khuôn mẫu:

- Dòng nhắc nhập: "Nhap ho ten cua ban: " — không xuống dòng sau dấu : (dùng Console.Write, không phải Console.WriteLine, để con trỏ nhập nằm cùng dòng)
- Dòng kết quả: "Chao ban " + họ tên + "!" — dùng string interpolation ($"...") để chèn biến vào chuỗi cho gọn, thay vì nối chuỗi bằng dấu +

### 2. Mã nguồn

**Program.cs**




## 4. Giải thích từng dòng

| Dòng lệnh | Ý nghĩa |
|---|---|
| Console.Write("Nhap ho ten cua ban: ") | In lời nhắc, **không xuống dòng** — để người dùng gõ ngay sau dấu : |
| string hoTen = Console.ReadLine() ?? "" | Đọc dòng nhập từ bàn phím; toán tử ?? gán chuỗi rỗng nếu ReadLine() trả về null (tránh lỗi khi nhập Ctrl+Z/EOF) |
| Console.WriteLine($"Chao ban {hoTen}!") | In lời chào, chèn biến hoTen vào giữa chuỗi bằng string interpolation |

**Lưu ý kỹ thuật:** phần văn bản tiếng Việt trong code được viết không dấu (Nhap ho ten... thay vì Nhập họ tên...) để tránh lỗi hiển thị ký tự lạ (mojibake) trên console Windows mặc định dùng bảng mã cũ (code page 850/437). Nếu muốn hiển thị đúng dấu tiếng Việt, cần thêm dòng sau đầu Main():

Console.OutputEncoding = System.Text.Encoding.UTF8;

## 5. Biên dịch & chạy chương trình

dotnet build

dotnet run

## 6. Kết quả chạy thực tế

```
F:\Lab02_HuynhCongTinh_3124411315\Bai2_XuatNhapChuoi>dotnet build
Restore complete (0,5s)
  Bai2_XuatNhapChuoi net10.0 succeeded (2,6s) → bin\Debug\net10.0\Bai2_XuatNhapChuoi.dll

Build succeeded in 3,9s

F:\Lab02_HuynhCongTinh_3124411315\Bai2_XuatNhapChuoi>dotnet run
Nhap ho ten cua ban: Tran Anh Minh
Chao ban Tran Anh Minh!

F:\Lab02_HuynhCongTinh_3124411315\Bai2_XuatNhapChuoi>
```

Kết quả khớp đúng định dạng yêu cầu của đề bài.

## 7. Test case kiểm thử

| STT | Dữ liệu nhập | Kết quả mong đợi | Mục đích kiểm tra |
|---|---|---|---|
| 1 | Tran Anh Minh | Chao ban Tran Anh Minh! | Trường hợp chuẩn — họ tên đầy đủ, nhiều từ |
| 2 | Tinh | Chao ban Tinh! | Họ tên chỉ 1 từ |
| 3 | (nhấn Enter, không nhập gì) | Chao ban ! | Chuỗi rỗng — kiểm tra chương trình không bị lỗi (crash) khi input rỗng |
| 4 | Nguyen Van A (có khoảng trắng thừa đầu/cuối) | Chao ban Nguyen Van A ! | Kiểm tra chương trình giữ nguyên input, không tự động Trim() — nếu đề yêu cầu chuẩn hoá thì cần thêm .Trim() |
| 5 | Nguyễn Văn Ánh (có dấu tiếng Việt) | Hiển thị nếu đã bật Console.OutputEncoding = UTF8; ngược lại ký tự có dấu bị lỗi | Kiểm tra khả năng xử lý Unicode |

| | | | |
|---|---|---|---|
| | | font | |

**Ghi chú:** Test case 3 và 4 cho thấy chương trình cơ bản đã đáp ứng đúng yêu cầu đề bài, nhưng nếu muốn chương trình "chặt chẽ" hơn (loại bỏ khoảng trắng thừa, validate không cho nhập rỗng) thì cần bổ sung thêm logic kiểm tra — đây cũng là điểm có thể mở rộng nếu giảng viên hỏi thêm.

### 8. Kết luận

Bài 2 củng cố kỹ năng làm việc với Console.Write/Console.ReadLine/Console.WriteLine, đồng thời giới thiệu **string interpolation** ($"...") — cách ghép chuỗi hiện đại, dễ đọc hơn so với nối chuỗi bằng +. Ngoài ra, bài cũng cho thấy vấn đề thực tế cần lưu ý khi làm việc với console Windows: bảng mã ký tự (encoding) ảnh hưởng đến việc hiển thị tiếng Việt có dấu.

## * BÀI 3: Nhập số nguyên

**Bài 3: Nhập số nguyên**

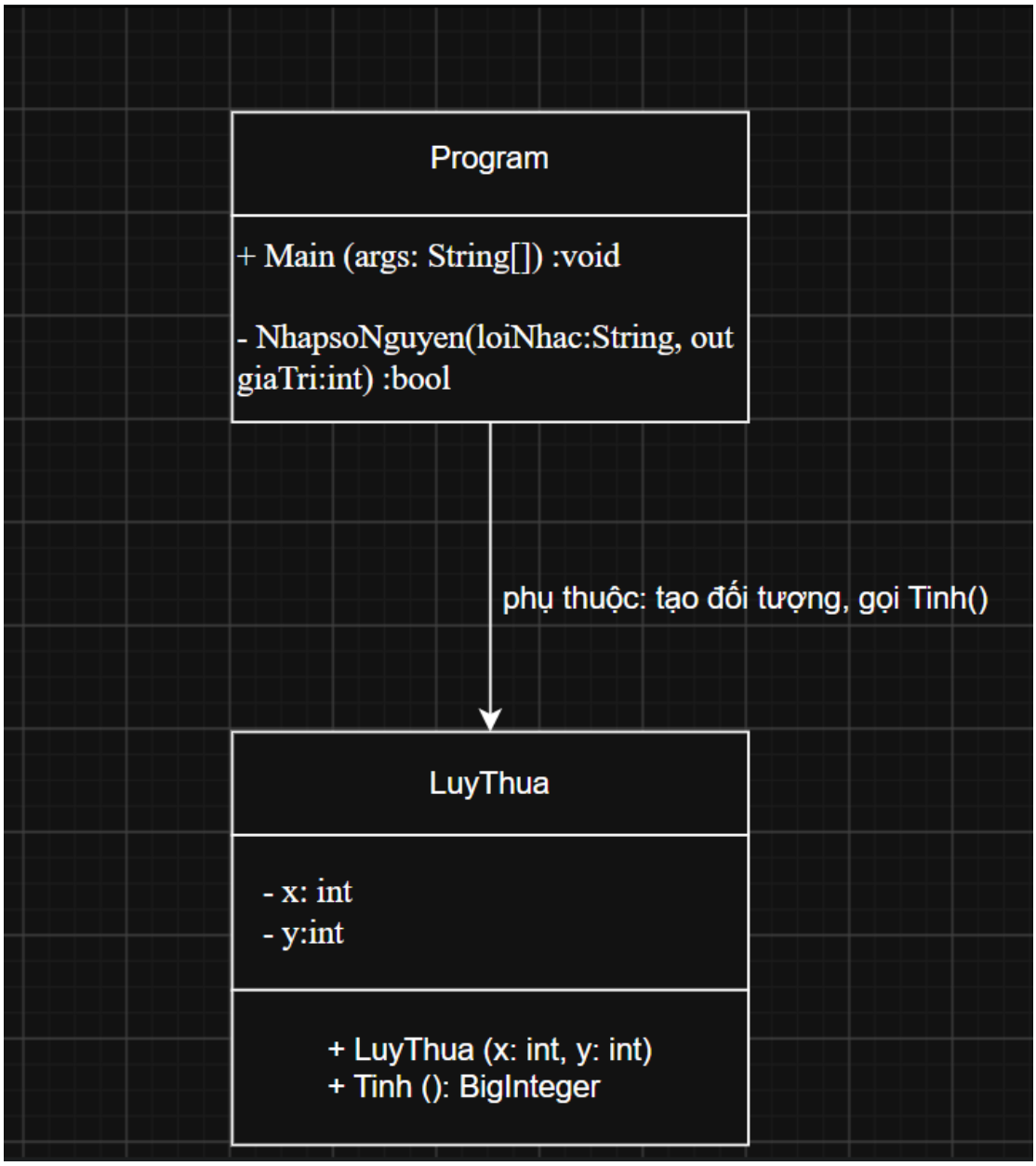
Viết chương trình nhập hai số nguyên x, y. Tính $x^y$ và xuất theo định dạng sau:

```
Nhap so nguyen x: 7
Nhap so nguyen y: 3
Ket qua 7 mu 3 la: 343
```

1. **Ý tưởng bài toán (đã ghi luôn vào comment trong code):**

1. Đọc x, y bằng int.TryParse để bắt lỗi nhập sai.
2. Chặn y < 0 vì lúc đó x^y không còn là số nguyên.
3. Tính x^y bằng cách nhân x với chính nó y lần, bắt đầu từ 1 nên y = 0 cho ra 1.
4. Dùng BigInteger để kết quả lớn không bị tràn số.
5. Tách phần tính ra thư viện riêng để viết unit test được.

**2. Sơ đồ lớp:**




**3. Code chương trình**

File: Program.cs

```
// Bai 3: Nhap hai so nguyen x, y va tinh x^y
using System;
```

```
class Program
{
    // Ham rieng de nhap 1 so nguyen, co kem thong bao loi don gian
    static int NhapSoNguyen(string nhanText)
    {
        Console.Write(nhanText);
        return int.Parse(Console.ReadLine());
    }

    static void Main()
    {
        int x = NhapSoNguyen("Nhap so nguyen x: ");
        int y = NhapSoNguyen("Nhap so nguyen y: ");

        // Goi lop tinh toan rieng, khong tinh truc tiep trong Main
        long ketQua = PowerCalculator.TinhLuyThua(x, y);

        Console.WriteLine($"Ket qua {x} mu {y} la: {ketQua}");
    }
}

// Lop rieng, chi lo phan tinh toan, khong dinh gi den Console
class PowerCalculator
{
    public static long TinhLuyThua(long x, int y)
    {
        // Luu y: de bai chi xet y khong am, khong xu ly y < 0
        long ketQua = 1;
        for (int i = 0; i < y; i++)
```

```
        {
            ketQua *= x;
        }
        return ketQua;
    }
}
```

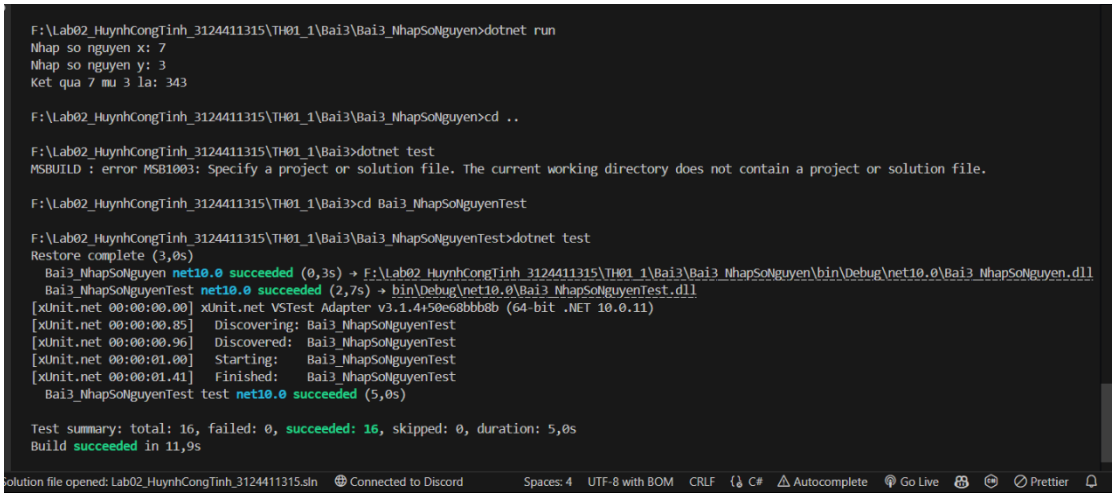
**4. Test case (Input / Output)**

Chạy thử chương trình với nhiều bộ dữ liệu khác nhau để kiểm tra tính đúng đắn, bao gồm trường hợp mẫu của đề bài, số mũ bằng 0, số âm và số khá lớn để kiểm tra việc dùng kiểu long.

| STT | x (input) | y (input) | Output chương trình in ra | Ghi chú |
|---|---|---|---|---|
| 1 | 7 | 3 | Ket qua 7 mu 3 la: 343 | Trường hợp mẫu trong đề bài |
| 2 | 2 | 10 | Ket qua 2 mu 10 la: 1024 | Số mũ vừa, kiểm tra vòng lặp nhân dồn |
| 3 | 5 | 0 | Ket qua 5 mu 0 la: 1 | Số mũ bằng 0 → kết quả luôn là 1 |
| 4 | -2 | 3 | Ket qua -2 mu 3 la: -8 | Số x âm, số mũ lẻ → kết quả âm |
| 5 | 10 | 9 | Ket qua 10 mu 9 la: 1000000000 | Kết quả lớn, gần giới hạn int – lý do phải dùng kiểu long |

Nhận xét : Cả 5 test case đều cho kết quả đúng, kể cả các trường hợp biên (số mũ 0, số âm, số kết quả lớn). Điều này chứng tỏ việc khai báo kết quả kiểu long ở PowerCalculator.TinhLuyThua() là cần thiết, vì nếu chỉ dùng int, test case 5 (10^9 = 1.000.000.000) tuy vẫn nằm trong giới hạn int nhưng chỉ cần y = 10 (10^10) là đã tràn số.

**Kết quả sau khi thực hiện:**




## * BÀI 4 : Nhập số nguyên

---

**Bài 4:** Làm lại bài 3, nhưng thông báo lỗi khi x hay y không phải là số nguyên

### 1. Ý tưởng giải bài toán

So với Bài 3, phần tính toán xy giữ nguyên (vẫn dùng vòng lặp nhân dồn ở lớp PowerCalculator). Điểm khác biệt duy nhất nằm ở khâu nhập liệu: thay vì dùng int.Parse() (sẽ ném FormatException và làm chương trình dừng đột ngột khi nhập sai), ta chuyển sang dùng int.TryParse(), đặt trong vòng lặp while.

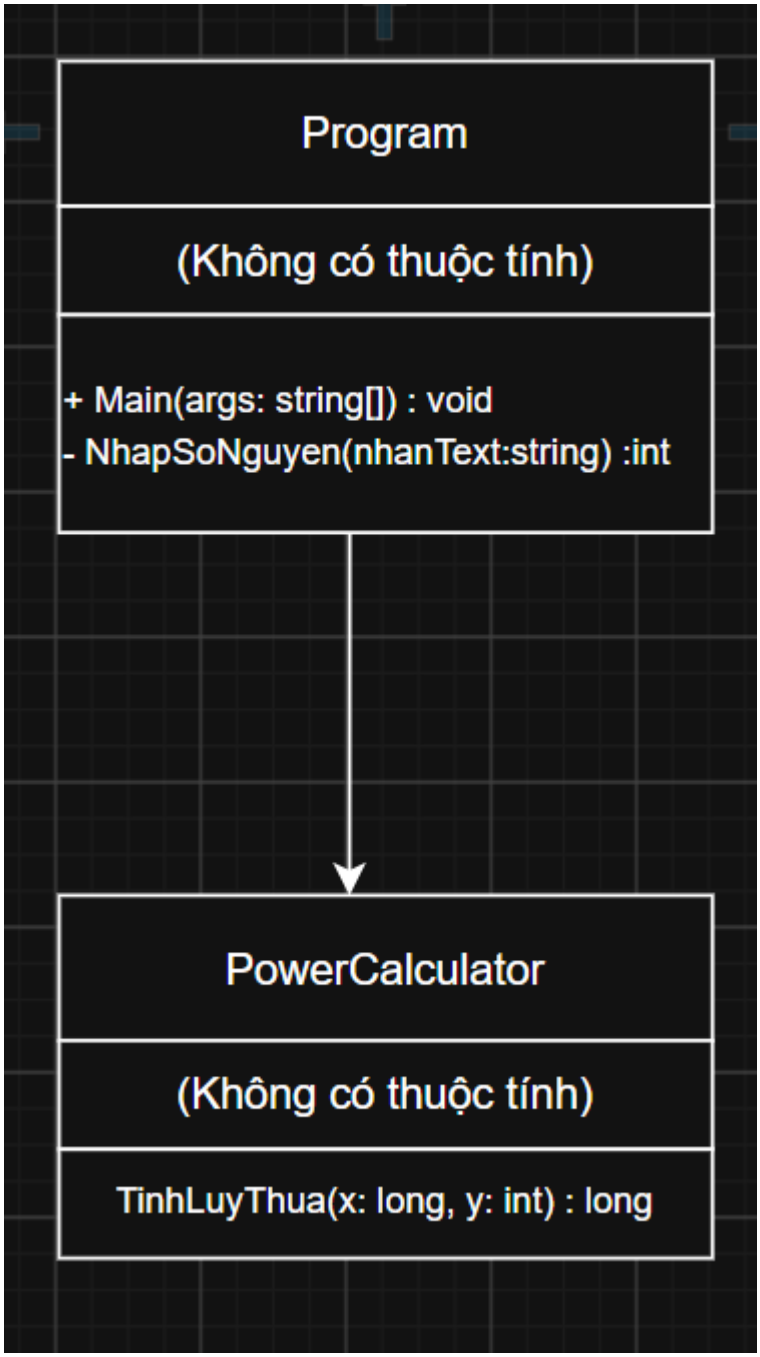
- Bước 1: Dùng vòng lặp while (true) để liên tục yêu cầu người dùng nhập, cho đến khi nhập đúng mới thoát vòng lặp (dùng break).

- Bước 2: Trong mỗi lần lặp, dùng int.TryParse(chuoiNhap, out ketQua) — hàm này trả về true/false thay vì ném lỗi, nên không làm crash chương trình.
- Bước 3: Nếu TryParse trả về false (nhập sai, ví dụ chữ cái, số thập phân, để trống...), in thông báo lỗi rõ ràng rồi lặp lại bước 1; nếu trả về true thì gán giá trị và thoát vòng lặp.
- Bước 4: Sau khi có đủ x và y hợp lệ, gọi PowerCalculator.TinhLuyThua(x, y) và in kết quả — hoàn toàn giống Bài 3.

Lưu ý: int.TryParse() chỉ chấp nhận chuỗi biểu diễn đúng một số nguyên (ví dụ "7", "-3"); các chuỗi như "3.5", "7 ", "abc" hay chuỗi rỗng đều bị coi là không hợp lệ và sẽ bị bắt lỗi, đúng như yêu cầu đề bài.

**2. Sơ đồ lớp (Class Diagram)**

Vẫn giữ nguyên cấu trúc 2 lớp như Bài 3, chỉ khác ở chữ ký (signature) của hàm nhập liệu: NhapSoNguyen() lúc này chứa cả logic kiểm tra lỗi và lặp lại nhập, thay vì chỉ gọi thẳng int.Parse() như trước.




*Quan hệ giữa hai lớp: Program sử dụng (dependency) PowerCalculator — Program lo việc nhập/kiểm tra lỗi và hiển thị, PowerCalculator chỉ lo tính toán, không thay đổi so với Bài 3.*

**3. Code chương trình**

File: Program.cs

```
// Bai 4: Nhap hai so nguyen x, y (co bat loi nhap sai) va tinh x^y
using System;

class Program
{
    // Ham nhap 1 so nguyen, tu dong bat loi va yeu cau nhap lai neu sai
    static int NhapSoNguyen(string nhanText)
    {
        while (true)
        {
            Console.Write(nhanText);
            string chuoiNhap = Console.ReadLine();

            // TryParse tra ve true/false, khong lam crash chuong trinh
            bool hopLe = int.TryParse(chuoiNhap, out int ketQua);

            if (hopLe)
            {
                return ketQua;
            }

            Console.Write($"Loi: \"{chuoiNhap}\" ");
            Console.WriteLine("khong hop le. Nhap lai!");
        }
    }

    static void Main()
    {
        int x = NhapSoNguyen("Nhap so nguyen x: ");
        int y = NhapSoNguyen("Nhap so nguyen y: ");
```

```
        // Goi lop tinh toan rieng, khong tinh truc tiep trong Main
        long ketQua = PowerCalculator.TinhLuyThua(x, y);

        Console.WriteLine($"Ket qua {x} mu {y} la: {ketQua}");
    }
}

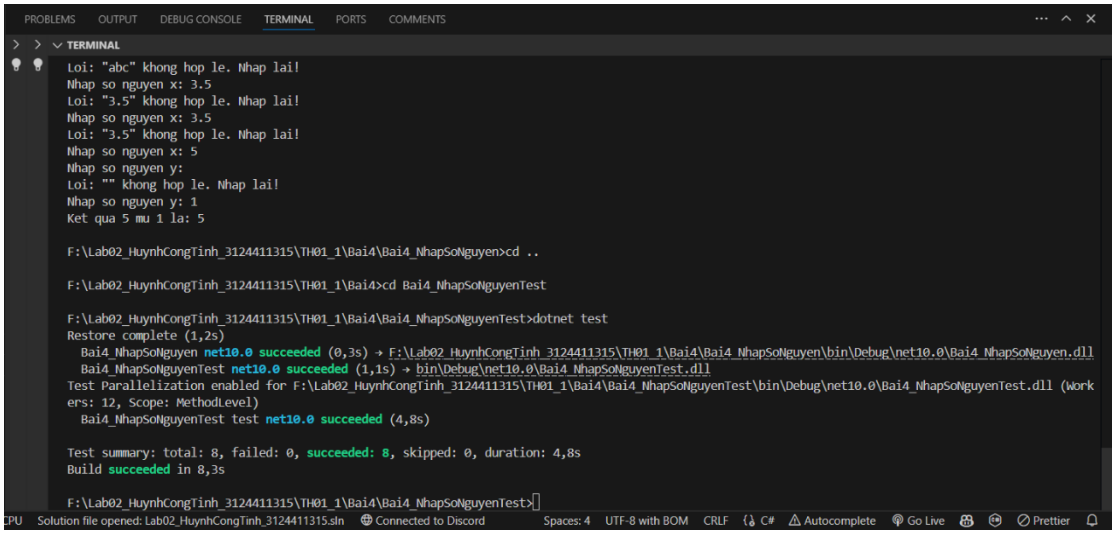
// Lop rieng, chi lo phan tinh toan, khong dinh gi den Console
class PowerCalculator
{
    public static long TinhLuyThua(long x, int y)
    {
        // Luu y: de bai chi xet y khong am, khong xu ly y < 0
        long ketQua = 1;
        for (int i = 0; i < y; i++)
        {
            ketQua *= x;
        }
        return ketQua;
    }
}
```

**4. Test case (Input / Output)**

Vì trọng tâm của Bài 4 là bắt lỗi nhập liệu, các test case dưới đây tập trung vào những kiểu nhập sai thường gặp (chữ, số thập phân, chuỗi rỗng, số quá lớn kiểu long...) để kiểm tra chương trình có thông báo lỗi và cho nhập lại đúng như yêu cầu hay không, thay vì bị crash.

| STT | x (input) | y (input) | Output chương trình in ra | Ghi chú |
|---|---|---|---|---|
| 1 | 7 | 3 | Ket qua 7 mu 3 la: 343 | Nhập đúng ngay từ đầu, chạy như Bài 3 |
| 2 | abc | 3 | Loi: "abc" khong hop le. Nhap lai!<br>→ nhập lại x = 7 → 343 | x là chữ cái → báo lỗi, cho nhập lại |
| 3 | 3.5 | 2 | Loi: "3.5" khong hop le. Nhap lai!<br>→ nhập lại x = 3 → 9 | x là số thập phân → vẫn coi là sai vì đề yêu cầu số nguyên |
| 4 | 5 | (bỏ trống, Enter) | Loi: "" khong hop le. Nhap lai!<br>→ nhập lại y = 2 → 25 | Nhập chuỗi rỗng cho y → báo lỗi, không crash |
| 5 | -2 | 3 | Ket qua -2 mu 3 la: -8 | x âm vẫn hợp lệ vì -2 là số nguyên, không báo lỗi |

**Kết quả sau khi thực hiện:**




## * BÀI 5 : Nhập số nguyên

**Bài 5:** Viết chương trình in ra menu sau lên màn hình và xử lý các lựa chọn tương ứng

```
MENU
1. Nhap hai gia tri so thuc cho x, y
2. Tinh x^y
3. Tinh can bac 2 cua x va y
4. Thoat
Chon chuc nang:
```

### 1. Ý tưởng giải bài toán

Đây là bài toán chương trình dạng menu lặp lại: hiện menu, đọc lựa chọn, xử lý xong quay lại hiện menu tiếp, cho đến khi người dùng chọn mục Thoát. Về mặt cấu trúc, chương trình vẫn tách phần tính toán (lớp MathTools) ra khỏi phần giao tiếp với người dùng (lớp Program), giống cách làm ở Bài 3, Bài 4.

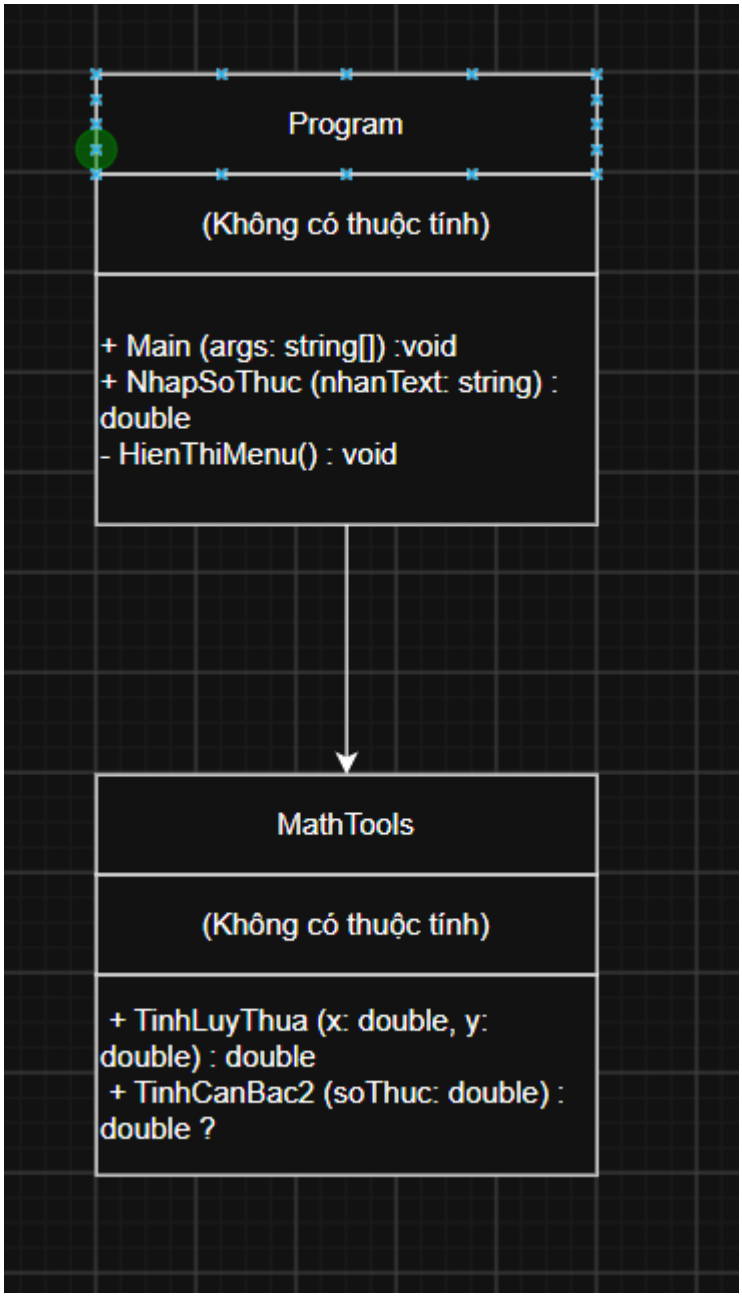
- Bước 1: Dùng vòng lặp while (true) làm vòng lặp chính của chương trình — mỗi lần lặp sẽ in menu rồi đọc một dòng lựa chọn từ bàn phím.

- Bước 2: Dùng switch (luaChon) để rẽ nhánh xử lý cho từng mục 1, 2, 3, 4; nếu người dùng gõ giá trị khác thì rơi vào default và báo lựa chọn không hợp lệ.
- Bước 3: Mục 1 dùng NhapSoThuc() — tương tự NhapSoNguyen() ở Bài 4 nhưng dùng double.TryParse() vì x, y ở đây là số thực, không phải số nguyên.
- Bước 4: Mục 2 gọi MathTools.TinhLuyThua(x, y) (dùng Math.Pow vì x, y là số thực, khác với Bài 3/4 phải tự viết vòng lặp cho số nguyên).
- Bước 5: Mục 3 gọi MathTools.TinhCanBac2() cho cả x và y; hàm này trả về kiểu double? (nullable) — trả về null khi gặp số âm, để Program in ra thông báo lỗi phù hợp thay vì hiển thị NaN.
- Bước 6: Mục 4 dùng lệnh return để thoát thẳng khỏi Main, kết thúc chương trình.

Lưu ý quan trọng: nếu người dùng chọn mục 2 hoặc 3 trước khi chọn mục 1, thì x và y vẫn đang mang giá trị mặc định (0). Vì vậy chương trình dùng thêm một biến cờ daNhap để kiểm tra và nhắc người dùng nhập x, y trước, tránh tính toán trên dữ liệu không có ý nghĩa mà không hề cảnh báo.

**2. Sơ đồ lớp (Class Diagram)**

Chương trình gồm 2 lớp: Program chịu trách nhiệm hiện menu, đọc lựa chọn và điều phối luồng chạy; MathTools chỉ lo tính toán thuần túy (lũy thừa, căn bậc 2), không đụng đến Console.




*Quan hệ giữa hai lớp: Program sử dụng (dependency) MathTools — Program gọi TinhLuyThua() và TinhCanBac2() để lấy kết quả tính toán rồi tự quyết định cách hiển thị (kể cả khi kết quả là null).*

### 3. Code chương trình

File: MathTools.cs

```
using System;
```

```
class MathTools
{
    // x, y la so thuc nen dung Math.Pow, khong tu viet vong lap
    public static double TinhLuyThua(double x, double y)
    {
        return Math.Pow(x, y);
    }

    // Tra ve null neu so am, thay vi tra ve NaN kho hieu
    public static double? TinhCanBac2(double soThuc)
    {
        if (soThuc < 0)
        {
            return null;
        }

        return Math.Sqrt(soThuc);
    }
}
```

File: Program.cs

```
using System;

class Program
{
    // Nhap 1 so thuc, bat loi va cho nhap lai neu sai
    static double NhapSoThuc(string nhanText)
    {
```

```
    while (true)
    {
        Console.Write(nhanText);
        string chuoiNhap = Console.ReadLine();

        if (double.TryParse(chuoiNhap, out double ketQua))
        {
            return ketQua;
        }

        Console.Write($"Loi: \"{chuoiNhap}\" ");
        Console.WriteLine("khong hop le. Nhap lai!");
    }
}

// In dung dinh dang menu de bai yeu cau
static void HienThiMenu()
{
    Console.WriteLine();
    Console.WriteLine("MENU");
    Console.WriteLine("1. Nhap hai gia tri so thuc cho x, y");
    Console.WriteLine("2. Tinh x^y");
    Console.WriteLine("3. Tinh can bac 2 cua x va y");
    Console.WriteLine("4. Thoat");
    Console.Write("Chon chuc nang: ");
}

static void Main()
{
    double x = 0, y = 0;
```

```
// Co nhac neu chua chon 1 ma da chon 2 hoac 3
bool daNhap = false;

while (true)
{
    HienThiMenu();
    string luaChon = Console.ReadLine();

    switch (luaChon)
    {
        case "1":
            x = NhapSoThuc("Nhap x: ");
            y = NhapSoThuc("Nhap y: ");
            daNhap = true;
            break;

        case "2":
            if (!daNhap)
            {
                Console.WriteLine("Ban chua nhap x, y. Chon 1 truoc!");
                break;
            }
            double ketQuaLuyThua = MathTools.TinhLuyThua(x, y);
            Console.WriteLine($"Ket qua {x} mu {y} la: {ketQuaLuyThua}");
            break;

        case "3":
            if (!daNhap)
            {
                Console.WriteLine("Ban chua nhap x, y. Chon 1 truoc!");
```

```
                    break;
                }
                double? canX = MathTools.TinhCanBac2(x);
                double? canY = MathTools.TinhCanBac2(y);

                if (canX == null)
                    Console.WriteLine($"x = {x} la so am, khong co can bac 2");
                else
                    Console.WriteLine($"Can bac 2 cua x la: {canX}");

                if (canY == null)
                    Console.WriteLine($"y = {y} la so am, khong co can bac 2");
                else
                    Console.WriteLine($"Can bac 2 cua y la: {canY}");
                break;

            case "4":
                Console.WriteLine("Tam biet!");
                return;

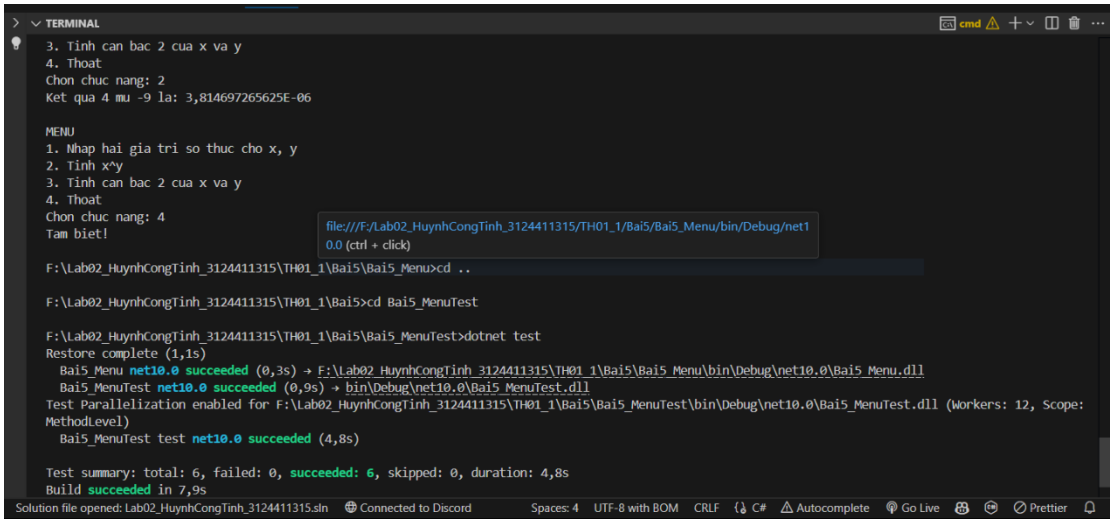
            default:
                Console.WriteLine("Lua chon khong hop le!");
                break;
        }
    }
}
```

**4. Test case (Input / Output)**

Vì đây là chương trình dạng menu chạy nhiều vòng liên tiếp, test case được trình bày theo một kịch bản chạy thử liền mạch (mỗi bước là một lượt người dùng gõ vào), thay vì các bộ input độc lập như Bài 3/4.

| Bước | Người dùng nhập | Output chương trình in ra | Ghi chú |
|---|---|---|---|
| 1 | 2 | Ban chua nhap x, y. Chon 1 truoc! | Chọn tính toán trước khi nhập x, y → chương trình nhắc, không tính bừa |
| 2 | 1<br>x = 4<br>y = -9 | (không có, chỉ lưu x, y) | Chọn mục 1, nhập x = 4, y = -9 (y âm để test căn bậc 2 sau) |
| 3 | 2 | Ket qua 4 mu -9 la: 3.8147E-06 | Tính 4^-9 — số mũ âm vẫn tính đúng nhờ Math.Pow |
| 4 | 3 | Can bac 2 cua x la: 2<br>y = -9 la so am, khong co can bac 2 | x = 4 tính được, y = -9 báo lỗi thay vì hiện NaN |
| 5 | abc | Lua chon khong hop le! | Gõ lựa chọn không phải 1–4 → báo lỗi, không crash |
| 6 | 4 | Tam biet! | Chọn thoát → chương trình kết thúc |

**Kết quả sau khi thực hiện:**




## Viết phương thức: truyền tham số bằng tham trị, tham chiếu: ref, out

## * Bài 6. return giá trị

**Viết phương thức: truyền tham số bằng tham trị, tham chiếu: ref, out**

**Bài 6. return giá trị**

Xây dựng lớp có phương thức tìm giá trị lớn nhất của ba số nguyên.

### 1. Ý tưởng giải bài toán

Tách riêng một lớp BaSoNguyen chứa phương thức TimGiaTriLonNhat(a, b, c) — đây là phương thức thể hiện (instance method), nên trước khi gọi phải tạo đối tượng bằng new BaSoNguyen(), khác với các bài trước dùng static.

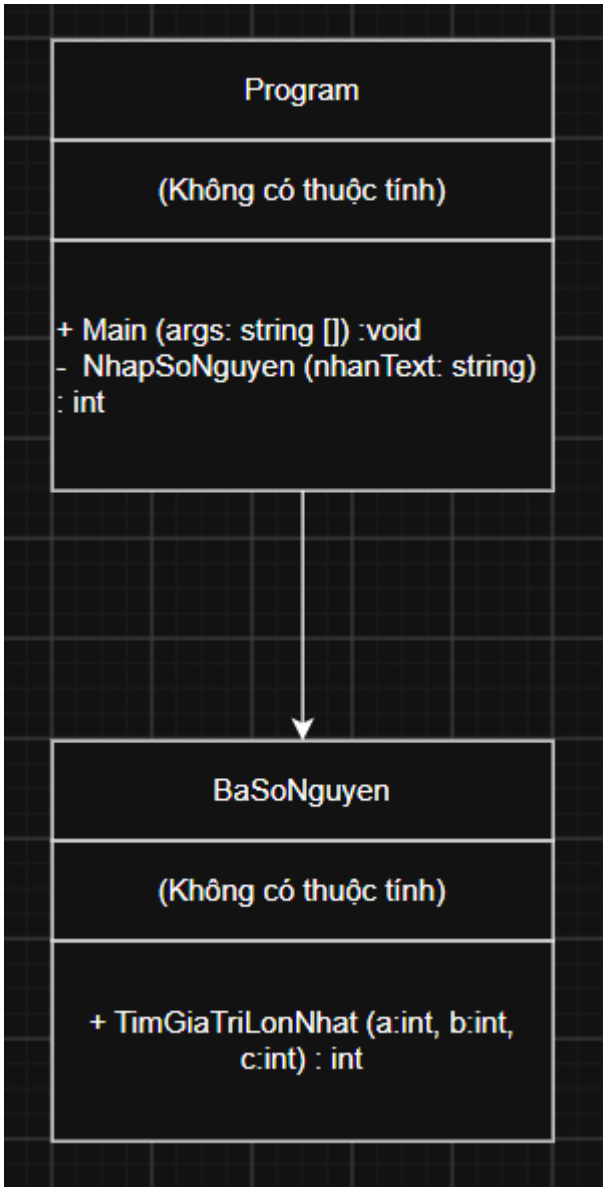
- Bước 1: Trong Main, nhập 3 số nguyên a, b, c (dùng lại hàm NhapSoNguyen có bắt lỗi TryParse như Bài 4).
- Bước 2: Tạo một đối tượng BaSoNguyen, rồi gọi phương thức TimGiaTriLonNhat(a, b, c) trên đối tượng đó.
- Bước 3: Bên trong phương thức, giả sử a là số lớn nhất, sau đó lần lượt so sánh với b rồi với c, số nào lớn hơn thì cập nhật lại biến max.
- Bước 4: Dùng return max; để trả kết quả ra ngoài — đây là điểm mấu chốt của bài: a, b, c là tham số truyền theo tham trị, phương thức chỉ nhận được BẢN SAO giá trị của chúng, nên muốn lấy kết quả tính toán

ra ngoài bắt buộc phải dùng return (không thể sửa thẳng vào a, b, c như ref/out).

Vì truyền theo tham trị, dù bên trong phương thức có lỡ gán lại giá trị cho tham số a, b hay c thì các biến gốc a, b, c ở Main vẫn không hề thay đổi sau khi gọi hàm xong — điều này được kiểm chứng cụ thể ở phần test case bên dưới.

## 2. Sơ đồ lớp (Class Diagram)

Chương trình gồm 2 lớp: Program lo nhập liệu và hiển thị kết quả;
BaSoNguyen chứa đúng 1 phương thức nghiệp vụ là tìm giá trị lớn nhất trong 3 số nguyên.




*Quan hệ giữa hai lớp: Program sử dụng (dependency) BaSoNguyen — cụ thể là tạo một đối tượng BaSoNguyen rồi gọi phương thức TimGiaTriLonNhat() của đối tượng đó để lấy kết quả.*

**3. Code chương trình**

File: BaSoNguyen.cs

```
using System;

class BaSoNguyen
```

```
{
    // a, b, c la tham so truyen theo THAM TRI (mac dinh cua int):
    // ham chi nhan ban sao gia tri, khong lam thay doi bien goc ben ngoai.
    // Ket qua duoc lay ra bang return (khac voi ref/out se hoc o bai sau).
    public int TimGiaTriLonNhat(int a, int b, int c)
    {
        int max = a;

        if (b > max)
        {
            max = b;
        }

        if (c > max)
        {
            max = c;
        }

        return max;
    }
}
```

File: Program.cs

```
using System;

class Program
{
    // Nhap 1 so nguyen, bat loi va cho nhap lai neu sai (giong bai 4)
```

```
    static int NhapSoNguyen(string nhanText)
    {
        while (true)
        {
            Console.Write(nhanText);
            string chuoiNhap = Console.ReadLine();

            if (int.TryParse(chuoiNhap, out int ketQua))
            {
                return ketQua;
            }

            Console.Write($"Loi: \"{chuoiNhap}\" ");
            Console.WriteLine("khong hop le. Nhap lai!");
        }
    }

    static void Main(string[] args)
    {
        int a = NhapSoNguyen("Nhap so a: ");
        int b = NhapSoNguyen("Nhap so b: ");
        int c = NhapSoNguyen("Nhap so c: ");

        // Tao doi tuong tu lop BaSoNguyen roi goi phuong thuc cua no
        BaSoNguyen baSo = new BaSoNguyen();
        int max = baSo.TimGiaTriLonNhat(a, b, c);

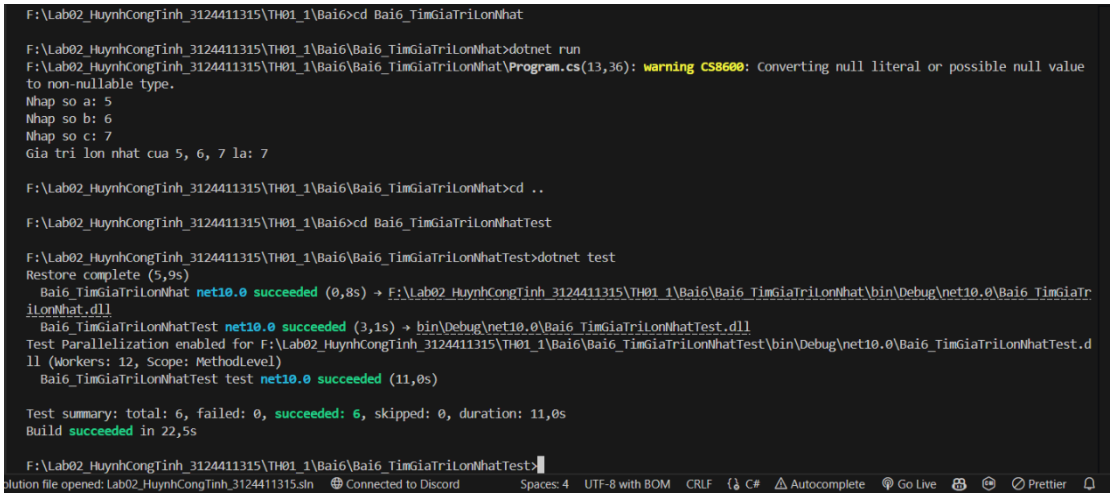
        Console.WriteLine($"Gia tri lon nhat cua {a}, {b}, {c} la: {max}");
    }
}
```

**4. Test case (Input / Output)**

Test case dưới đây phủ các trường hợp: mỗi số a, b, c lần lượt là số lớn nhất, trường hợp 3 số bằng nhau, trường hợp cả 3 số âm, và một test riêng để minh chứng đặc điểm truyền tham trị — sau khi gọi phương thức, các biến gốc a, b, c không hề bị thay đổi.

| STT | a, b, c (input) | Output chương trình in ra | Ghi chú |
|---|---|---|---|
| 1 | a=10, b=3, c=5 | Gia tri lon nhat cua 10, 3, 5 la: 10 | a là số lớn nhất |
| 2 | a=2, b=9, c=5 | Gia tri lon nhat cua 2, 9, 5 la: 9 | b là số lớn nhất |
| 3 | a=2, b=3, c=7 | Gia tri lon nhat cua 2, 3, 7 la: 7 | c là số lớn nhất |
| 4 | a=5, b=5, c=5 | Gia tri lon nhat cua 5, 5, 5 la: 5 | 3 số bằng nhau, kết quả vẫn đúng |
| 5 | a=-10, b=-3, c=-7 | Gia tri lon nhat cua -10, -3, -7 la: -3 | Cả 3 số âm, số "gần 0" nhất là lớn nhất |
| 6 | a=10, b=3, c=5 (kiểm tra sau khi gọi hàm) | a = 10, b = 3, c = 5 (không đổi) | Minh chứng truyền tham trị: gọi TimGiaTriLonNhat() xong, a, b, c gốc ở Main vẫn giữ nguyên |

**Kết quả sau khi thực hiện:**




## Bài 7. Phương thức bool

**Bài 7. Phương thức bool**

**Ngôn ngữ lập trình C#**

Xây dựng lớp có phương thức kiểm tra n có phải là số nguyên tố hay không.

### 1. Ý tưởng giải bài toán

Số nguyên tố là số tự nhiên lớn hơn 1, chỉ chia hết cho 1 và chính nó. Từ định nghĩa này, cách kiểm tra đơn giản nhất là thử chia n cho tất cả các số từ 2 trở đi; nếu tìm được một ước nào đó thì chắc chắn không phải số nguyên tố.

- Bước 1: Xử lý biên trước — nếu n <= 1 (gồm số 0, số 1, số âm) thì trả về false ngay vì không nằm trong định nghĩa số nguyên tố.
- Bước 2: Xử lý riêng n == 2 (số nguyên tố chẵn duy nhất) → trả về true; sau đó nếu n là số chẵn khác thì chắc chắn trả về false, không cần kiểm tra tiếp.
- Bước 3: Với các số lẻ còn lại, chỉ cần thử chia cho các số lẻ từ 3 đến căn bậc hai của n (Math.Sqrt(n)) — đây là điểm tối ưu quan

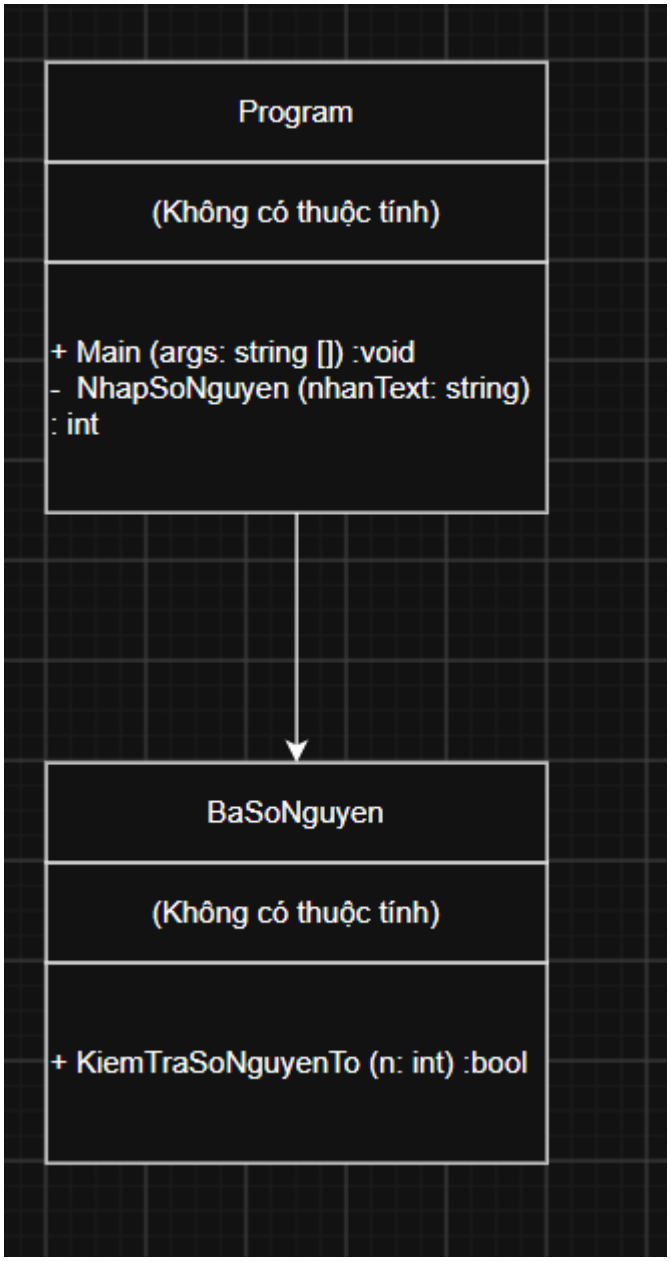
trọng: nếu n có một ước a lớn hơn căn bậc hai của n, thì n cũng phải có một ước b = n / a nhỏ hơn căn bậc hai của n, nên chỉ cần thử đến căn bậc hai là đủ, không bỏ sót trường hợp nào.

- Bước 4: Trong vòng lặp, hễ tìm thấy một số i mà n % i == 0 thì trả về false ngay lập tức (không cần chạy hết vòng lặp); nếu chạy hết vòng lặp mà không tìm thấy ước nào thì trả về true.

Vì phương thức trả về bool, phần Main chỉ cần gọi phương thức rồi dùng if để quyết định in thông báo phù hợp — tách bạch rõ ràng giữa phần "tính toán/kiểm tra logic" (trả bool) và phần "hiển thị kết quả" (in ra màn hình).

**2. Sơ đồ lớp (Class Diagram)**

Chương trình gồm 2 lớp: Program lo nhập liệu và hiển thị kết quả; SoNguyenTo chứa đúng 1 phương thức nghiệp vụ, trả về kiểu bool theo đúng yêu cầu đề bài.




*Quan hệ giữa hai lớp: Program sử dụng (dependency) SoNguyenTo — tạo một đối tượng SoNguyenTo rồi gọi phương thức KiemTraSoNguyenTo() để lấy kết quả bool, sau đó tự quyết định thông báo hiển thị.*

### 3. Code chương trình

File: SoNguyenTo.cs

```
using System;
```

```
class SoNguyenTo
{
    // Phuong thuc tra ve bool: true neu n la so nguyen to
    public bool KiemTraSoNguyenTo(int n)
    {
        // So nguyen to phai lon hon 1
        if (n <= 1)
        {
            return false;
        }

        // 2 la so nguyen to nho nhat, xu ly rieng cho gon
        if (n == 2)
        {
            return true;
        }

        // So chan lon hon 2 chac chan khong phai so nguyen to
        if (n % 2 == 0)
        {
            return false;
        }

        // Chi can thu cac uoc le tu 3 den can bac hai cua n
        int gioiHan = (int)Math.Sqrt(n);

        for (int i = 3; i <= gioiHan; i += 2)
```

```
        {
            if (n % i == 0)
            {
                return false;
            }
        }

        return true;
    }
}
```

File: Program.cs

```
using System;

class Program
{
    // Nhap 1 so nguyen, bat loi va cho nhap lai neu sai (giong bai 4, 6)
    static int NhapSoNguyen(string nhanText)
    {
        while (true)
        {
            Console.Write(nhanText);
            string? chuoiNhap = Console.ReadLine();

            if (int.TryParse(chuoiNhap, out int ketQua))
            {
```

```
            return ketQua;
        }

        Console.Write($"Loi: \"{chuoiNhap}\" ");
        Console.WriteLine("khong hop le. Nhap lai!");
      }
    }

    static void Main(string[] args)
    {
      int n = NhapSoNguyen("Nhap so n: ");

      SoNguyenTo kiemTra = new SoNguyenTo();
      bool ketQua = kiemTra.KiemTraSoNguyenTo(n);

      if (ketQua)
      {
        Console.WriteLine($"{n} la so nguyen to");
      }
      else
      {
        Console.WriteLine($"{n} khong phai la so nguyen to");
      }
    }
}
```

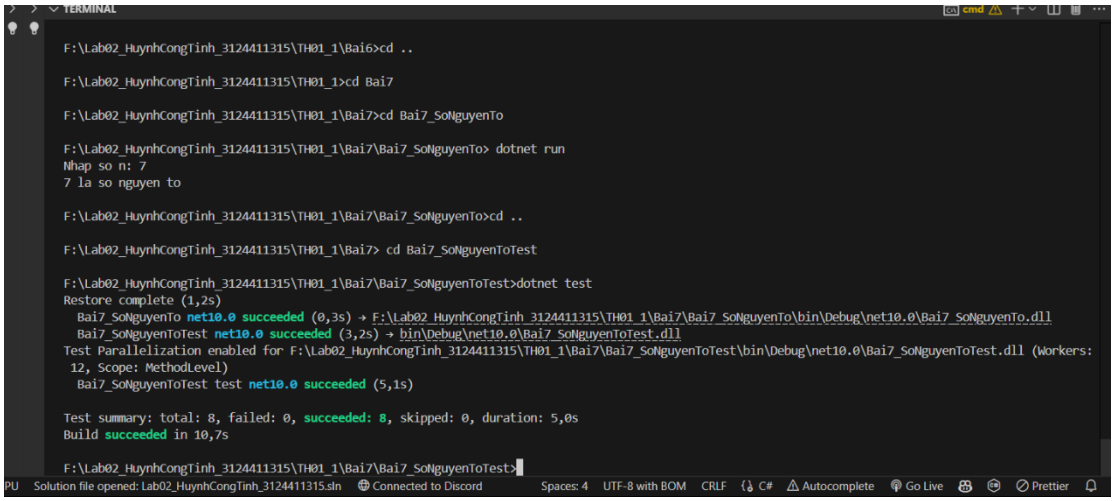
**4. Test case (Input / Output)**

Test case dưới đây phủ đầy đủ các trường hợp biên đặc trưng của bài toán số nguyên tố: số nguyên tố nhỏ nhất, số nguyên tố lẻ, số nguyên tố khá lớn (kiểm tra vòng lặp căn bậc hai), số chẵn, hợp số lẻ, và các giá trị biên 0, 1, số âm.

| STT | n (input) | Output chương trình in ra | Ghi chú |
|---|---|---|---|
| 1 | 2 | 2 la so nguyen to | Số nguyên tố nhỏ nhất, cũng là số chẵn duy nhất |
| 2 | 7 | 7 la so nguyen to | Số nguyên tố lẻ thông thường |
| 3 | 97 | 97 la so nguyen to | Số nguyên tố khá lớn, kiểm tra vòng lặp chạy đến căn(97) ≈ 9.8 |
| 4 | 8 | 8 khong phai la so nguyen to | Số chẵn > 2 → loại ngay, không cần chạy vòng lặp |
| 5 | 9 | 9 khong phai la so nguyen to | Hợp số lẻ (9 = 3×3) → phải chạy vòng lặp mới phát hiện được |
| 6 | 1 | 1 khong phai la so nguyen to | Trường hợp biên: 1 không nằm trong định nghĩa số nguyên tố |
| 7 | 0 | 0 khong phai la so nguyen to | Trường hợp biên: 0 bị loại ngay ở điều kiện n <= 1 |

| STT | n (input) | Output chương trình in ra | Ghi chú |
|---|---|---|---|
| 8 | -7 | -7 khong phai la so nguyen to | Số âm bị loại ngay ở điều kiện n <= 1 |

**Kết quả sau khi thực hiện:**




## * Bài 8. Tham chiếu ref

**Bài 8. Tham chiếu ref**

Xây dựng lớp có phương thức hoán vị hai số thực.

### 1. Ý tưởng giải bài toán

Tách riêng một lớp HoanVi chứa phương thức ThucHienHoanVi(ref double a, ref double b) — đây là phương thức thể hiện (instance method), nên trước khi gọi phải tạo đối tượng bằng new HoanVi(), giống Bài 6, 7.

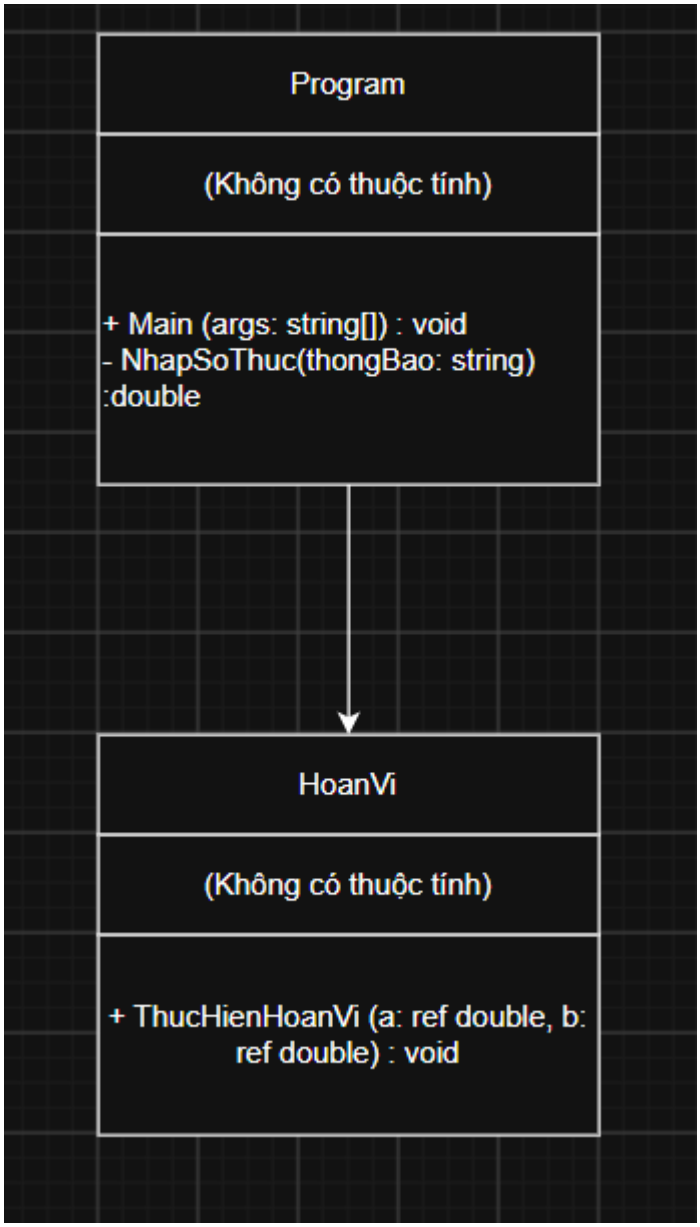
- Bước 1: Trong Main, nhập 2 số thực a, b (dùng hàm NhapSoThuc có bắt lỗi bằng double.TryParse, tương tự NhapSoNguyen ở các bài trước).
- Bước 2: Tạo một đối tượng HoanVi, rồi gọi phương thức ThucHienHoanVi(ref a, ref b) trên đối tượng đó, truyền thẳng 2 biến a, b của Main vào theo tham chiếu.

- Bước 3: Bên trong phương thức, dùng một biến tạm "tam" lưu giữ giá trị của a trước khi ghi đè, sau đó gán a = b rồi gán b = tam — đây là thuật toán hoán vị kinh điển bằng biến trung gian.
- Bước 4: Vì a, b là tham số truyền theo tham chiếu (ref), mọi thay đổi bên trong phương thức tác động thẳng lên chính biến gốc a, b khai báo trong Main, nên phương thức khai báo kiểu trả về void, không cần return như Bài 6.

Khác với Bài 6 (tham trị + return chỉ lấy ra được đúng 1 giá trị), ref cho phép một phương thức thay đổi cùng lúc nhiều biến ở nơi gọi — đặc điểm này được kiểm chứng cụ thể ở phần test case bên dưới, đặc biệt là test case so sánh trực tiếp với một hàm dùng tham trị.

**2. Sơ đồ lớp (Class Diagram)**

Chương trình gồm 2 lớp: Program lo nhập liệu và hiển thị kết quả; HoanVi chứa đúng 1 phương thức nghiệp vụ là hoán vị hai số thực bằng tham chiếu.




*Quan hệ giữa hai lớp: Program sử dụng (dependency) HoanVi — cụ thể là tạo một đối tượng HoanVi rồi gọi phương thức ThucHienHoanVi() của đối tượng đó để hoán vị 2 biến a, b khai báo trong Main.*

**3. Code chương trình**

File: HoanVi.cs

```
using System;

namespace Bai8_HoanVi
```

```
{
    // Lop xu ly nghiep vu hoan vi hai so thuc, khong dinh Console
    public class HoanVi
    {
        // Dung ref vi can thay doi truc tiep bien goc ben ngoai (tham chieu),
        // khac voi tham tri se khong lam thay doi gia tri sau khi ham ket thuc
        public void ThucHienHoanVi(ref double a, ref double b)
        {
            double tam = a;
            a = b;
            b = tam;
        }
    }
}
```

File: Program.cs

```
using System;

namespace Bai8_HoanVi
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== BAI 8: HOAN VI HAI SO THUC (THAM CHIEU
REF) ===");

            double soThu1 = NhapSoThuc("Nhap so thuc thu nhat: ");
```

```
        double soThu2 = NhapSoThuc("Nhap so thuc thu hai: ");

        Console.WriteLine($"\nTruoc khi hoan vi: a = {soThu1}, b = {soThu2}");

        HoanVi doiTuong = new HoanVi();
        doiTuong.ThucHienHoanVi(ref soThu1, ref soThu2);

        Console.WriteLine($"Sau khi hoan vi:  a = {soThu1}, b = {soThu2}");
    }

    // Nhap so thuc, dung TryParse + while(true) de bat loi, khong dung Parse tran tranh crash
    static double NhapSoThuc(string thongBao)
    {
        double ketQua;
        while (true)
        {
            Console.Write(thongBao);
            string? dongNhap = Console.ReadLine();

            if (double.TryParse(dongNhap, out ketQua))
            {
                return ketQua;
            }

            Console.WriteLine("Loi: vui long nhap dung dinh dang so thuc (vi du 3.14). Nhap lai!");
        }
    }
}
```

```
}
```

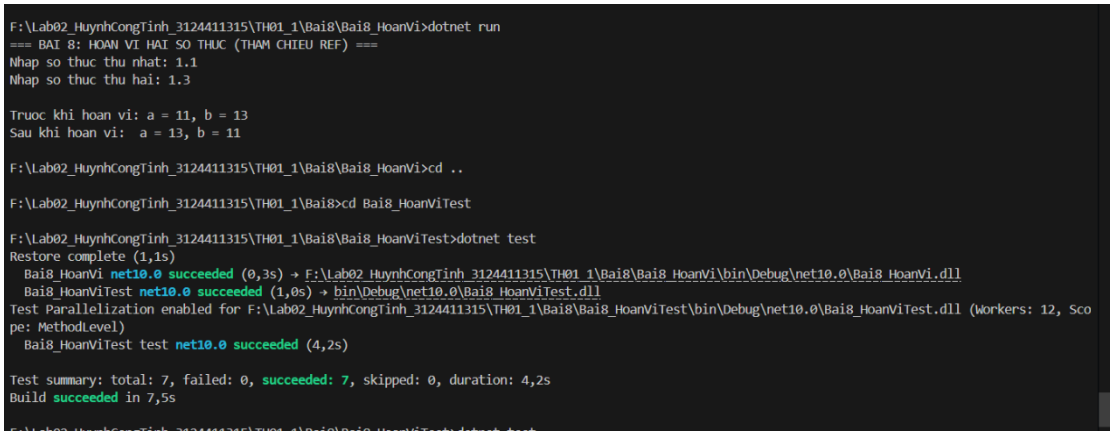
**4. Test case (Input / Output)**

Test case dưới đây phủ các trường hợp: hoán vị bình thường, hai số âm, một âm một dương, trường hợp biên (hai số bằng nhau, một số bằng 0), hoán vị 2 lần liên tiếp, và một test riêng để minh chứng đặc điểm truyền tham chiếu — khác với tham trị, ref làm thay đổi trực tiếp biến gốc a, b ở Main.

| STT | a, b (input) | Output chương trình in ra | Ghi chú |
|---|---|---|---|
| 1 | a = 3.5, b = 7.2 | Truoc khi hoan vi: a = 3.5, b = 7.2<br>Sau khi hoan vi: a = 7.2, b = 3.5 | Trường hợp bình thường |
| 2 | a = -5.5, b = -1.1 | Truoc khi hoan vi: a = -5.5, b = -1.1<br>Sau khi hoan vi: a = -1.1, b = -5.5 | Hai số đều âm |
| 3 | a = -4.2, b = 8.8 | Truoc khi hoan vi: a = -4.2, b = 8.8<br>Sau khi hoan vi: a = 8.8, b = -4.2 | Một số âm, một số dương |
| 4 | a = 5.0, b = 5.0 | Truoc khi hoan vi: a = 5, b = 5<br>Sau khi hoan vi: | Trường hợp biên: hai số bằng nhau, hoán vị xong giá trị không đổi |

| STT | a, b (input) | Output chương trình in ra | Ghi chú |
|---|---|---|---|
| | | a = 5, b = 5 | |
| 5 | a = 0.0, b = 9.9 | Truoc khi hoan vi: a = 0, b = 9.9<br>Sau khi hoan vi: a = 9.9, b = 0 | Trường hợp biên: một trong hai số bằng 0 |
| 6 | a = 12.34, b = 56.78 (gọi ThucHienHoanVi 2 lần liên tiếp) | a = 12.34, b = 56.78 (trở về đúng giá trị ban đầu) | Minh chứng hoán vị 2 lần liên tiếp cho kết quả đúng như chưa hoán vị |
| 7 | a = 1.1, b = 2.2 (kiểm tra sau khi gọi hàm) | Goi ham tham tri: a, b khong doi<br>Goi ham ref: a = 2.2, b = 1.1 (da doi cho) | Minh chứng khác biệt giữa tham chiếu (ref) và tham trị: chỉ ref mới làm thay đổi biến gốc bên ngoài |

**Kết quả sau khi thực hiện:**



## * Bài 9. Tham chiếu out

**Bài 9. Tham chiếu out**

Xây dựng lớp có phương thức tìm giá trị lớn nhất và giá trị nhỏ nhất của ba số thực.

### 1. Ý tưởng giải bài toán

Tách riêng một lớp TimMaxMin chứa phương thức TimGiaTriLonNhoNhat(a, b, c, out gtLon, out gtNho) — đây là phương thức thể hiện (instance method), nên trước khi gọi phải tạo đối tượng bằng new TimMaxMin(), giống Bài 6, 7, 8.

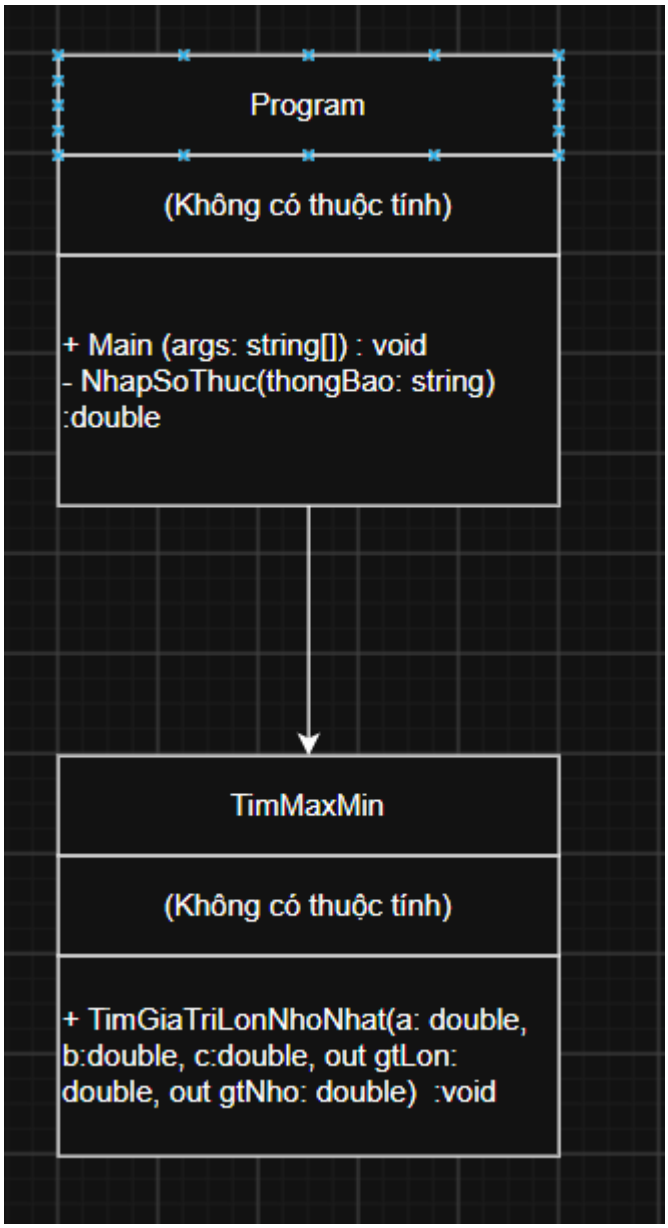
- Bước 1: Trong Main, nhập 3 số thực a, b, c (dùng hàm NhapSoThuc có bắt lỗi bằng double.TryParse, tương tự các bài trước).
- Bước 2: Tạo một đối tượng TimMaxMin, rồi gọi phương thức TimGiaTriLonNhoNhat(a, b, c, out gtLon, out gtNho) — 2 biến gtLon, gtNho có thể khai báo ngay tại chỗ gọi (out double gtLon) mà KHÔNG cần gán giá trị khởi tạo trước, khác hẳn với ref ở Bài 8 (ref bắt buộc biến phải có giá trị trước khi truyền vào).
- Bước 3: Bên trong phương thức, giả sử a vừa là lớn nhất vừa là nhỏ nhất, sau đó lần lượt so sánh với b rồi với c để cập nhật lại gtLon và gtNho.
- Bước 4: Trước khi phương thức kết thúc, bắt buộc phải gán giá trị cho cả gtLon lẫn gtNho (trình biên dịch báo lỗi CS0177 nếu thiếu) — đây chính là ràng buộc riêng của out, đảm bảo nơi gọi hàm luôn nhận được kết quả hợp lệ ở cả hai biến.

Nhờ out, một phương thức có thể trả về nhiều kết quả cùng lúc mà không cần gộp chúng vào một kiểu dữ liệu phức tạp (như tuple hay class riêng) — đặc

điểm này, cùng với sự khác biệt so với ref về việc khởi tạo giá trị ban đầu, được kiểm chứng cụ thể ở phần test case bên dưới.

## 2. Sơ đồ lớp (Class Diagram)

Chương trình gồm 2 lớp: Program lo nhập liệu và hiển thị kết quả; TimMaxMin chứa đúng 1 phương thức nghiệp vụ là tìm giá trị lớn nhất và nhỏ nhất của ba số thực bằng tham chiếu out.



*Quan hệ giữa hai lớp: Program sử dụng (dependency) TimMaxMin — cụ thể là tạo một đối tượng TimMaxMin rồi gọi phương thức TimGiaTriLonNhoNhat() của đối tượng đó để lấy ra 2 giá trị gtLon, gtNho cùng lúc.*

**3. Code chương trình**

File: TimMaxMin.cs

```
using System;

namespace Bai9_TimMaxMin
{
   // Lop xu ly nghiep vu tim gia tri lon nhat va nho nhat cua 3 so thuc, khong dinh Console
   public class TimMaxMin
   {
      // Dung out vi can tra ra CUNG LUC 2 ket qua (lon nhat va nho nhat).
      // Khac voi ref: bien truyen vao out KHONG can khoi tao gia tri truoc do,
      // nhung ben trong phuong thuc BAT BUOC phai gan gia tri cho moi bien out
      // truoc khi ham ket thuc (trinh bien dich se bao loi CS0177 neu thieu).
      public void TimGiaTriLonNhoNhat(double a, double b, double c, out double gtLon, out double gtNho)
      {
         // Gia su a vua la lon nhat vua la nho nhat, roi so sanh dan voi b, c
         gtLon = a;
         gtNho = a;

         if (b > gtLon) gtLon = b;
         if (b < gtNho) gtNho = b;

         if (c > gtLon) gtLon = c;
         if (c < gtNho) gtNho = c;
```

```
        }
    }
}
```

File: Program.cs

```
using System;

namespace Bai9_TimMaxMin
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== BAI 9: TIM GTLN & GTNN CUA BA SO THUC (THAM CHIEU OUT) ===");

            double soA = NhapSoThuc("Nhap so thuc a: ");
            double soB = NhapSoThuc("Nhap so thuc b: ");
            double soC = NhapSoThuc("Nhap so thuc c: ");

            TimMaxMin doiTuong = new TimMaxMin();

            // Khong can khoi tao gtLon, gtNho truoc khi truyen vao tham so out
            doiTuong.TimGiaTriLonNhoNhat(soA, soB, soC, out double gtLon, out double gtNho);

            Console.WriteLine($"Gia tri lon nhat cua {soA}, {soB}, {soC} la: {gtLon}");
            Console.WriteLine($"Gia tri nho nhat cua {soA}, {soB}, {soC} la: {gtNho}");
```

```
    }

    // Nhap so thuc, dung TryParse + while(true) de bat loi, khong dung Parse tran tranh crash
    static double NhapSoThuc(string thongBao)
    {
        double ketQua;
        while (true)
        {
            Console.Write(thongBao);
            string? dongNhap = Console.ReadLine();

            if (double.TryParse(dongNhap, out ketQua))
            {
                return ketQua;
            }

            Console.WriteLine("Loi: vui long nhap dung dinh dang so thuc (vi du 3.14). Nhap lai!");
        }
    }
  }
}
```

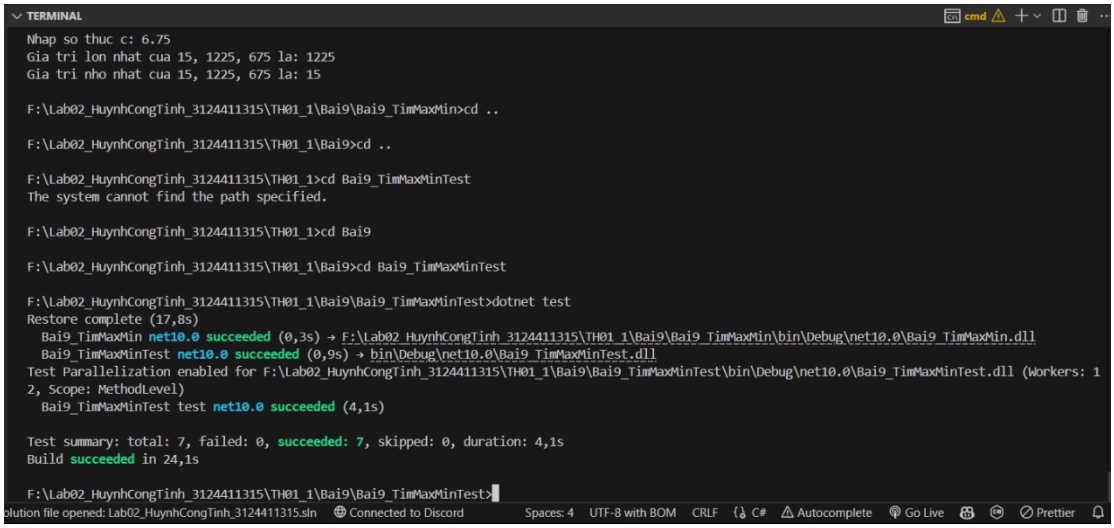
**4. Test case (Input / Output)**

Test case dưới đây phủ các trường hợp: tìm max/min bình thường, có số âm, cả ba số âm, trường hợp biên (ba số bằng nhau, có số bằng 0, hai số bằng nhau là cực trị), và một test riêng để minh chứng đặc điểm truyền tham chiếu out — biến truyền vào không cần khởi tạo giá trị trước, khác với ref.

| STT | a, b, c (input) | Output chương trình in ra | Ghi chú |
|---|---|---|---|
| 1 | a=10.5, b=3.2, c=7.8 | Gia tri lon nhat cua 10.5, 3.2, 7.8 la: 10.5<br>Gia tri nho nhat cua 10.5, 3.2, 7.8 la: 3.2 | Trường hợp bình thường, a là lớn nhất |
| 2 | a=-2.5, b=6.6, c=-9.1 | Gia tri lon nhat cua -2.5, 6.6, -9.1 la: 6.6<br>Gia tri nho nhat cua -2.5, 6.6, -9.1 la: -9.1 | Hai số âm, một số dương |
| 3 | a=-10.0, b=-3.5, c=-7.2 | Gia tri lon nhat cua -10, -3.5, -7.2 la: -3.5<br>Gia tri nho nhat cua -10, -3.5, -7.2 la: -10 | Cả ba số đều âm |
| 4 | a=4.0, b=4.0, c=4.0 | Gia tri lon nhat cua 4, 4, 4 la: 4<br>Gia tri nho nhat cua 4, 4, 4 la: 4 | Trường hợp biên: ba số bằng nhau, max = min |
| 5 | a=0.0, b=-5.5, c=8.1 | Gia tri lon nhat cua 0, -5.5, 8.1 la: 8.1<br>Gia tri nho nhat | Trường hợp biên: có một số bằng 0 |

| STT | a, b, c (input) | Output chương trình in ra | Ghi chú |
|---|---|---|---|
| | | cua 0, -5.5, 8.1<br>la: -5.5 | |
| 6 | a=9.0, b=9.0, c=2.0 | Gia tri lon nhat cua 9, 9, 2 la: 9<br>Gia tri nho nhat cua 9, 9, 2 la: 2 | Hai trong ba số bằng nhau và là giá trị lớn nhất |
| 7 | double gtLon, gtNho; (khong gan gia tri ban dau)<br>a=1.5, b=12.25, c=6.75 | Sau khi goi ham: gtLon = 12.25, gtNho = 1.5 | Minh chứng đặc điểm riêng của out: biến truyền vào không cần khởi tạo trước (khác ref), vì hàm bắt buộc phải gán giá trị cho tham số out trước khi kết thúc |

## Kết quả sau khi thực hiện:



## Chuỗi (string, StringBuilder)

**Bài 10:** Viết phương thức thành viên kiểm tra chuỗi có đối xứng hay không

### 1. Ý tưởng giải bài toán

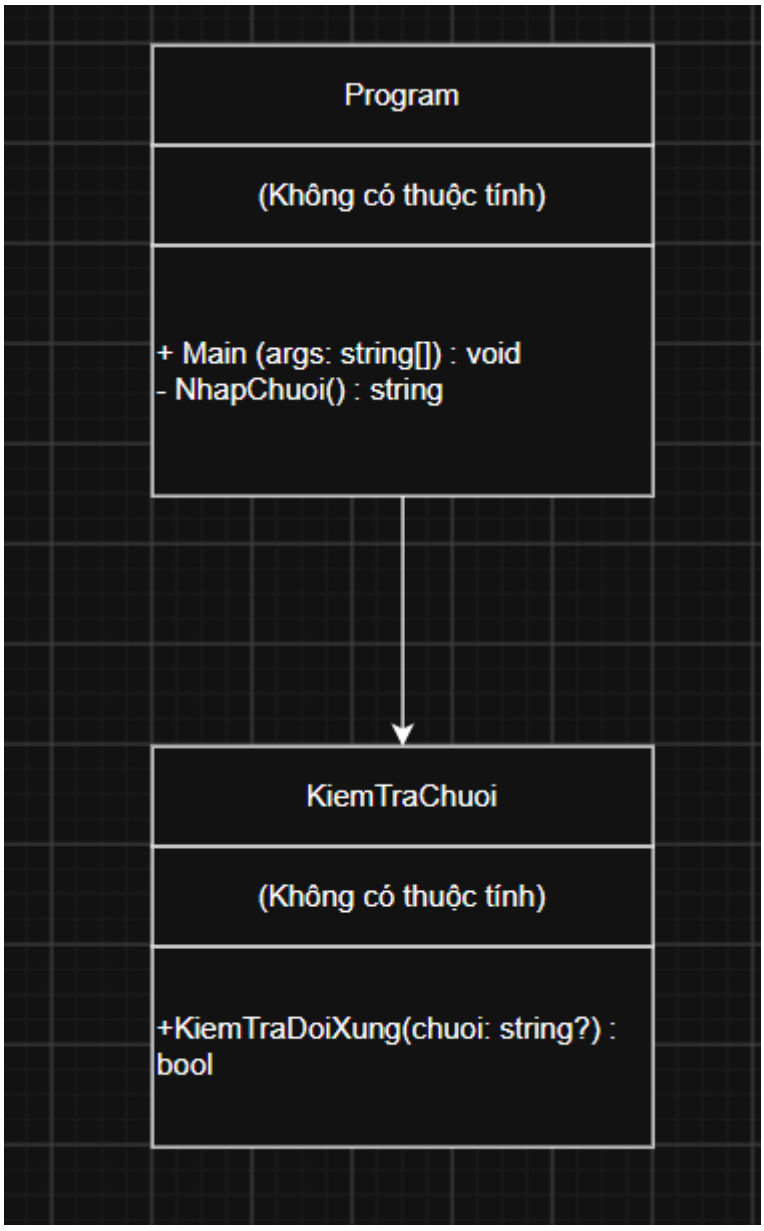
Tách riêng lớp KiemTraChuoi chứa phương thức KiemTraDoiXung(chuoi). Đề yêu cầu phương thức thành viên nên đây là phương thức thể hiện (instance method, không có static), muốn gọi phải tạo đối tượng bằng new KiemTraChuoi(). Lớp này không đụng đến Console, phần nhập xuất để Program lo.

- **Bước 1:** Trong Main, nhập chuỗi bằng hàm NhapChuoi(). Hàm này lặp while (true), nếu người dùng chỉ bấm Enter hoặc nhập toàn khoảng trắng thì báo lỗi và cho nhập lại.
- **Bước 2:** Tạo đối tượng KiemTraChuoi rồi gọi kiemTra.KiemTraDoiXung(chuoi) để lấy kết quả kiểu bool.
- **Bước 3:** Trong phương thức, nếu chuỗi là null thì trả về false luôn. Sau đó đặt 2 con trỏ: trai = 0 (đầu chuỗi) và phai = chuoi.Length - 1 (cuối chuỗi).
- **Bước 4:** Lặp khi trai < phai. Nếu ký tự ở trai hoặc phai là khoảng trắng thì nhảy qua ký tự đó. Ngược lại đổi cả hai về chữ thường (ToLowerInvariant) rồi so sánh, khác nhau thì return false ngay, giống nhau thì trai++, phai--.
- **Bước 5:** Ra khỏi vòng lặp mà không có cặp nào bị lệch thì return true. Chuỗi rỗng và chuỗi 1 ký tự không vào vòng lặp nên cũng trả về true.

Cách khác là đảo ngược chuỗi rồi so sánh với chuỗi ban đầu, nhưng như vậy phải tạo thêm một chuỗi mới và còn phải xử lý khoảng trắng riêng. Dùng 2 con trỏ thì chỉ duyệt tối đa một nửa chuỗi, không tốn thêm bộ nhớ, và cặp nào lệch thì dừng ngay tại đó.

**2. Sơ đồ lớp (Class Diagram)**

Chương trình gồm 2 lớp: Program lo nhập chuỗi và in kết quả; KiemTraChuoi chứa phương thức kiểm tra chuỗi đối xứng.




*Quan hệ giữa hai lớp: Program phụ thuộc (dependency) vào KiemTraChuoi, cụ thể là tạo một đối tượng KiemTraChuoi rồi gọi phương thức KiemTraDoiXung() của đối tượng đó.*

**3. Code chương trình**

File: KiemTraChuoi.cs

```
namespace Bai10_KiemTraDoiXung
{
   public class KiemTraChuoi
   {
      // Kiem tra chuoi co doi xung hay khong (doc xuoi va doc nguoc giong nhau).
      // Quy uoc: bo qua khoang trang, khong phan biet hoa thuong.
      // Dung 2 con tro trai/phai thay vi dao nguoc chuoi de khong phai tao them chuoi moi.
      public bool KiemTraDoiXung(string? chuoi)
      {
         // null khong phai chuoi hop le -> false. Chuoi rong "" thi khong vao vong lap
         // ben duoi nen tra ve true (chuoi rong doc xuoi hay nguoc deu nhu nhau)
         if (chuoi == null)
            return false;

         int trai = 0;                  // con tro dau chuoi
         int phai = chuoi.Length - 1;   // con tro cuoi chuoi

         // dung khi 2 con tro gap nhau (chuoi le) hoac di qua nhau (chuoi chan)
         while (trai < phai)
         {
            // gap khoang trang o ben nao thi bo qua ben do, roi xet lai tu dau vong lap
            if (char.IsWhiteSpace(chuoi[trai]))
            {
               trai++;
               continue;
            }
```

```
            if (char.IsWhiteSpace(chuoi[phai]))
            {
                phai--;
                continue;
            }

            // ToLowerInvariant de "A" == "a", khong phu thuoc cai dat ngon ngu cua may
            if (char.ToLowerInvariant(chuoi[trai]) != char.ToLowerInvariant(chuoi[phai]))
                return false;          // co 1 cap lech la ket luan luon, khong can xet tiep

            trai++;
            phai--;
        }

        // xet het ma khong cap nao lech -> doi xung
        return true;
    }
  }
}
```

File: Program.cs

```
using System.Text;

namespace Bai10_KiemTraDoiXung
{
  internal class Program
  {
    // Bat nhap lai cho toi khi co chuoi hop le (khong rong, khong toan khoang trang)
```

```
    static string NhapChuoi()
    {
        while (true)
        {
            Console.Write("Nhap chuoi can kiem tra: ");
            string? nhap = Console.ReadLine();   // string? vi ReadLine co the tra ve
null

            // sau khi qua dieu kien nay, compiler biet nhap khac null nen return duoc
            if (!string.IsNullOrWhiteSpace(nhap))
                return nhap;

            Console.WriteLine("Chuoi khong duoc de trong, moi nhap lai!");
        }
    }

    static void Main(string[] args)
    {
        // dat UTF-8 de nhap/xuat duoc chuoi co dau tieng Viet, khong bi loi font
        Console.InputEncoding = Encoding.UTF8;
        Console.OutputEncoding = Encoding.UTF8;

        string chuoi = NhapChuoi();

        // KiemTraDoiXung la instance method nen phai new doi tuong moi goi duoc
        KiemTraChuoi kiemTra = new KiemTraChuoi();

        if (kiemTra.KiemTraDoiXung(chuoi))
            Console.WriteLine("Chuoi \"" + chuoi + "\" la chuoi doi xung");
        else
```

```
            Console.WriteLine("Chuoi \"" + chuoi + "\" khong phai chuoi doi xung");
        }
    }
}
```

File: Test1.cs (project Bai10_KiemTraDoiXungTest, dùng MSTest)

```
using Bai10_KiemTraDoiXung;

namespace Bai10_KiemTraDoiXungTest
{
    [TestClass]
    public class Test1
    {
        // MSTest tao doi tuong test moi cho tung [TestMethod] nen dung chung field nay duoc
        private readonly KiemTraChuoi kiemTra = new KiemTraChuoi();

        // ---- Truong hop thuong ----

        [TestMethod]
        public void ChuoiDoDaiLe_DoiXung_TraVeTrue()
        {
            Assert.IsTrue(kiemTra.KiemTraDoiXung("aba"));    // ky tu giua tu doi xung voi chinh no
        }

        [TestMethod]
        public void ChuoiDoDaiChan_DoiXung_TraVeTrue()
        {
            Assert.IsTrue(kiemTra.KiemTraDoiXung("abba"));
```

```
}

[TestMethod]
public void ChuoiKhongDoiXung_TraVeFalse()
{
    Assert.IsFalse(kiemTra.KiemTraDoiXung("abc"));
}

[TestMethod]
public void KhacBietOGiuaChuoi_TraVeFalse()
{
    // 2 dau giong nhau ("a"), chi lech o cap ben trong ("b" va "d")
    Assert.IsFalse(kiemTra.KiemTraDoiXung("abcda"));
}

[TestMethod]
public void HaiKyTuKhacNhau_TraVeFalse()
{
    Assert.IsFalse(kiemTra.KiemTraDoiXung("ab"));
}

[TestMethod]
public void ChuoiSoDoiXung_TraVeTrue()
{
    Assert.IsTrue(kiemTra.KiemTraDoiXung("12321"));
}

[TestMethod]
public void ChuoiSoKhongDoiXung_TraVeFalse()
{
```

```
        Assert.IsFalse(kiemTra.KiemTraDoiXung("12345"));
    }

    // ---- Truong hop bien ----

    [TestMethod]
    public void MotKyTu_TraVeTrue()
    {
        Assert.IsTrue(kiemTra.KiemTraDoiXung("a"));
    }

    [TestMethod]
    public void ChuoiRong_TraVeTrue()
    {
        Assert.IsTrue(kiemTra.KiemTraDoiXung(""));
    }

    [TestMethod]
    public void ChuoiNull_TraVeFalse()
    {
        Assert.IsFalse(kiemTra.KiemTraDoiXung(null));
    }

    // ---- Kiem tra quy uoc bo qua hoa thuong / khoang trang ----

    [TestMethod]
    public void KhacHoaThuong_VanDoiXung_TraVeTrue()
    {
        Assert.IsTrue(kiemTra.KiemTraDoiXung("Aba"));    // 'A' va 'a' coi nhu giong
nhau
```

```
        }

        [TestMethod]
        public void CoKhoangTrangGiua_VanDoiXung_TraVeTrue()
        {
            Assert.IsTrue(kiemTra.KiemTraDoiXung("a man a plan a canal panama"));
        }

        [TestMethod]
        public void CoKhoangTrangDauCuoi_VanDoiXung_TraVeTrue()
        {
            Assert.IsTrue(kiemTra.KiemTraDoiXung("  aba  "));
        }
    }
}
```

**4. Test case (Input / Output)**

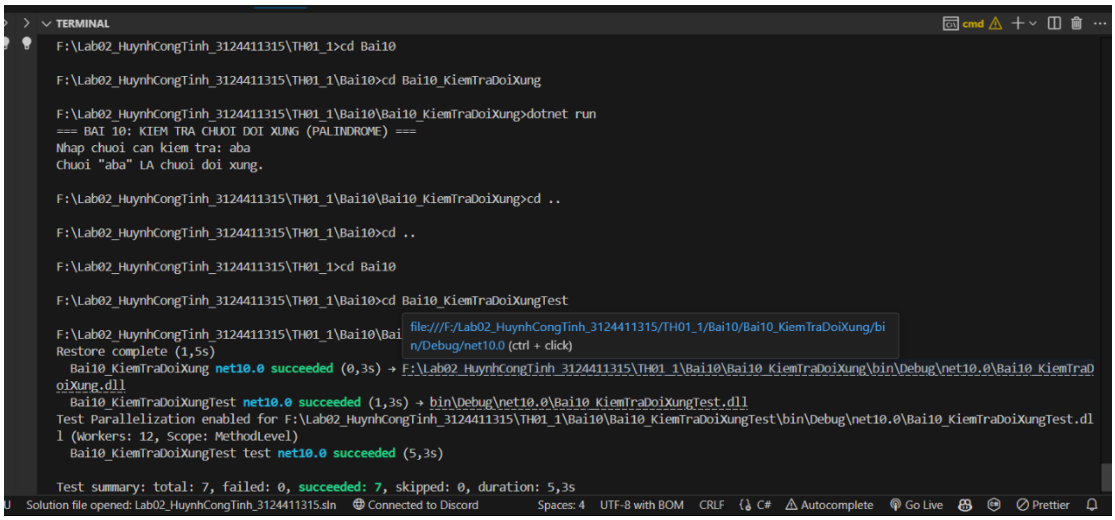
Các test case dưới đây gồm chuỗi đối xứng độ dài lẻ và chẵn, chuỗi không đối xứng, các trường hợp biên (1 ký tự, chuỗi rỗng, null) và các trường hợp để kiểm tra quy ước đã chọn (chữ hoa, chữ thường, khoảng trắng).

| STT | Input (chuỗi nhập) | Output | Ghi chú |
|---|---|---|---|
| 1 | aba | Chuoi "aba" la chuoi doi xung | Độ dài lẻ, ký tự ở giữa tự đối xứng với chính nó |
| 2 | abba | Chuoi "abba" la chuoi doi xung | Độ dài chẵn |
| 3 | abc | Chuoi "abc" khong phai chuoi doi xung | Cặp đầu tiên (a, c) đã khác nhau nên dừng ngay |

| STT | Input (chuỗi nhập) | Output | Ghi chú |
|---|---|---|---|
| 4 | abcda | Chuoi "abcda" khong phai chuoi doi xung | Hai đầu đều là a, chỉ lệch ở cặp bên trong (b, d) |
| 5 | ab | Chuoi "ab" khong phai chuoi doi xung | Chuỗi 2 ký tự khác nhau |
| 6 | 12321 | Chuoi "12321" la chuoi doi xung | Chuỗi toàn chữ số |
| 7 | 12345 | Chuoi "12345" khong phai chuoi doi xung | Chuỗi số không đối xứng |
| 8 | a | Chuoi "a" la chuoi doi xung | Biên: 1 ký tự, không vào vòng lặp |
| 9 | Aba | Chuoi "Aba" la chuoi doi xung | A hoa và a thường được coi là giống nhau |
| 10 | a man a plan a canal panama | Chuoi "a man a plan a canal panama" la chuoi doi xung | Có khoảng trắng ở giữa, được bỏ qua khi so sánh |
| 11 | aba | Chuoi "  aba  " la chuoi doi xung | Có 2 khoảng trắng ở mỗi đầu, vẫn đối xứng |
| 12 | (bấm Enter, không nhập gì) | Chuoi khong duoc de trong, moi nhap lai! | Program chặn chuỗi rỗng và bắt nhập lại |
| 13 | "" (chuỗi rỗng, gọi trực tiếp trong test) | true | Biên: giá trị phương thức trả về là true, chỉ kiểm tra được qua unit test |

| STT | Input (chuỗi nhập) | Output | Ghi chú |
|---|---|---|---|
| 14 | null (gọi trực tiếp trong test) | false | Biên: giá trị phương thức trả về là false, chỉ kiểm tra được qua unit test |

**Kết quả sau khi thực hiện:**




## * Chuỗi (string, StringBuilder)

**Bài 11:** Viết phương thức thành viên trả về chuỗi là đảo của một chuỗi

### 1. Ý tưởng giải bài toán

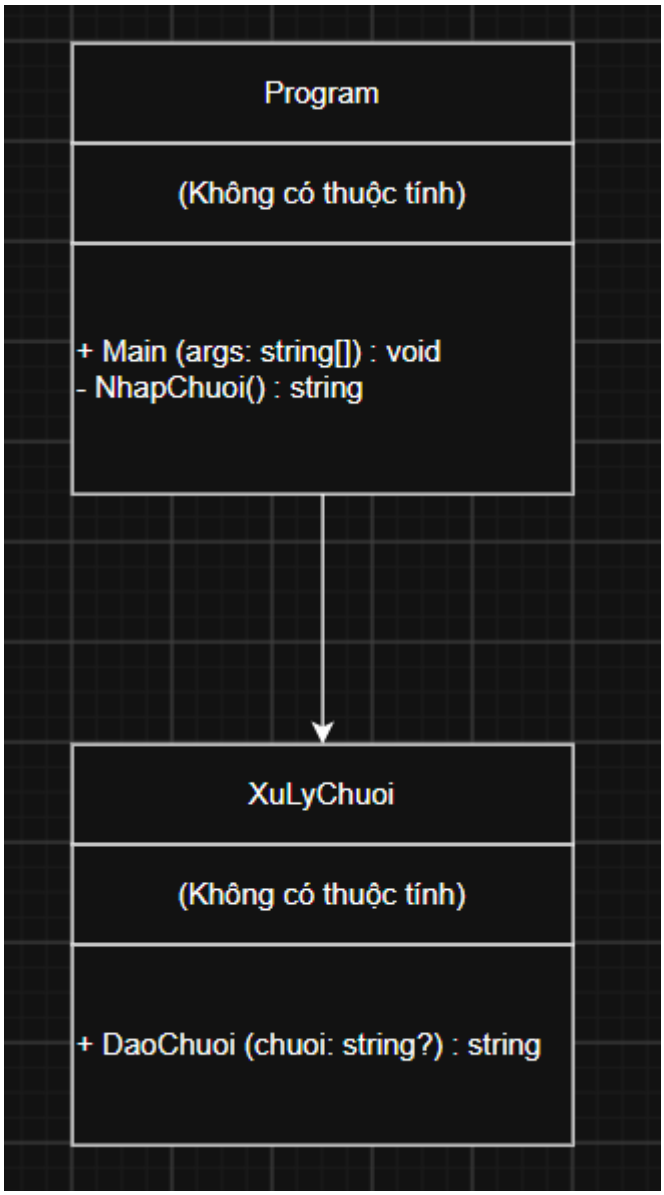
Tách riêng lớp XuLyChuoi chứa phương thức DaoChuoi(chuoi). Đề yêu cầu phương thức thành viên nên đây là phương thức thể hiện (instance method, không có static), muốn gọi phải tạo đối tượng bằng new XuLyChuoi(). Lớp này không đụng đến Console, phần nhập xuất để Program lo.

- **Bước 1:** Trong Main, nhập chuỗi bằng hàm NhapChuoi(). Hàm này lặp while (true), nếu người dùng chỉ bấm Enter hoặc nhập toàn khoảng trắng thì báo lỗi và cho nhập lại.
- **Bước 2:** Tạo đối tượng XuLyChuoi rồi gọi xuLy.DaoChuoi(chuoi), kết quả nhận về là một chuỗi mới.
- **Bước 3:** Trong phương thức, nếu chuỗi là null hoặc rỗng thì return string.Empty luôn vì không có gì để đảo.
- **Bước 4:** Tạo một StringBuilder có sức chứa bằng chuoi.Length, rồi cho biến i chạy từ chuoi.Length - 1 xuống 0, mỗi lần Append ký tự chuoi[i] vào.
- **Bước 5:** Gọi ketQua.ToString() để lấy chuỗi đã đảo và return.

Kiểu string trong C# là bất biến, không thể đổi chỗ các ký tự ngay trên chuỗi gốc nên bắt buộc phải tạo chuỗi mới. Nếu viết ketQua += chuoi[i] thì mỗi lần cộng lại sinh ra thêm một chuỗi mới, chuỗi càng dài càng tốn. Dùng StringBuilder thì các ký tự được ghi vào một vùng đệm, cuối cùng mới ToString() một lần. Phương thức trả về string.Empty thay vì null để chỗ gọi không phải kiểm tra null cho kết quả. Vì chỉ trả về chuỗi mới nên chuỗi gốc ở Main không bị thay đổi, điều này được kiểm tra ở test cuối.

**2. Sơ đồ lớp (Class Diagram)**

Chương trình gồm 2 lớp: Program lo nhập chuỗi và in kết quả; XuLyChuoi chứa phương thức đảo chuỗi.




*Quan hệ giữa hai lớp: Program phụ thuộc (dependency) vào XuLyChuoi, cụ thể là tạo một đối tượng XuLyChuoi rồi gọi phương thức DaoChuoi() của đối tượng đó để lấy chuỗi đảo.*

**3. Code chương trình**

File: XuLyChuoi.cs

```
using System.Text;

namespace Bai11_DaoChuoi
{
   public class XuLyChuoi
   {
      // Tra ve chuoi dao nguoc cua chuoi dau vao, vi du "abc" -> "cba".
      // string trong C# la bat bien nen khong the doi cho ky tu ngay tren chuoi goc,
      // phai tao chuoi moi. Dung StringBuilder de khong sinh ra nhieu chuoi trung
gian
      // (neu dung ketQua += ky tu thi moi lan cong la tao them 1 chuoi moi).
      public string DaoChuoi(string? chuoi)
      {
         // null hoac rong thi khong co gi de dao, tra ve chuoi rong (khong tra null
         // de noi khac khong phai kiem tra null ket qua)
         if (string.IsNullOrEmpty(chuoi))
            return string.Empty;

         // biet truoc do dai nen cap phat san dung dung so cho can dung
         StringBuilder ketQua = new StringBuilder(chuoi.Length);

         // duyet tu ky tu cuoi ve ky tu dau
         for (int i = chuoi.Length - 1; i >= 0; i--)
         {
            ketQua.Append(chuoi[i]);
         }
```

```
            return ketQua.ToString();
        }
    }
}
```

File: Program.cs

```
using System.Text;

namespace Bai11_DaoChuoi
{
    internal class Program
    {
        // Bat nhap lai cho toi khi co chuoi hop le (khong rong, khong toan khoang trang)
        static string NhapChuoi()
        {
            while (true)
            {
                Console.Write("Nhap chuoi can dao: ");
                string? nhap = Console.ReadLine();   // string? vi ReadLine co the tra ve
null

                if (!string.IsNullOrWhiteSpace(nhap))
                    return nhap;

                Console.WriteLine("Chuoi khong duoc de trong, moi nhap lai!");
            }
        }

        static void Main(string[] args)
        {
```

```
        // dat UTF-8 de nhap/xuat duoc chuoi co dau tieng Viet, khong bi loi font
        Console.InputEncoding = Encoding.UTF8;
        Console.OutputEncoding = Encoding.UTF8;

        string chuoi = NhapChuoi();

        // DaoChuoi la instance method nen phai new doi tuong moi goi duoc
        XuLyChuoi xuLy = new XuLyChuoi();
        string chuoiDao = xuLy.DaoChuoi(chuoi);

        Console.WriteLine("Chuoi ban dau  : " + chuoi);
        Console.WriteLine("Chuoi dao nguoc: " + chuoiDao);
    }
  }
}
```

File: Test1.cs (project Bai11_DaoChuoiTest, dùng MSTest)

```
using Bai11_DaoChuoi;

namespace Bai11_DaoChuoiTest
{
  [TestClass]
  public class Test1
  {
    // MSTest tao doi tuong test moi cho tung [TestMethod] nen dung chung field nay duoc
    private readonly XuLyChuoi xuLy = new XuLyChuoi();

    // ---- Truong hop thuong ----
```

```
[TestMethod]
public void ChuoiBinhThuong_TraVeChuoiDao()
{
    Assert.AreEqual("cba", xuLy.DaoChuoi("abc"));
}

[TestMethod]
public void ChuoiCoKhoangTrang_KhoangTrangDoiVeDungViTri()
{
    Assert.AreEqual("oahc nix", xuLy.DaoChuoi("xin chao"));
}

[TestMethod]
public void ChuoiSo_TraVeChuoiDao()
{
    Assert.AreEqual("54321", xuLy.DaoChuoi("12345"));
}

[TestMethod]
public void ChuoiDoiXung_DaoXongVanGiuNguyen()
{
    Assert.AreEqual("abba", xuLy.DaoChuoi("abba"));
}

[TestMethod]
public void ChuoiHoaThuong_GiuNguyenKieuChu()
{
    // chi doi vi tri, khong doi hoa/thuong cua tung ky tu
    Assert.AreEqual("CbA", xuLy.DaoChuoi("AbC"));
}
```

```
[TestMethod]
public void ChuoiTiengVietCoDau_DaoDungKyTu()
{
    Assert.AreEqual("maN tệiV", xuLy.DaoChuoi("Việt Nam"));
}

// ---- Truong hop bien ----

[TestMethod]
public void MotKyTu_TraVeChinNo()
{
    Assert.AreEqual("a", xuLy.DaoChuoi("a"));
}

[TestMethod]
public void ChuoiRong_TraVeChuoiRong()
{
    Assert.AreEqual("", xuLy.DaoChuoi(""));
}

[TestMethod]
public void ChuoiNull_TraVeChuoiRong()
{
    Assert.AreEqual("", xuLy.DaoChuoi(null));
}

[TestMethod]
public void KhoangTrangODauChuoi_ChuyenSangCuoiChuoi()
{
```

```
        Assert.AreEqual("ba  ", xuLy.DaoChuoi("  ab"));
    }

    // ---- Kiem tra tinh chat ----

    [TestMethod]
    public void DaoHaiLan_RaLaiChuoiGoc()
    {
        string goc = "Hello World";
        Assert.AreEqual(goc, xuLy.DaoChuoi(xuLy.DaoChuoi(goc)));
    }

    [TestMethod]
    public void SauKhiDao_ChuoiGocKhongBiThayDoi()
    {
        // string bat bien nen ham chi tra ve chuoi moi, chuoi goc van nhu cu
        string goc = "abc";
        xuLy.DaoChuoi(goc);
        Assert.AreEqual("abc", goc);
    }
  }
}
```

**4. Test case (Input / Output)**

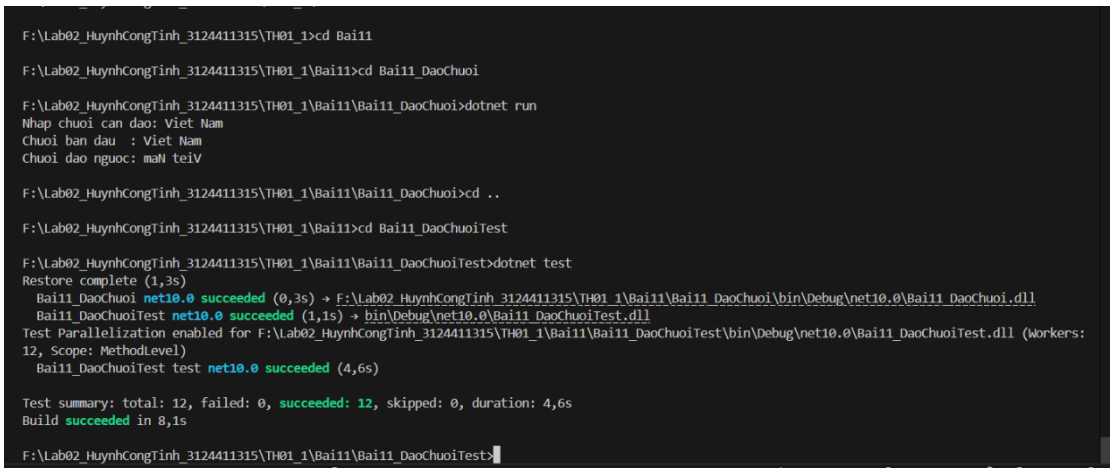
Các test case dưới đây gồm chuỗi thường, chuỗi có khoảng trắng, chuỗi số, chuỗi đối xứng, chuỗi tiếng Việt có dấu, các trường hợp biên (1 ký tự, chuỗi rỗng, null) và hai test kiểm tra tính chất của phép đảo.

| STT | Input (chuỗi nhập) | Output | Ghi chú |
|---|---|---|---|

| STT | Input (chuỗi nhập) | Output | Ghi chú |
|---|---|---|---|
| 1 | abc | Chuoi ban dau : abc<br>Chuoi dao nguoc: cba | Trường hợp thường |
| 2 | xin chao | Chuoi ban dau : xin chao<br>Chuoi dao nguoc: oahc nix | Khoảng trắng đổi vị trí như một ký tự bình thường |
| 3 | 12345 | Chuoi ban dau : 12345<br>Chuoi dao nguoc: 54321 | Chuỗi toàn chữ số |
| 4 | abba | Chuoi ban dau : abba<br>Chuoi dao nguoc: abba | Chuỗi đối xứng, đảo xong vẫn như cũ |
| 5 | AbC | Chuoi ban dau : AbC<br>Chuoi dao nguoc: CbA | Chỉ đổi vị trí, chữ hoa chữ thường của từng ký tự giữ nguyên |
| 6 | Việt Nam | Chuoi ban dau : Việt Nam<br>Chuoi dao nguoc: maN tệiV | Có dấu tiếng Việt, mỗi ký tự có dấu vẫn được đảo nguyên vẹn |
| 7 | a | Chuoi ban dau : a<br>Chuoi dao nguoc: a | Biên: chuỗi 1 ký tự |
| 8 | ab (2 khoảng trắng đầu) | Chuoi ban dau : ab<br>Chuoi dao nguoc: ba | Biên: khoảng trắng ở đầu chuyển xuống cuối chuỗi |
| 9 | (bấm Enter, không nhập gì) | Chuoi khong duoc de trong, moi nhap lai! | Program chặn chuỗi rỗng và bắt nhập lại |
| 10 | "" (chuỗi rỗng, gọi trực tiếp trong test) | "" (chuỗi rỗng) | Biên: không có gì để đảo nên trả về chuỗi rỗng, chỉ kiểm tra được qua unit test |
| 11 | null (gọi trực tiếp trong test) | "" (chuỗi rỗng) | Biên: null cũng trả về chuỗi rỗng, không bị lỗi |

| STT | Input (chuỗi nhập) | Output | Ghi chú |
|---|---|---|---|
| 12 | Hello World (đảo 2 lần) | Hello World | Gọi DaoChuoi(DaoChuoi(goc)) phải ra lại chuỗi gốc |
| 13 | abc (kiểm tra biến gốc sau khi gọi) | goc = "abc" (không đổi) | string bất biến, phương thức chỉ trả về chuỗi mới, chuỗi gốc không bị sửa |

**Kết quả sau khi thực hiện:**




*** Chuỗi (string, StringBuilder)**

**Bài 12:** Nhập một chuỗi gồm nhiều từ. Hãy chuyển chuỗi đó sang ký tự thường, sang ký tự hoa, đếm số từ trong chuỗi.

## 1. Ý tưởng giải bài toán

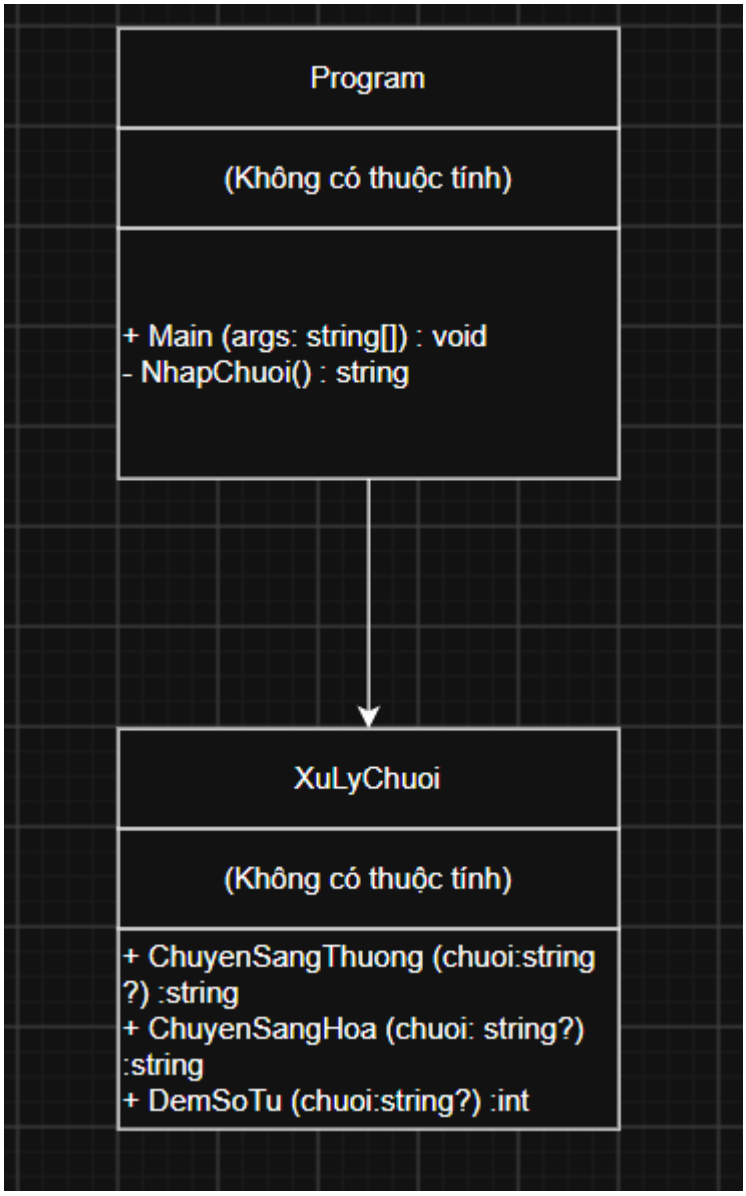
Tách riêng lớp XuLyChuoi chứa 3 phương thức tương ứng với 3 yêu cầu của đề: ChuyenSangThuong(chuoi), ChuyenSangHoa(chuoi) và DemSoTu(chuoi). Cả 3 đều là phương thức thể hiện (instance method, không có static), muốn gọi phải tạo đối tượng bằng new XuLyChuoi(). Lớp này không đụng đến Console, phần nhập xuất để Program lo.

- Bước 1: Trong Main, nhập chuỗi bằng hàm NhapChuoi(). Hàm này lặp while (true), nếu người dùng chỉ bấm Enter hoặc nhập toàn khoảng trắng thì báo lỗi và cho nhập lại.
- Bước 2: Tạo đối tượng XuLyChuoi rồi gọi lần lượt 3 phương thức trên cùng chuỗi vừa nhập.
- Bước 3: ChuyenSangThuong và ChuyenSangHoa: nếu chuỗi là null hoặc rỗng thì return string.Empty, còn lại gọi ToLowerInvariant() hoặc ToUpperInvariant() rồi trả về kết quả.
- Bước 4: DemSoTu: nếu chuỗi rỗng hoặc toàn khoảng trắng thì trả về 0. Ngược lại tách chuỗi bằng Split theo dấu cách và Tab, kèm StringSplitOptions.RemoveEmptyEntries, số phần tử của mảng thu được chính là số từ.
- Bước 5: Trong Main, in ra chuỗi ban đầu, chuỗi thường, chuỗi hoa và số từ.

Dùng phiên bản Invariant của ToLower/ToUpper để kết quả không phụ thuộc vào cài đặt ngôn ngữ của máy, chữ tiếng Việt có dấu vẫn đổi đúng (ví dụ ệ thành Ệ). Nếu Split không có RemoveEmptyEntries thì chuỗi như "xin  chao" hoặc chuỗi có khoảng trắng ở đầu và cuối sẽ sinh ra các phần tử rỗng và bị đếm thừa, nên phải bỏ các phần tử rỗng đó đi. Các phương thức chỉ trả về chuỗi mới (string là kiểu bất biến) nên chuỗi gốc vẫn còn nguyên để in ra cùng với kết quả.

**2. Sơ đồ lớp (Class Diagram)**

Chương trình gồm 2 lớp: Program lo nhập chuỗi và in kết quả; XuLyChuoi chứa 3 phương thức xử lý chuỗi.




*Quan hệ giữa hai lớp: Program phụ thuộc (dependency) vào XuLyChuoi, cụ thể là tạo một đối tượng XuLyChuoi rồi gọi 3 phương thức của đối tượng đó để lấy kết quả.*

**3. Code chương trình**

File: XuLyChuoi.cs

```
namespace Bai12_ChuoiNhieuTu
{
```

```
public class XuLyChuoi
{
    // Doi toan bo chuoi sang chu thuong.
    // Dung ToLowerInvariant de ket qua giong nhau tren moi may, khong phu thuoc
    // cai dat ngon ngu/vung cua Windows. Chu co dau tieng Viet van doi dung.
    public string ChuyenSangThuong(string? chuoi)
    {
        // null hoac rong thi tra ve chuoi rong, noi goi khong phai kiem tra null
        if (string.IsNullOrEmpty(chuoi))
            return string.Empty;

        return chuoi.ToLowerInvariant();
    }

    // Doi toan bo chuoi sang chu hoa (cach lam giong ham tren)
    public string ChuyenSangHoa(string? chuoi)
    {
        if (string.IsNullOrEmpty(chuoi))
            return string.Empty;

        return chuoi.ToUpperInvariant();
    }

    // Dem so tu: tu la day ky tu lien nhau, cach nhau boi khoang trang hoac tab.
    // RemoveEmptyEntries de nhieu khoang trang lien tiep, hoac khoang trang o
    // dau/cuoi chuoi, khong bi dem thanh "tu rong".
    public int DemSoTu(string? chuoi)
    {
        // chuoi rong hoac toan khoang trang thi khong co tu nao
        if (string.IsNullOrWhiteSpace(chuoi))
```

```
            return 0;

        string[] cacTu = chuoi.Split(new char[] { ' ', '\t' },
StringSplitOptions.RemoveEmptyEntries);
        return cacTu.Length;
    }
  }
}
```

File: Program.cs

```
using System.Text;

namespace Bai12_ChuoiNhieuTu
{
  internal class Program
  {
    // Bat nhap lai cho toi khi co chuoi hop le (khong rong, khong toan khoang trang)
    static string NhapChuoi()
    {
      while (true)
      {
        Console.Write("Nhap chuoi (gom nhieu tu): ");
        string? nhap = Console.ReadLine();   // string? vi ReadLine co the tra ve
null

        if (!string.IsNullOrWhiteSpace(nhap))
          return nhap;

        Console.WriteLine("Chuoi khong duoc de trong, moi nhap lai!");
      }
```

```
        }

        static void Main(string[] args)
        {
            // dat UTF-8 de nhap/xuat duoc chuoi co dau tieng Viet, khong bi loi font
            Console.InputEncoding = Encoding.UTF8;
            Console.OutputEncoding = Encoding.UTF8;

            string chuoi = NhapChuoi();

            // cac ham xu ly la instance method nen phai new doi tuong moi goi duoc
            XuLyChuoi xuLy = new XuLyChuoi();

            Console.WriteLine("Chuoi ban dau : " + chuoi);
            Console.WriteLine("Chuoi thuong  : " + xuLy.ChuyenSangThuong(chuoi));
            Console.WriteLine("Chuoi hoa     : " + xuLy.ChuyenSangHoa(chuoi));
            Console.WriteLine("So tu         : " + xuLy.DemSoTu(chuoi));
        }
    }
}
```

File: Test1.cs (project Bai12_ChuoiNhieuTuTest, dùng MSTest)

```
using Bai12_ChuoiNhieuTu;

namespace Bai12_ChuoiNhieuTuTest
{
    [TestClass]
    public class Test1
    {
        // MSTest tao doi tuong test moi cho tung [TestMethod] nen dung chung field nay
```

```
duoc
        private readonly XuLyChuoi xuLy = new XuLyChuoi();

        // ---- ChuyenSangThuong ----

        [TestMethod]
        public void Thuong_ChuoiHoaThuongLanLon_TraVeToanChuThuong()
        {
            Assert.AreEqual("xin chao cac ban", xuLy.ChuyenSangThuong("Xin Chao
CAC Ban"));
        }

        [TestMethod]
        public void Thuong_ChuoiTiengVietCoDau_DoiDungChuCoDau()
        {
            Assert.AreEqual("việt nam", xuLy.ChuyenSangThuong("VIỆT Nam"));
        }

        [TestMethod]
        public void
Thuong_ChuoiCoSoVaKyTuDacBiet_GiuNguyenPhanKhongPhaiChu()
        {
            Assert.AreEqual("abc 123 !@#", xuLy.ChuyenSangThuong("ABC 123
!@#"));
        }

        [TestMethod]
        public void Thuong_ChuoiRong_TraVeChuoiRong()
        {
            Assert.AreEqual("", xuLy.ChuyenSangThuong(""));
```

```
    }

    [TestMethod]
    public void Thuong_ChuoiNull_TraVeChuoiRong()
    {
        Assert.AreEqual("", xuLy.ChuyenSangThuong(null));
    }

    // ---- ChuyenSangHoa ----

    [TestMethod]
    public void Hoa_ChuoiHoaThuongLanLon_TraVeToanChuHoa()
    {
        Assert.AreEqual("XIN CHAO CAC BAN", xuLy.ChuyenSangHoa("Xin Chao cac Ban"));
    }

    [TestMethod]
    public void Hoa_ChuoiTiengVietCoDau_DoiDungChuCoDau()
    {
        Assert.AreEqual("VIỆT NAM", xuLy.ChuyenSangHoa("Việt Nam"));
    }

    [TestMethod]
    public void Hoa_ChuoiRong_TraVeChuoiRong()
    {
        Assert.AreEqual("", xuLy.ChuyenSangHoa(""));
    }

    [TestMethod]
```

```
public void Hoa_ChuoiNull_TraVeChuoiRong()
{
    Assert.AreEqual("", xuLy.ChuyenSangHoa(null));
}

// ---- DemSoTu ----

[TestMethod]
public void DemTu_ChuoiBinhThuong_TraVeDungSoTu()
{
    Assert.AreEqual(4, xuLy.DemSoTu("Xin chao cac ban"));
}

[TestMethod]
public void DemTu_MotTu_TraVeMot()
{
    Assert.AreEqual(1, xuLy.DemSoTu("hello"));
}

[TestMethod]
public void DemTu_NhieuKhoangTrangGiuaCacTu_KhongDemThuaTu()
{
    Assert.AreEqual(2, xuLy.DemSoTu("xin     chao"));
}

[TestMethod]
public void DemTu_KhoangTrangDauVaCuoi_KhongDemThuaTu()
{
    Assert.AreEqual(2, xuLy.DemSoTu("  hello world  "));
}
```

```
[TestMethod]
public void DemTu_NganCachBangTab_VanTinhLaHaiTu()
{
    Assert.AreEqual(2, xuLy.DemSoTu("xin\tchao"));
}

[TestMethod]
public void DemTu_ChuoiTiengVietVaKyTuDacBiet_TraVeDungSoTu()
{
    // "C#" van tinh la 1 tu
    Assert.AreEqual(3, xuLy.DemSoTu("Lập trình C#"));
}

[TestMethod]
public void DemTu_ChuoiRong_TraVeKhong()
{
    Assert.AreEqual(0, xuLy.DemSoTu(""));
}

[TestMethod]
public void DemTu_ToanKhoangTrang_TraVeKhong()
{
    Assert.AreEqual(0, xuLy.DemSoTu("     "));
}

[TestMethod]
public void DemTu_ChuoiNull_TraVeKhong()
{
    Assert.AreEqual(0, xuLy.DemSoTu(null));
```

```
        }
    }
}
```

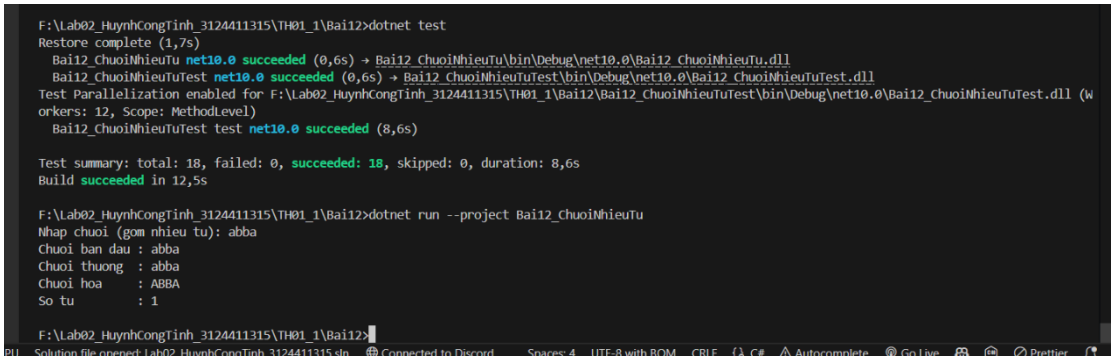
5. Test case (Input / Output)

Cột Output ghi các dòng kết quả mà chương trình in ra (hoặc giá trị các phương thức trả về với những trường hợp chỉ gọi trực tiếp trong unit test). Các test case gồm chuỗi hoa thường lẫn lộn, chuỗi tiếng Việt có dấu, chuỗi có số và ký tự đặc biệt, các trường hợp về khoảng trắng và các trường hợp biên (chuỗi rỗng, null).

| STT | Input (chuỗi nhập) | Output | Ghi chú |
|---|---|---|---|
| 1 | Xin Chao CAC Ban | Chuoi thuong : xin chao cac ban<br>Chuoi hoa : XIN CHAO CAC BAN<br>So tu : 4 | Hoa thường lẫn lộn, 4 từ |
| 2 | VIỆT Nam | Chuoi thuong : việt nam<br>Chuoi hoa : VIỆT NAM<br>So tu : 2 | Tiếng Việt có dấu, chữ có dấu vẫn đổi đúng |
| 3 | ABC 123 !@# | Chuoi thuong : abc 123 !@#<br>Chuoi hoa : ABC 123 !@#<br>So tu : 3 | Số và ký tự đặc biệt giữ nguyên, chỉ đổi chữ cái |
| 4 | hello | Chuoi thuong : hello<br>Chuoi hoa : HELLO<br>So tu : 1 | Chuỗi chỉ có 1 từ |
| 5 | xin chao (nhiều khoảng trắng giữa 2 từ) | Chuoi thuong : xin chao<br>Chuoi hoa : XIN CHAO<br>So tu : 2 | Khoảng trắng thừa vẫn được giữ khi đổi chữ nhưng không bị đếm thành từ |

| STT | Input (chuỗi nhập) | Output | Ghi chú |
|---|---|---|---|
| 6 | hello world (có khoảng trắng đầu và cuối) | Chuoi thuong : hello world<br>Chuoi hoa : HELLO WORLD<br>So tu : 2 | Khoảng trắng ở hai đầu không được tính là từ |
| 7 | xin chao (ngăn cách bằng 1 phím Tab) | So tu : 2 | Tab cũng được coi là dấu ngăn cách giữa các từ |
| 8 | Lập trình C# | Chuoi thuong : lập trình c#<br>Chuoi hoa : LẬP TRÌNH C#<br>So tu : 3 | Từ có ký tự đặc biệt (C#) vẫn tính là 1 từ |
| 9 | (bấm Enter, không nhập gì) | Chuoi khong duoc de trong, moi nhap lai! | Program chặn chuỗi rỗng và bắt nhập lại |
| 10 | "" (chuỗi rỗng, gọi trực tiếp trong test) | ChuyenSangThuong = ""<br>ChuyenSangHoa = ""<br>DemSoTu = 0 | Biên: cả 3 phương thức đều xử lý được, chỉ kiểm tra được qua unit test |
| 11 | null (gọi trực tiếp trong test) | ChuyenSangThuong = ""<br>ChuyenSangHoa = ""<br>DemSoTu = 0 | Biên: null không gây lỗi, trả về chuỗi rỗng hoặc 0 |
| 12 | 5 khoảng trắng liền nhau (gọi trực tiếp trong test) | DemSoTu = 0 | Toàn khoảng trắng thì không có từ nào |

Kết quả sau khi thực hiện:




## Xây dựng lớp cơ bản

## * Bài 13. Nhập xuất thông tin sinh viên

**Xây dựng lớp cơ bản**

**Bài 13. Nhập xuất thông tin sinh viên**

Xây dựng lớp sinh viên để lưu trữ 1 sinh viên (mã sinh viên, họ tên, địa chỉ, sinh viên năm thứ mấy). Hãy nhập xuất 1 sinh viên.

### 1. Ý tưởng giải bài toán

Tách riêng lớp SinhVien để lưu thông tin một sinh viên. Lớp gồm 4 thuộc tính (MaSinhVien, HoTen, DiaChi, NamThu), một constructor nhận đủ 4 giá trị và phương thức LayThongTin() trả về thông tin dưới dạng chuỗi. Lớp này không đụng đến Console, phần nhập xuất để Program lo.

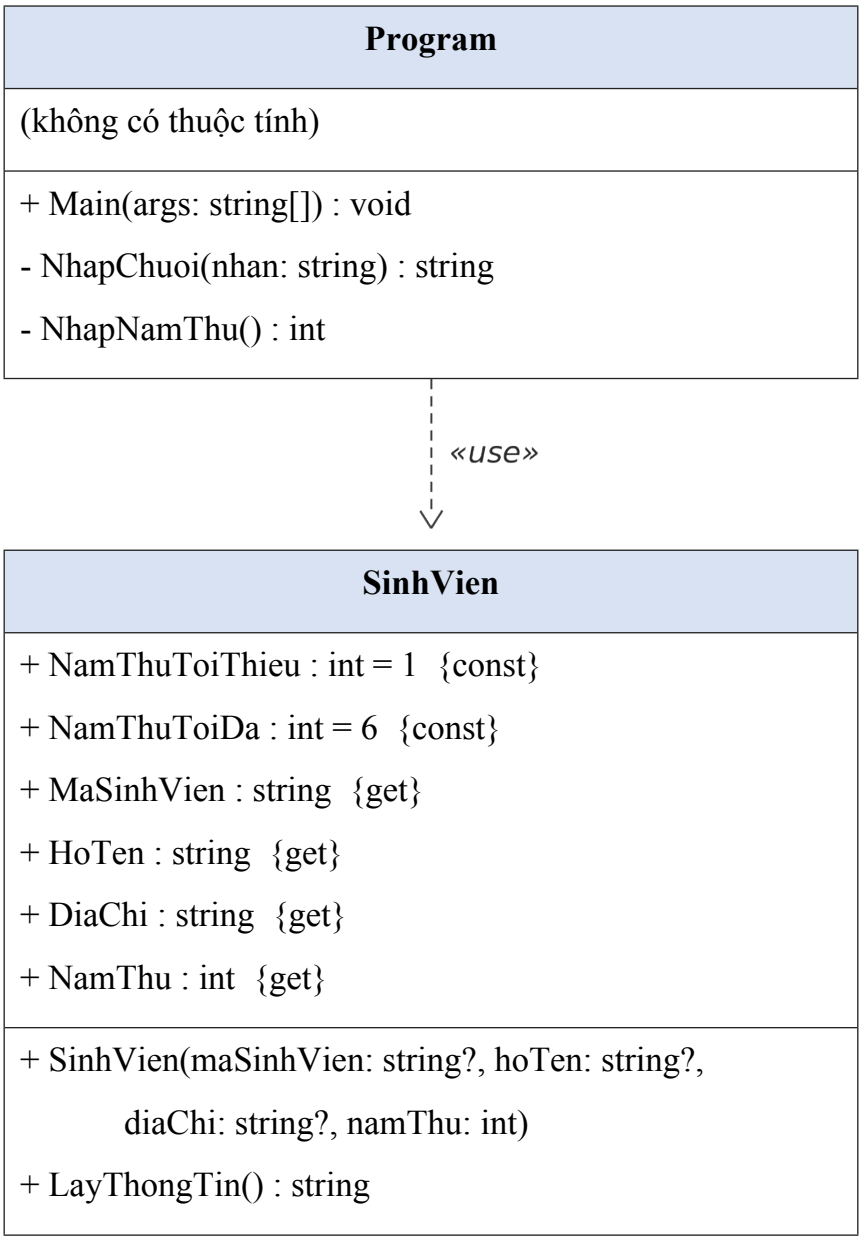
- **Bước 1:** Trong Main, nhập mã sinh viên, họ tên, địa chỉ bằng hàm NhapChuoi() (lặp while (true), không cho để trống, cắt khoảng trắng thừa). Nhập năm thứ bằng hàm NhapNamThu() (dùng int.TryParse và kiểm tra nằm trong khoảng 1 đến 6, sai thì báo lỗi và cho nhập lại).
- **Bước 2:** Tạo đối tượng bằng new SinhVien(maSinhVien, hoTen, diaChi, namThu), constructor sẽ lưu 4 giá trị vào các thuộc tính.
- **Bước 3:** Trong constructor, kiểm tra dữ liệu: 3 chuỗi bị null, rỗng hoặc toàn khoảng trắng thì ném ArgumentException, năm thứ ngoài khoảng 1 đến 6 thì ném ArgumentOutOfRangeException. Hợp lệ thì cắt khoảng trắng hai đầu (Trim) rồi gán vào thuộc tính.

- **Bước 4:** Gọi sinhVien.LayThongTin() để lấy chuỗi 4 dòng (mã, họ tên, địa chỉ, năm thứ) rồi in ra bằng Console.WriteLine.

Các thuộc tính chỉ có get và chỉ được gán trong constructor, nhờ vậy đối tượng SinhVien đã tạo ra thì luôn hợp lệ và không bị sửa thành giá trị sai ở bên ngoài. Program đã kiểm tra dữ liệu lúc nhập nhưng constructor vẫn kiểm tra lại, vì lớp SinhVien có thể được dùng ở nơi khác không đi qua Program. Phương thức LayThongTin() trả về string thay vì in trực tiếp để lớp chỉ lo dữ liệu, việc in ra màn hình là của Program, và cũng dễ viết test hơn. Hai hằng số NamThuToiThieu và NamThuToiDa được khai báo trong lớp để constructor và Program dùng chung, không phải viết số 1 và 6 rải rác.

## 2. Sơ đồ lớp (Class Diagram)

Chương trình gồm 2 lớp: Program lo nhập dữ liệu và in kết quả; SinhVien lưu thông tin một sinh viên và kiểm tra dữ liệu hợp lệ.




*Quan hệ giữa hai lớp: Program phụ thuộc (dependency) vào SinhVien, cụ thể là tạo một đối tượng SinhVien từ dữ liệu vừa nhập, dùng hai hằng số NamThuToiThieu, NamThuToiDa để kiểm tra năm thứ và gọi phương thức LayThongTin() để lấy chuỗi in ra.*

4. Code chương trình

File: SinhVien.cs

```
namespace Bai13_SinhVien
{
   public class SinhVien
   {
      // Sinh vien hoc tu nam 1 den nam 6 (nganh y co the hoc 6 nam)
      public const int NamThuToiThieu = 1;
      public const int NamThuToiDa = 6;

      // Chi cho gan gia tri qua constructor (thuoc tinh chi co get), de moi doi tuong
      // tao ra luon hop le, khong ai sua thanh gia tri sai o ben ngoai duoc.
      public string MaSinhVien { get; }
      public string HoTen { get; }
      public string DiaChi { get; }
      public int NamThu { get; }

      public SinhVien(string? maSinhVien, string? hoTen, string? diaChi, int namThu)
      {
         // IsNullOrWhiteSpace bat ca null, rong va chuoi chi toan khoang trang
         if (string.IsNullOrWhiteSpace(maSinhVien))
            throw new ArgumentException("Ma sinh vien khong duoc de trong",
nameof(maSinhVien));

         if (string.IsNullOrWhiteSpace(hoTen))
            throw new ArgumentException("Ho ten khong duoc de trong",
nameof(hoTen));

         if (string.IsNullOrWhiteSpace(diaChi))
```

```
            throw new ArgumentException("Dia chi khong duoc de trong",
nameof(diaChi));

        if (namThu < NamThuToiThieu || namThu > NamThuToiDa)
            throw new ArgumentOutOfRangeException(nameof(namThu), "Nam thu
phai tu 1 den 6");

        // Trim de bo khoang trang thua o dau/cuoi khi nguoi dung go du
        MaSinhVien = maSinhVien.Trim();
        HoTen = hoTen.Trim();
        DiaChi = diaChi.Trim();
        NamThu = namThu;
    }

    // Tra ve thong tin dang chuoi nhieu dong de Program in ra.
    // Khong dung Console.WriteLine ngay trong lop nay de lop chi lo du lieu,
    // viec in ra man hinh la cua Program (cung de viet test cho de).
    public string LayThongTin()
    {
        return "Ma sinh vien: " + MaSinhVien + Environment.NewLine +
               "Ho ten      : " + HoTen + Environment.NewLine +
               "Dia chi     : " + DiaChi + Environment.NewLine +
               "Nam thu     : " + NamThu;
    }
  }
}
```

File: Program.cs

```
using System.Text;
```

```
namespace Bai13_SinhVien
{
    internal class Program
    {
        // Nhap 1 chuoi khong rong, sai thi bao loi va cho nhap lai
        static string NhapChuoi(string nhan)
        {
            while (true)
            {
                Console.Write(nhan);
                string? nhap = Console.ReadLine();   // string? vi ReadLine co the tra ve
null

                if (!string.IsNullOrWhiteSpace(nhap))
                    return nhap.Trim();

                Console.WriteLine("Khong duoc de trong, moi nhap lai!");
            }
        }

        // Nhap nam thu: phai la so nguyen nam trong khoang cho phep cua lop SinhVien
        static int NhapNamThu()
        {
            while (true)
            {
                Console.Write("Nhap sinh vien nam thu may: ");
                string? nhap = Console.ReadLine();

                if (int.TryParse(nhap, out int namThu)
                    && namThu >= SinhVien.NamThuToiThieu
```

```
                && namThu <= SinhVien.NamThuToiDa)
            {
                return namThu;
            }

            Console.WriteLine("Nam thu phai la so nguyen tu 1 den 6, moi nhap lai!");
        }
    }

    static void Main(string[] args)
    {
        // dat UTF-8 de nhap/xuat duoc chuoi co dau tieng Viet, khong bi loi font
        Console.InputEncoding = Encoding.UTF8;
        Console.OutputEncoding = Encoding.UTF8;

        string maSinhVien = NhapChuoi("Nhap ma sinh vien: ");
        string hoTen = NhapChuoi("Nhap ho ten: ");
        string diaChi = NhapChuoi("Nhap dia chi: ");
        int namThu = NhapNamThu();

        SinhVien sinhVien = new SinhVien(maSinhVien, hoTen, diaChi, namThu);

        Console.WriteLine();
        Console.WriteLine("Thong tin sinh vien:");
        Console.WriteLine(sinhVien.LayThongTin());
    }
  }
}
```

File: Test1.cs (project Bai13_SinhVienTest, dùng MSTest)

```
using Bai13_SinhVien;

namespace Bai13_SinhVienTest
{
   [TestClass]
   public class Test1
   {
      // ---- Khoi tao hop le ----

      [TestMethod]
      public void KhoiTao_DuLieuHopLe_CacThuocTinhDungGiaTri()
      {
         SinhVien sv = new SinhVien("SV001", "Nguyen Van A", "12 Tran Hung
Dao", 2);

         Assert.AreEqual("SV001", sv.MaSinhVien);
         Assert.AreEqual("Nguyen Van A", sv.HoTen);
         Assert.AreEqual("12 Tran Hung Dao", sv.DiaChi);
         Assert.AreEqual(2, sv.NamThu);
      }

      [TestMethod]
      public void KhoiTao_ThongTinCoKhoangTrangDauCuoi_DuocCatBo()
      {
         SinhVien sv = new SinhVien("  SV001 ", "  Nguyen Van A ", " 12 Tran Hung
Dao  ", 2);

         Assert.AreEqual("SV001", sv.MaSinhVien);
         Assert.AreEqual("Nguyen Van A", sv.HoTen);
```

```
        Assert.AreEqual("12 Tran Hung Dao", sv.DiaChi);
    }

    [TestMethod]
    public void KhoiTao_TiengVietCoDau_GiuNguyen()
    {
        SinhVien sv = new SinhVien("SV002", "Nguyễn Văn An", "Quận 5, TP. Hồ
Chí Minh", 3);

        Assert.AreEqual("Nguyễn Văn An", sv.HoTen);
        Assert.AreEqual("Quận 5, TP. Hồ Chí Minh", sv.DiaChi);
    }

    // ---- Bien cua nam thu ----

    [TestMethod]
    public void KhoiTao_NamThuNhoNhat_HopLe()
    {
        SinhVien sv = new SinhVien("SV001", "Nguyen Van A", "HCM", 1);
        Assert.AreEqual(1, sv.NamThu);
    }

    [TestMethod]
    public void KhoiTao_NamThuLonNhat_HopLe()
    {
        SinhVien sv = new SinhVien("SV001", "Nguyen Van A", "HCM", 6);
        Assert.AreEqual(6, sv.NamThu);
    }

    [TestMethod]
```

```
public void KhoiTao_NamThuBangKhong_NemLoi()
{
    Assert.ThrowsExactly<ArgumentOutOfRangeException>(
        () => new SinhVien("SV001", "Nguyen Van A", "HCM", 0));
}

[TestMethod]
public void KhoiTao_NamThuLonHonToiDa_NemLoi()
{
    Assert.ThrowsExactly<ArgumentOutOfRangeException>(
        () => new SinhVien("SV001", "Nguyen Van A", "HCM", 7));
}

[TestMethod]
public void KhoiTao_NamThuAm_NemLoi()
{
    Assert.ThrowsExactly<ArgumentOutOfRangeException>(
        () => new SinhVien("SV001", "Nguyen Van A", "HCM", -1));
}

// ---- Du lieu chuoi khong hop le ----

[TestMethod]
public void KhoiTao_MaSinhVienRong_NemLoi()
{
    Assert.ThrowsExactly<ArgumentException>(
        () => new SinhVien("", "Nguyen Van A", "HCM", 2));
}

[TestMethod]
```

```
    public void KhoiTao_MaSinhVienNull_NemLoi()
    {
        Assert.ThrowsExactly<ArgumentException>(
            () => new SinhVien(null, "Nguyen Van A", "HCM", 2));
    }

    [TestMethod]
    public void KhoiTao_HoTenToanKhoangTrang_NemLoi()
    {
        Assert.ThrowsExactly<ArgumentException>(
            () => new SinhVien("SV001", "     ", "HCM", 2));
    }

    [TestMethod]
    public void KhoiTao_DiaChiRong_NemLoi()
    {
        Assert.ThrowsExactly<ArgumentException>(
            () => new SinhVien("SV001", "Nguyen Van A", "", 2));
    }

    // ---- Xuat thong tin va doi tuong doc lap ----

    [TestMethod]
    public void LayThongTin_TraVeDungDinhDang()
    {
        SinhVien sv = new SinhVien("SV001", "Nguyen Van A", "12 Tran Hung
Dao", 2);

        string mongDoi = "Ma sinh vien: SV001" + Environment.NewLine +
                         "Ho ten      : Nguyen Van A" + Environment.NewLine +
```

```
                    "Dia chi     : 12 Tran Hung Dao" + Environment.NewLine +
                    "Nam thu     : 2";

        Assert.AreEqual(mongDoi, sv.LayThongTin());
    }

    [TestMethod]
    public void HaiDoiTuong_DuLieuDocLapNhau()
    {
        // moi doi tuong giu du lieu rieng cua minh
        SinhVien sv1 = new SinhVien("SV001", "Nguyen Van A", "HCM", 1);
        SinhVien sv2 = new SinhVien("SV002", "Tran Thi B", "Ha Noi", 4);

        Assert.AreEqual("SV001", sv1.MaSinhVien);
        Assert.AreEqual("Tran Thi B", sv2.HoTen);
        Assert.AreNotEqual(sv1.NamThu, sv2.NamThu);
    }
  }
}
```

**4. Test case (Input / Output)**

Cột Input ghi 4 giá trị nhập lần lượt (mã sinh viên, họ tên, địa chỉ, năm thứ). Cột Output ghi kết quả chương trình in ra, hoặc ngoại lệ mà constructor ném ra với các trường hợp gọi thẳng constructor trong unit test. Các test case gồm dữ liệu hợp lệ, hai biên của năm thứ, dữ liệu sai và kiểm tra hai đối tượng độc lập.

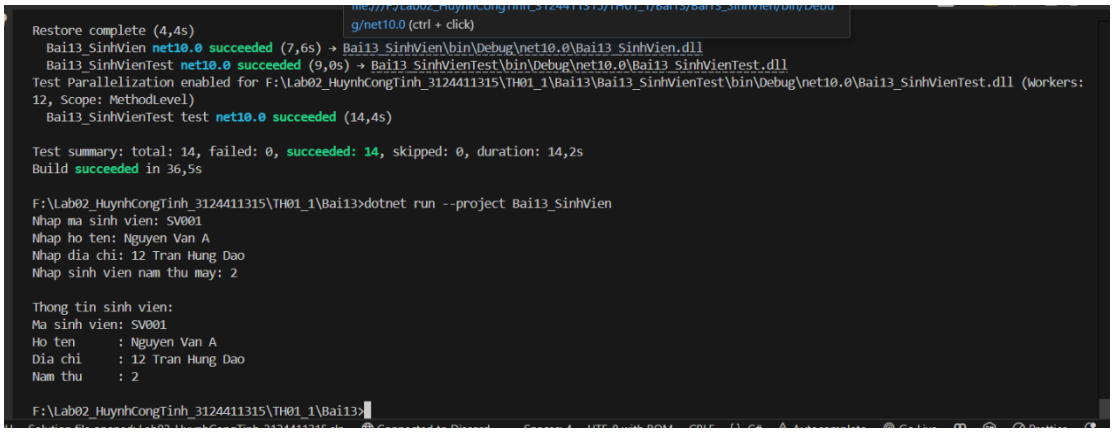
| STT | Input (nhập lần lượt) | Output | Ghi chú |
|---|---|---|---|

| STT | Input (nhập lần lượt) | Output | Ghi chú |
|---|---|---|---|
| 1 | SV001<br>Nguyen Van A<br>12 Tran Hung Dao<br>2 | Ma sinh vien: SV001<br>Ho ten : Nguyen Van A<br>Dia chi : 12 Tran Hung Dao<br>Nam thu : 2 | Dữ liệu hợp lệ, in đủ 4 dòng thông tin |
| 2 | SV001<br>Nguyen Van A<br>12 Tran Hung Dao<br>2 | Ma sinh vien: SV001<br>Ho ten : Nguyen Van A<br>Dia chi : 12 Tran Hung Dao<br>Nam thu : 2 | Khoảng trắng thừa ở đầu, cuối được cắt bỏ |
| 3 | SV002<br>Nguyễn Văn An<br>Quận 5, TP. Hồ Chí Minh<br>3 | Ma sinh vien: SV002<br>Ho ten : Nguyễn Văn An<br>Dia chi : Quận 5, TP. Hồ Chí Minh<br>Nam thu : 3 | Tiếng Việt có dấu được giữ nguyên |
| 4 | SV001<br>Nguyen Van A<br>HCM<br>1 | Ma sinh vien: SV001<br>Ho ten : Nguyen Van A<br>Dia chi : HCM<br>Nam thu : 1 | Biên dưới: sinh viên năm 1 |
| 5 | SV001<br>Nguyen Van A<br>HCM<br>6 | Ma sinh vien: SV001<br>Ho ten : Nguyen Van A<br>Dia chi : HCM<br>Nam thu : 6 | Biên trên: sinh viên năm 6 |
| 6 | SV001<br>Nguyen Van A<br>HCM<br>0 | Nam thu phai la so nguyen tu 1 den 6, moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentOutOfRangeException) | Năm 0 nhỏ hơn biên dưới nên không hợp lệ |

| STT | Input (nhập lần lượt) | Output | Ghi chú |
|---|---|---|---|
| 7 | SV001<br>Nguyen Van A<br>HCM<br>7 | Nam thu phai la so nguyen tu 1 den 6, moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentOutOfRangeException) | Năm 7 lớn hơn biên trên |
| 8 | SV001<br>Nguyen Van A<br>HCM<br>-1 | Nam thu phai la so nguyen tu 1 den 6, moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentOutOfRangeException) | Năm là số âm |
| 9 | SV001<br>Nguyen Van A<br>HCM<br>abc | Nam thu phai la so nguyen tu 1 den 6, moi nhap lai! | Không phải số nguyên, TryParse thất bại nên Program báo lỗi và cho nhập lại |
| 10 | (bấm Enter, không nhập gì)<br>Nguyen Van A<br>HCM<br>2 | Khong duoc de trong, moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentException) | Mã sinh viên rỗng |
| 11 | null<br>Nguyen Van A<br>HCM<br>2 | (gọi thẳng constructor: ném ArgumentException) | Mã sinh viên null, chỉ kiểm tra được qua unit test |
| 12 | SV001<br>(5 khoảng trắng)<br>HCM<br>2 | Khong duoc de trong, moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentException) | Họ tên toàn khoảng trắng cũng bị coi là trống |

| STT | Input (nhập lần lượt) | Output | Ghi chú |
|---|---|---|---|
| 13 | SV001<br>Nguyen Van A<br>(bấm Enter, không nhập gì)<br>2 | Khong duoc de trong, moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentException) | Địa chỉ rỗng |
| 14 | sv1 = SV001, Nguyen Van A, HCM, 1<br>sv2 = SV002, Tran Thi B, Ha Noi, 4 | sv1.MaSinhVien = SV001<br>sv2.HoTen = Tran Thi B<br>sv1.NamThu khac sv2.NamThu | Hai đối tượng cùng lớp, mỗi đối tượng giữ dữ liệu riêng |

**Kết quả sau khi thực hiện:**




## * Bài 14. Tính lương 1 nhân viên

Nhập xuất 1 sinh viên.

**Bài 14. Tính lương 1 nhân viên**

Viết chương trình nhập thông tin một nhân viên (họ tên, mức lương, số ngày vắng). Tính và xuất lương của nhân viên, biết rằng một ngày vắng sẽ bị trừ 100.000 VNĐ.

## 1. Ý tưởng giải bài toán

Tách riêng lớp NhanVien để lưu thông tin và tính lương. Lớp gồm 3 thuộc tính (HoTen, MucLuong, SoNgayVang), một constructor nhận đủ 3 giá trị và 3 phương thức TinhTienTru(), TinhLuong(), LayThongTin(). Lớp này không đụng đến Console, phần nhập xuất để Program lo.

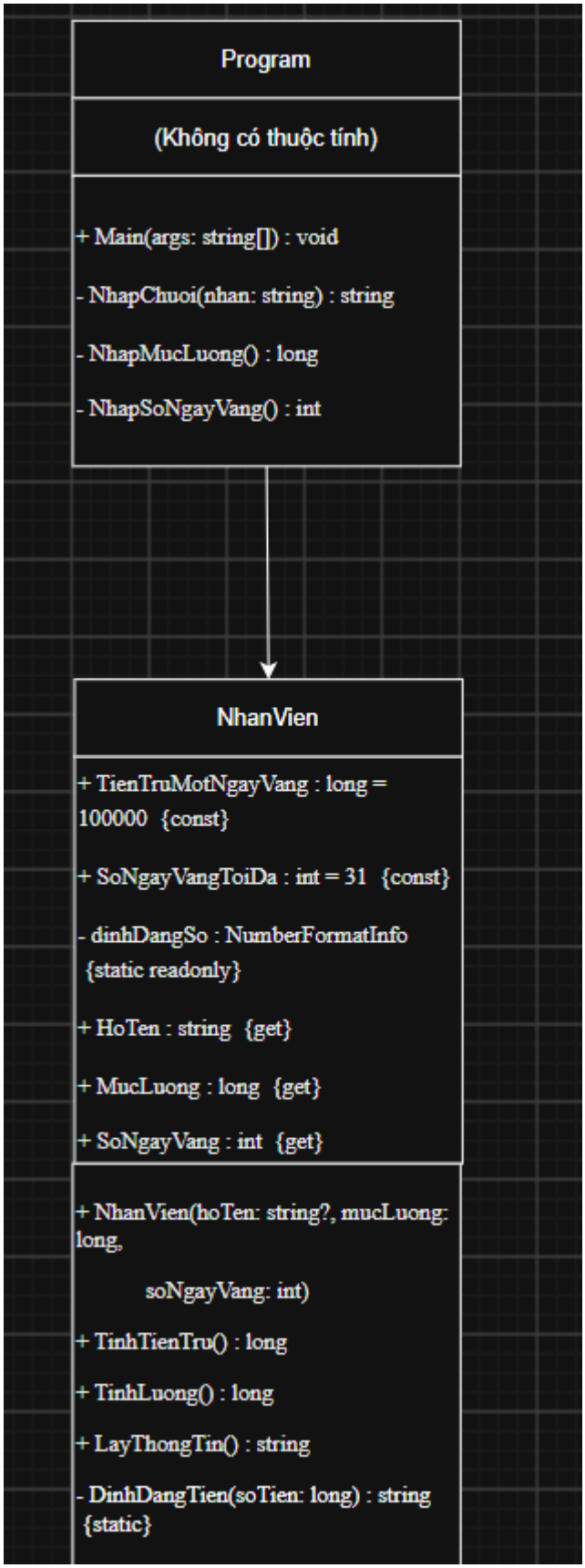
- **Bước 1:** Trong Main, nhập họ tên bằng hàm NhapChuoi() (không cho để trống). Nhập mức lương bằng NhapMucLuong() (dùng long.TryParse, phải là số nguyên không âm) và số ngày vắng bằng NhapSoNgayVang() (dùng int.TryParse, phải từ 0 đến 31). Nhập sai thì báo lỗi và cho nhập lại.
- **Bước 2:** Tạo đối tượng bằng new NhanVien(hoTen, mucLuong, soNgayVang). Constructor kiểm tra lại dữ liệu (họ tên trống thì ném ArgumentException, mức lương âm hoặc số ngày vắng ngoài khoảng 0 đến 31 thì ném ArgumentOutOfRangeException) rồi mới lưu vào các thuộc tính.
- **Bước 3:** TinhTienTru() trả về SoNgayVang * TienTruMotNgayVang, trong đó TienTruMotNgayVang là hằng số 100000.
- **Bước 4:** TinhLuong() lấy MucLuong trừ đi TinhTienTru(). Nếu kết quả âm thì trả về 0.
- **Bước 5:** LayThongTin() ghép thành chuỗi 5 dòng (họ tên, mức lương, số ngày vắng, tiền bị trừ, lương thực nhận), số tiền được định dạng có dấu chấm ngăn cách hàng nghìn kèm chữ VND. Main in chuỗi này ra màn hình.

Số tiền dùng kiểu long vì tiền VND không có phần lẻ nên không cần double (tránh sai số khi làm tròn), và long đủ rộng cho mức lương lớn trong khi int chỉ chứa được khoảng 2,1 tỷ. Số tiền trừ mỗi ngày là hằng số nên nếu đổi mức phạt chỉ cần sửa đúng một chỗ. Khi định dạng tiền, lớp tự tạo NumberFormatInfo với dấu chấm ngăn cách hàng nghìn (10.000.000) để kết quả giống nhau trên

mọi máy, không phụ thuộc cài đặt ngôn ngữ. Mức lương phải nhập liền (ví dụ 10000000) vì dấu chấm và dấu phẩy được hiểu khác nhau tùy cài đặt của máy.

## 2. Sơ đồ lớp (Class Diagram)

Chương trình gồm 2 lớp: Program lo nhập dữ liệu và in kết quả; NhanVien lưu thông tin nhân viên, kiểm tra dữ liệu và tính lương.


Program
(Không có thuộc tính)
+ Main(args: string[]) : void
- NhapChuoi(nhan: string) : string
- NhapMucLuong() : long
- NhapSoNgayVang() : int
NhanVien
+ TienTruMotNgayVang : long = 100000 {const}
+ SoNgayVangToiDa : int = 31 {const}
- dinhDangSo : NumberFormatInfo {static readonly}
+ HoTen : string {get}
+ MucLuong : long {get}
+ SoNgayVang : int {get}
+ NhanVien(hoTen: string?, mucLuong: long,
soNgayVang: int)
+ TinhTienTru() : long
+ TinhLuong() : long
+ LayThongTin() : string
- DinhDangTien(soTien: long) : string {static}

*Quan hệ giữa hai lớp: Program phụ thuộc (dependency) vào NhanVien, cụ thể là tạo một đối tượng NhanVien từ dữ liệu vừa nhập, dùng hằng số SoNgayVangToiDa để kiểm tra số ngày vắng và gọi phương thức LayThongTin() để lấy chuỗi in ra.*

**3. Code chương trình**

File: NhanVien.cs

```
using System.Globalization;

namespace Bai14_TinhLuong
{
   public class NhanVien
   {
      // Moi ngay vang bi tru 100.000 VND
      public const long TienTruMotNgayVang = 100000;

      // Mot thang nhieu nhat co 31 ngay nen khong the vang qua so nay
      public const int SoNgayVangToiDa = 31;

      // Viet tien theo kieu Viet Nam (10.000.000) nen dung dau cham ngan cach hang nghin.
      // Tu tao NumberFormatInfo rieng de ket qua khong phu thuoc cai dat ngon ngu cua may.
      private static readonly NumberFormatInfo dinhDangSo = new NumberFormatInfo { NumberGroupSeparator = "." };

      public string HoTen { get; }

      // Tien VND khong co phan le nen dung long, khong can decimal/double.
      // long cung du rong cho muc luong lon (int chi toi khoang 2,1 ty).
```

```
        public long MucLuong { get; }

        public int SoNgayVang { get; }

        public NhanVien(string? hoTen, long mucLuong, int soNgayVang)
        {
            if (string.IsNullOrWhiteSpace(hoTen))
                throw new ArgumentException("Ho ten khong duoc de trong",
nameof(hoTen));

            if (mucLuong < 0)
                throw new ArgumentOutOfRangeException(nameof(mucLuong), "Muc
luong khong duoc am");

            if (soNgayVang < 0 || soNgayVang > SoNgayVangToiDa)
                throw new ArgumentOutOfRangeException(nameof(soNgayVang), "So
ngay vang phai tu 0 den 31");

            HoTen = hoTen.Trim();
            MucLuong = mucLuong;
            SoNgayVang = soNgayVang;
        }

        // Tong tien bi tru = so ngay vang x 100.000
        public long TinhTienTru()
        {
            return SoNgayVang * TienTruMotNgayVang;
        }

        // Luong thuc nhan = muc luong - tien bi tru
```

```
        public long TinhLuong()
        {
            long luong = MucLuong - TinhTienTru();

            // vang nhieu den muc tien tru vuot muc luong thi luong nhan duoc la 0,
            // khong de luong bi am
            if (luong < 0)
                return 0;

            return luong;
        }

        // Tra ve thong tin nhieu dong de Program in ra (lop khong tu in ra man hinh)
        public string LayThongTin()
        {
            return "Ho ten         : " + HoTen + Environment.NewLine +
                   "Muc luong      : " + DinhDangTien(MucLuong) + Environment.NewLine
+
                   "So ngay vang   : " + SoNgayVang + Environment.NewLine +
                   "Tien bi tru    : " + DinhDangTien(TinhTienTru()) +
Environment.NewLine +
                   "Luong thuc nhan: " + DinhDangTien(TinhLuong());
        }

        // Vi du 9800000 -> "9.800.000 VND"
        private static string DinhDangTien(long soTien)
        {
            return soTien.ToString("N0", dinhDangSo) + " VND";
        }
    }
```

```
}
```

File: Program.cs

```
using System.Text;

namespace Bai14_TinhLuong
{
    internal class Program
    {
        // Nhap 1 chuoi khong rong, sai thi bao loi va cho nhap lai
        static string NhapChuoi(string nhan)
        {
            while (true)
            {
                Console.Write(nhan);
                string? nhap = Console.ReadLine();   // string? vi ReadLine co the tra ve null

                if (!string.IsNullOrWhiteSpace(nhap))
                    return nhap.Trim();

                Console.WriteLine("Khong duoc de trong, moi nhap lai!");
            }
        }

        // Nhap muc luong: so nguyen khong am, viet lien (khong co dau cham/phay ngan cach)
        static long NhapMucLuong()
        {
            while (true)
```

```
        {
            Console.Write("Nhap muc luong (VND): ");
            string? nhap = Console.ReadLine();

            if (long.TryParse(nhap, out long mucLuong) && mucLuong >= 0)
                return mucLuong;

            Console.WriteLine("Muc luong phai la so nguyen khong am (vi du
10000000), moi nhap lai!");
        }
    }

    // Nhap so ngay vang: so nguyen tu 0 den 31
    static int NhapSoNgayVang()
    {
        while (true)
        {
            Console.Write("Nhap so ngay vang: ");
            string? nhap = Console.ReadLine();

            if (int.TryParse(nhap, out int soNgay)
                && soNgay >= 0
                && soNgay <= NhanVien.SoNgayVangToiDa)
            {
                return soNgay;
            }

            Console.WriteLine("So ngay vang phai la so nguyen tu 0 den 31, moi nhap
lai!");
        }
```

```
        }

        static void Main(string[] args)
        {
            // dat UTF-8 de nhap/xuat duoc chuoi co dau tieng Viet, khong bi loi font
            Console.InputEncoding = Encoding.UTF8;
            Console.OutputEncoding = Encoding.UTF8;

            string hoTen = NhapChuoi("Nhap ho ten: ");
            long mucLuong = NhapMucLuong();
            int soNgayVang = NhapSoNgayVang();

            NhanVien nhanVien = new NhanVien(hoTen, mucLuong, soNgayVang);

            Console.WriteLine();
            Console.WriteLine("Thong tin luong:");
            Console.WriteLine(nhanVien.LayThongTin());
        }
    }
}
```

File: Test1.cs (project Bai14_TinhLuongTest, dùng MSTest)

```
using Bai14_TinhLuong;

namespace Bai14_TinhLuongTest
{
    [TestClass]
    public class Test1
    {
        // ---- Khoi tao ----
```

```
[TestMethod]
public void KhoiTao_DuLieuHopLe_CacThuocTinhDungGiaTri()
{
    NhanVien nv = new NhanVien("Nguyen Van A", 10000000, 2);

    Assert.AreEqual("Nguyen Van A", nv.HoTen);
    Assert.AreEqual(10000000L, nv.MucLuong);
    Assert.AreEqual(2, nv.SoNgayVang);
}

[TestMethod]
public void KhoiTao_HoTenCoKhoangTrangDauCuoi_DuocCatBo()
{
    NhanVien nv = new NhanVien("  Nguyen Van A  ", 10000000, 0);
    Assert.AreEqual("Nguyen Van A", nv.HoTen);
}

[TestMethod]
public void KhoiTao_HoTenRong_NemLoi()
{
    Assert.ThrowsExactly<ArgumentException>(
        () => new NhanVien("", 10000000, 0));
}

[TestMethod]
public void KhoiTao_HoTenNull_NemLoi()
{
    Assert.ThrowsExactly<ArgumentException>(
        () => new NhanVien(null, 10000000, 0));
```

```
}

[TestMethod]
public void KhoiTao_MucLuongAm_NemLoi()
{
    Assert.ThrowsExactly<ArgumentOutOfRangeException>(
        () => new NhanVien("Nguyen Van A", -1, 0));
}

[TestMethod]
public void KhoiTao_MucLuongBangKhong_HopLe()
{
    NhanVien nv = new NhanVien("Nguyen Van A", 0, 0);
    Assert.AreEqual(0L, nv.MucLuong);
}

[TestMethod]
public void KhoiTao_SoNgayVangAm_NemLoi()
{
    Assert.ThrowsExactly<ArgumentOutOfRangeException>(
        () => new NhanVien("Nguyen Van A", 10000000, -1));
}

[TestMethod]
public void KhoiTao_SoNgayVangLonHon31_NemLoi()
{
    Assert.ThrowsExactly<ArgumentOutOfRangeException>(
        () => new NhanVien("Nguyen Van A", 10000000, 32));
}
```

```
// ---- Tinh luong ----

[TestMethod]
public void TinhLuong_KhongVangNgayNao_LuongBangMucLuong()
{
    NhanVien nv = new NhanVien("Nguyen Van A", 10000000, 0);
    Assert.AreEqual(10000000L, nv.TinhLuong());
}

[TestMethod]
public void TinhLuong_VangMotNgay_TruDung100Nghin()
{
    NhanVien nv = new NhanVien("Nguyen Van A", 10000000, 1);
    Assert.AreEqual(9900000L, nv.TinhLuong());
}

[TestMethod]
public void TinhLuong_VangNamNgay_TruDung500Nghin()
{
    NhanVien nv = new NhanVien("Nguyen Van A", 10000000, 5);
    Assert.AreEqual(9500000L, nv.TinhLuong());
}

[TestMethod]
public void TinhLuong_TienTruBangDungMucLuong_LuongBangKhong()
{
    NhanVien nv = new NhanVien("Nguyen Van A", 1000000, 10);
    Assert.AreEqual(0L, nv.TinhLuong());
}
```

```
[TestMethod]
public void TinhLuong_TienTruVuotMucLuong_LuongBangKhongKhongAm()
{
    // tru 8 x 100.000 = 800.000 > 500.000 nhung luong khong duoc am
    NhanVien nv = new NhanVien("Nguyen Van A", 500000, 8);
    Assert.AreEqual(0L, nv.TinhLuong());
}

[TestMethod]
public void TinhLuong_VangToiDa31Ngay_TruDung3Trieu100Nghin()
{
    NhanVien nv = new NhanVien("Nguyen Van A", 10000000, 31);
    Assert.AreEqual(6900000L, nv.TinhLuong());
}

[TestMethod]
public void TinhLuong_MucLuongRatLon_KhongTranSo()
{
    // 9 nghin ty vuot xa gioi han cua int, kiem tra dung long la du
    NhanVien nv = new NhanVien("Nguyen Van A", 9000000000000, 2);
    Assert.AreEqual(8999999800000L, nv.TinhLuong());
}

[TestMethod]
public void TinhTienTru_VangBaNgay_Tru300Nghin()
{
    NhanVien nv = new NhanVien("Nguyen Van A", 10000000, 3);
    Assert.AreEqual(300000L, nv.TinhTienTru());
}
```

```
        // ---- Xuat thong tin ----

        [TestMethod]
        public void LayThongTin_TraVeDungDinhDang()
        {
            NhanVien nv = new NhanVien("Nguyen Van A", 10000000, 2);

            string mongDoi = "Ho ten         : Nguyen Van A" + Environment.NewLine +
                             "Muc luong      : 10.000.000 VND" + Environment.NewLine +
                             "So ngay vang   : 2" + Environment.NewLine +
                             "Tien bi tru    : 200.000 VND" + Environment.NewLine +
                             "Luong thuc nhan: 9.800.000 VND";

            Assert.AreEqual(mongDoi, nv.LayThongTin());
        }
    }
}
```

**4. Test case (Input / Output)**

Cột Input ghi 3 giá trị nhập lần lượt (họ tên, mức lương, số ngày vắng). Cột Output ghi kết quả chương trình in ra, hoặc ngoại lệ mà constructor ném ra với các trường hợp gọi thẳng constructor trong unit test. Các test case gồm tính lương với số ngày vắng khác nhau, các giá trị biên và dữ liệu sai.

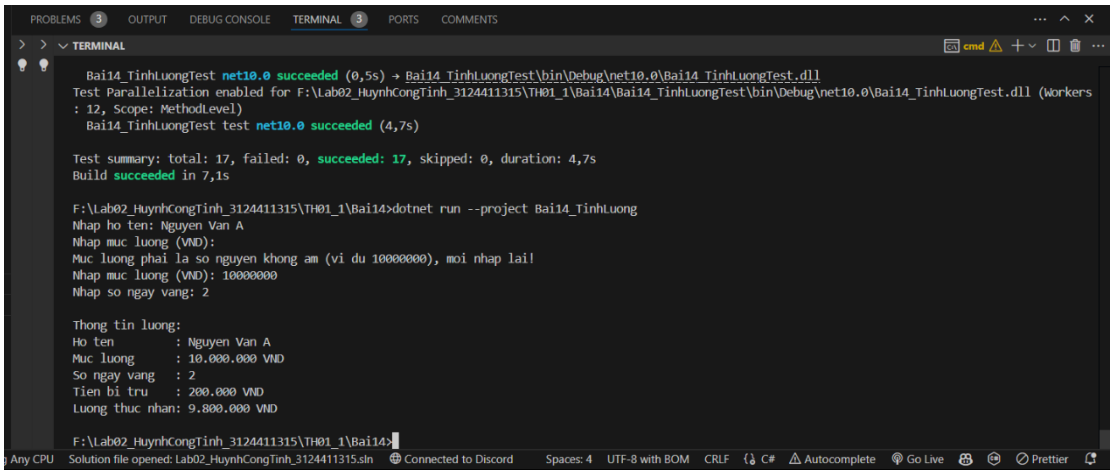
| STT | Input (nhập lần lượt) | Output | Ghi chú |
|---|---|---|---|

| STT | Input (nhập lần lượt) | Output | Ghi chú |
|---|---|---|---|
| 1 | Nguyen Van A<br>10000000<br>2 | Ho ten : Nguyen Van A<br>Muc luong : 10.000.000 VND<br>So ngay vang : 2<br>Tien bi tru : 200.000 VND<br>Luong thuc nhan: 9.800.000 VND | Trường hợp thường: trừ 2 x 100.000 = 200.000 |
| 2 | Nguyen Van A<br>10000000<br>0 | Ho ten : Nguyen Van A<br>Muc luong : 10.000.000 VND<br>So ngay vang : 0<br>Tien bi tru : 0 VND<br>Luong thuc nhan: 10.000.000 VND | Không vắng ngày nào nên lương bằng mức lương, khoảng trắng thừa ở họ tên được cắt bỏ |
| 3 | Nguyen Van A<br>10000000<br>1 | Ho ten : Nguyen Van A<br>Muc luong : 10.000.000 VND<br>So ngay vang : 1<br>Tien bi tru : 100.000 VND<br>Luong thuc nhan: 9.900.000 VND | Vắng 1 ngày, trừ đúng 100.000 |
| 4 | Nguyen Van A<br>10000000<br>5 | Ho ten : Nguyen Van A<br>Muc luong : 10.000.000 VND<br>So ngay vang : 5<br>Tien bi tru : 500.000 VND<br>Luong thuc nhan: 9.500.000 VND | Vắng 5 ngày, trừ 500.000 |
| 5 | Nguyen Van A<br>10000000<br>3 | Ho ten : Nguyen Van A<br>Muc luong : 10.000.000 VND<br>So ngay vang : 3<br>Tien bi tru : 300.000 VND<br>Luong thuc nhan: 9.700.000 VND | Kiểm tra riêng tiền bị trừ: 3 ngày là 300.000 |

| STT | Input (nhập lần lượt) | Output | Ghi chú |
|---|---|---|---|
| 6 | Nguyen Van A<br>10000000<br>31 | Ho ten : Nguyen Van A<br>Muc luong : 10.000.000 VND<br>So ngay vang : 31<br>Tien bi tru : 3.100.000 VND<br>Luong thuc nhan: 6.900.000 VND | Biên trên: vắng cả 31 ngày của tháng |
| 7 | Nguyen Van A<br>1000000<br>10 | Ho ten : Nguyen Van A<br>Muc luong : 1.000.000 VND<br>So ngay vang : 10<br>Tien bi tru : 1.000.000 VND<br>Luong thuc nhan: 0 VND | Tiền bị trừ đúng bằng mức lương nên lương thực nhận bằng 0 |
| 8 | Nguyen Van A<br>500000<br>8 | Ho ten : Nguyen Van A<br>Muc luong : 500.000 VND<br>So ngay vang : 8<br>Tien bi tru : 800.000 VND<br>Luong thuc nhan: 0 VND | Tiền trừ (800.000) lớn hơn mức lương nhưng lương vẫn là 0, không âm |
| 9 | Nguyen Van A<br>0<br>0 | Ho ten : Nguyen Van A<br>Muc luong : 0 VND<br>So ngay vang : 0<br>Tien bi tru : 0 VND<br>Luong thuc nhan: 0 VND | Biên: mức lương bằng 0 vẫn hợp lệ |
| 10 | Nguyen Van A<br>9000000000000<br>2 | Ho ten : Nguyen Van A<br>Muc luong : 9.000.000.000.000 VND<br>So ngay vang : 2<br>Tien bi tru : 200.000 VND<br>Luong thuc nhan: 8.999.999.800.000 VND | Mức lương rất lớn, vượt giới hạn của int nhưng không bị tràn nhờ dùng long |

| STT | Input (nhập lần lượt) | Output | Ghi chú |
|---|---|---|---|
| 11 | (bấm Enter, không nhập gì)<br>10000000<br>0 | Khong duoc de trong, moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentException) | Họ tên rỗng |
| 12 | null<br>10000000<br>0 | (gọi thẳng constructor: ném ArgumentException) | Họ tên null, chỉ kiểm tra được qua unit test |
| 13 | Nguyen Van A<br>-1<br>0 | Muc luong phai la so nguyen khong am (vi du 10000000), moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentOutOfRangeException) | Mức lương là số âm |
| 14 | Nguyen Van A<br>abc<br>0 | Muc luong phai la so nguyen khong am (vi du 10000000), moi nhap lai! | Không phải số nguyên, TryParse thất bại nên Program báo lỗi và cho nhập lại |
| 15 | Nguyen Van A<br>10000000<br>-1 | So ngay vang phai la so nguyen tu 0 den 31, moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentOutOfRangeException) | Số ngày vắng là số âm |
| 16 | Nguyen Van A<br>10000000<br>32 | So ngay vang phai la so nguyen tu 0 den 31, moi nhap lai!<br>(gọi thẳng constructor: ném ArgumentOutOfRangeException) | Số ngày vắng lớn hơn 31 |

**Kết quả sau khi thực hiện:**




## Mảng, ArrayList

**Mảng, ArrayList**

**Bài 15:** Viết các phương thức thành viên sau:

- Nhập mảng gồm n phần tử
- In mảng ra màn hình
- Tìm phần tử lớn nhất và nhỏ nhất trong mảng
- Trả về mảng các số nguyên tố

### 1. Ý tưởng giải bài toán

Tách riêng một lớp MangTools chứa toàn bộ các phương thức thao tác trên mảng (khác các bài truyền tham số trước đây, ở bài này đề yêu cầu chính là các phương thức thành viên xử lý mảng, nên các hàm Nhập/In cũng được đưa vào lớp nghiệp vụ thay vì để ở Program). Đây đều là các phương thức thể hiện (instance method), phải tạo đối tượng bằng new MangTools() trước khi gọi.

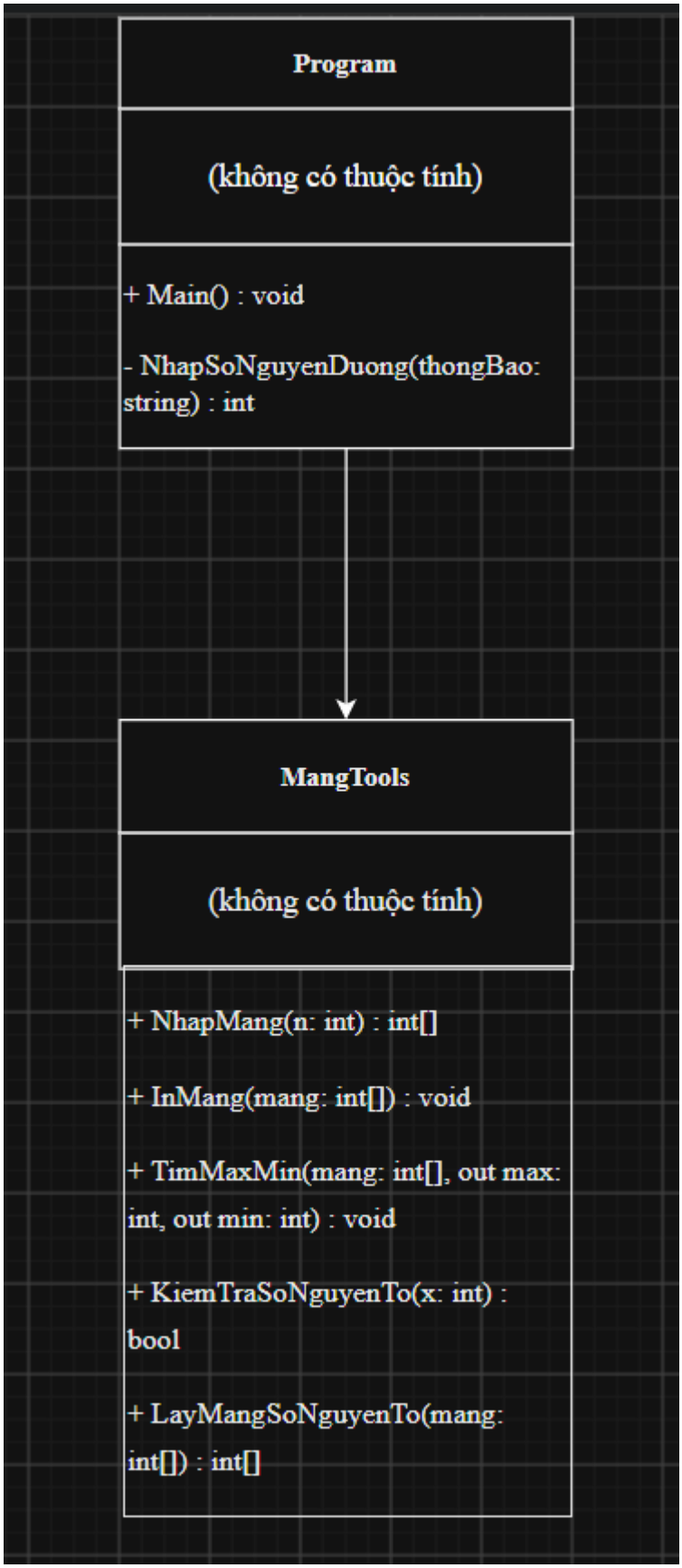
- Bước 1: Trong Main, nhập số phần tử n (dùng vòng lặp while(true) + TryParse, bắt lỗi và cho nhập lại nếu n <= 0 hoặc sai định dạng).
- Bước 2: Tạo đối tượng MangTools, gọi NhapMang(n) để nhập từng phần tử — mỗi phần tử cũng được bọc TryParse riêng để không bị crash khi nhập sai.
- Bước 3: Gọi InMang() để hiển thị mảng vừa nhập; hàm này xử lý cả trường hợp mảng rỗng để tránh in ra dòng trống khó hiểu.

- Bước 4: Gọi TimMaxMin(mang, out max, out min) — dùng out vì cần trả về 2 giá trị cùng lúc, không thể dùng return đơn thuần. Thuật toán: giả sử phần tử đầu là max/min, sau đó duyệt qua các phần tử còn lại để cập nhật.
- Bước 5: Gọi LayMangSoNguyenTo() để duyệt mảng gốc, kiểm tra từng phần tử bằng KiemTraSoNguyenTo() (chỉ xét ước số đến căn bậc hai để tối ưu tốc độ), gom các số nguyên tố vào một List<int> rồi trả về mảng mới bằng ToArray().

Điểm cần lưu ý: hàm TimMaxMin ném ngoại lệ ArgumentException nếu mảng rỗng, tránh lỗi IndexOutOfRange khó hiểu khi truy cập mang[0]. Đây cũng là trường hợp biên quan trọng được kiểm chứng ở phần test case bên dưới.

**2. Sơ đồ lớp (Class Diagram)**

Chương trình gồm 2 lớp: Program lo nhập số phần tử và điều phối chương trình; MangTools chứa toàn bộ các phương thức nghiệp vụ thao tác trên mảng.


Program
(không có thuộc tính)
+ Main() : void
- NhapSoNguyenDuong(thongBao: string) : int
MangTools
(không có thuộc tính)
+ NhapMang(n: int) : int[]
+ InMang(mang: int[]) : void
+ TimMaxMin(mang: int[], out max: int, out min: int) : void
+ KiemTraSoNguyenTo(x: int) : bool
+ LayMangSoNguyenTo(mang: int[]) : int[]

*Quan hệ giữa hai lớp: Program sử dụng (dependency) MangTools — cụ thể là tạo một đối tượng MangTools rồi gọi các phương thức nhập/in/tìm max-min/lọc số nguyên tố của đối tượng đó để thực hiện chương trình.*

**3. Code chương trình**

**File: MangTools.cs**

```
using System;
using System.Collections.Generic;

namespace Bai15_XuLyMang
{
    public class MangTools
    {
        // Nhap mang gom n phan tu
        // Dung while(true) + TryParse thay vi Parse de khong bi crash khi nhap sai dinh dang
        public int[] NhapMang(int n)
        {
            int[] mang = new int[n];
            for (int i = 0; i < n; i++)
            {
                while (true)
                {
                    Console.Write($"Nhap phan tu thu {i + 1}: ");
                    string? dong = Console.ReadLine(); // string? tranh canh bao CS8600
                    if (int.TryParse(dong, out int giaTri))
                    {
                        mang[i] = giaTri;
                        break;
```

```
                }
                Console.WriteLine("Loi: vui long nhap so nguyen hop le!");
            }
        }
        return mang;
    }

    // In mang ra man hinh, xu ly rieng truong hop mang rong
    public void InMang(int[] mang)
    {
        if (mang.Length == 0)
        {
            Console.WriteLine("(Mang rong)");
            return;
        }
        Console.WriteLine(string.Join(" ", mang));
    }

    // Tim dong thoi max va min, dung out vi can tra ve 2 gia tri cung luc
    public void TimMaxMin(int[] mang, out int max, out int min)
    {
        if (mang.Length == 0)
            throw new ArgumentException("Mang rong, khong the tim phan tu
lon nhat/nho nhat");

        max = mang[0];
        min = mang[0];
        for (int i = 1; i < mang.Length; i++)
        {
            if (mang[i] > max) max = mang[i];
            if (mang[i] < min) min = mang[i];
```

```
        }
    }

    // Kiem tra so nguyen to, chi xet uoc so den can bac hai de toi uu toc do
    public bool KiemTraSoNguyenTo(int x)
    {
        if (x < 2) return false;
        for (int i = 2; i * i <= x; i++)
        {
            if (x % i == 0) return false;
        }
        return true;
    }

    // Duyet mang goc, gom cac phan tu la so nguyen to vao mang moi
    public int[] LayMangSoNguyenTo(int[] mang)
    {
        List<int> ketQua = new List<int>();
        foreach (int x in mang)
        {
            if (KiemTraSoNguyenTo(x)) ketQua.Add(x);
        }
        return ketQua.ToArray();
    }
  }
}
```

**File: Program.cs**

```
using System;
```

```
namespace Bai15_XuLyMang
{
    class Program
    {
        static void Main()
        {
            int n = NhapSoNguyenDuong("Nhap so phan tu cua mang: ");
            MangTools tools = new MangTools(); // phai new doi tuong vi la
instance method

            int[] mang = tools.NhapMang(n);

            Console.WriteLine("Mang vua nhap:");
            tools.InMang(mang);

            tools.TimMaxMin(mang, out int max, out int min);
            Console.WriteLine($"Phan tu lon nhat: {max}");
            Console.WriteLine($"Phan tu nho nhat: {min}");

            int[] mangNguyenTo = tools.LayMangSoNguyenTo(mang);
            Console.WriteLine("Cac so nguyen to trong mang:");
            tools.InMang(mangNguyenTo);
        }

        // Nhap so phan tu mang, bat buoc phai > 0
        static int NhapSoNguyenDuong(string thongBao)
        {
            while (true)
            {
                Console.Write(thongBao);
                string? dong = Console.ReadLine();
```

```
            if (int.TryParse(dong, out int giaTri) && giaTri > 0)
                return giaTri;
            Console.WriteLine("Loi: vui long nhap so nguyen duong!");
        }
    }
  }
}
```

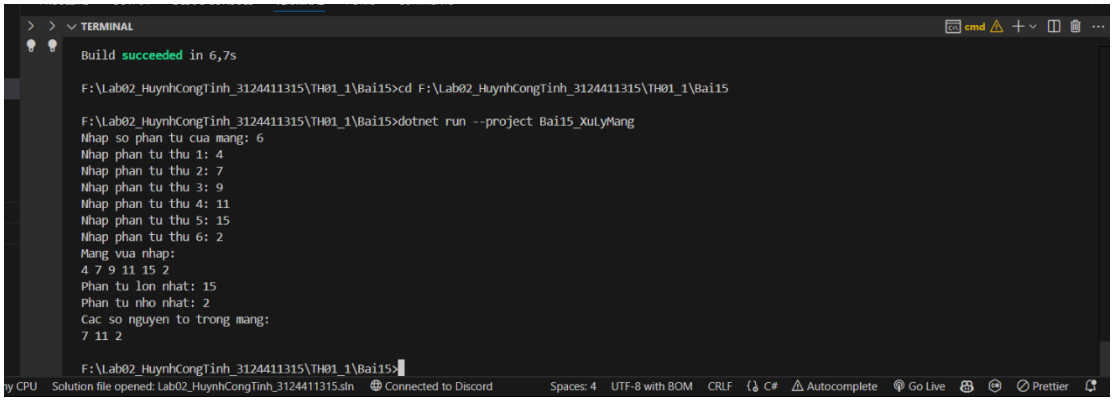
**4. Test case (Input / Output)**

Bộ 11 unit test (MSTest) phủ các trường hợp: mảng bình thường, mảng 1 phần tử, mảng toàn số âm, mảng rỗng (trường hợp biên quan trọng nhất của TimMaxMin), các trường hợp số nguyên tố / không nguyên tố / số biên (0, 1, số âm), và kiểm chứng định dạng in ra màn hình của InMang.

| STT | Input | Output mong đợi | Ghi chú |
|---|---|---|---|
| 1 | mang = {5, 1, 9, 3, 7} | max = 9, min = 1 | Trường hợp mảng bình thường |
| 2 | mang = {42} | max = 42, min = 42 | Mảng chỉ 1 phần tử, max = min |
| 3 | mang = {-5, -1, -9, -3} | max = -1, min = -9 | Mảng toàn số âm |
| 4 | mang = {} (rỗng) | Ném ArgumentException | Trường hợp biên: mảng rỗng phải báo lỗi rõ ràng, không crash IndexOutOfRange |
| 5 | x = 2, 13, 97 | true | Số nguyên tố (2 là biên dưới của số |

| STT | Input | Output mong đợi | Ghi chú |
|---|---|---|---|
| | | | nguyên tố) |
| 6 | x = 4, 100 | false | Hợp số |
| 7 | x = -7, 0, 1 | false | Trường hợp biên: số âm, 0, 1 không phải số nguyên tố |
| 8 | mang = {4, 7, 9, 11, 15, 2} | {7, 11, 2} | Lọc đúng các số nguyên tố, giữ nguyên thứ tự xuất hiện |
| 9 | mang = {4, 6, 8, 9, 10} | mảng rỗng (length = 0) | Không có số nguyên tố nào trong mảng |
| 10 | mang = {1, 2, 3} | In ra: "1 2 3" | Kiểm tra định dạng in mảng (InMang) |
| 11 | mang = {} (rỗng) | In ra: "(Mang rong)" | Kiểm tra InMang xử lý đúng mảng rỗng |

**Kết quả sau khi thực hiện:**




## * Mảng, ArrayList

**Bài 16:** Nhập vào một mảng họ tên của n người. Hãy sắp xếp mảng đó theo thứ tự tăng dần

## 2. Ý tưởng giải bài toán

Tách riêng một lớp MangHoTen chứa các phương thức thao tác trên mảng chuỗi họ tên (nhập, in, sắp xếp). Đây đều là các phương thức thể hiện (instance method), phải tạo đối tượng bằng new MangHoTen() trước khi gọi.

- Bước 1: Trong Main, nhập số người n (dùng vòng lặp while(true) + TryParse, bắt lỗi và cho nhập lại nếu n <= 0 hoặc sai định dạng).
- Bước 2: Tạo đối tượng MangHoTen, gọi NhapMang(n) để nhập từng họ tên — kiểm tra chuỗi rỗng/toàn khoảng trắng bằng string.IsNullOrWhiteSpace thay vì TryParse số, vì họ tên là chuỗi và có thể chứa khoảng trắng.
- Bước 3: Gọi InMang() để hiển thị danh sách trước khi sắp xếp, có đánh số thứ tự cho dễ nhìn.
- Bước 4: Gọi SapXepTangDan(mang) — tự viết thuật toán Selection Sort: mỗi lượt tìm vị trí có tên "nhỏ nhất" theo thứ tự từ điển trong phần

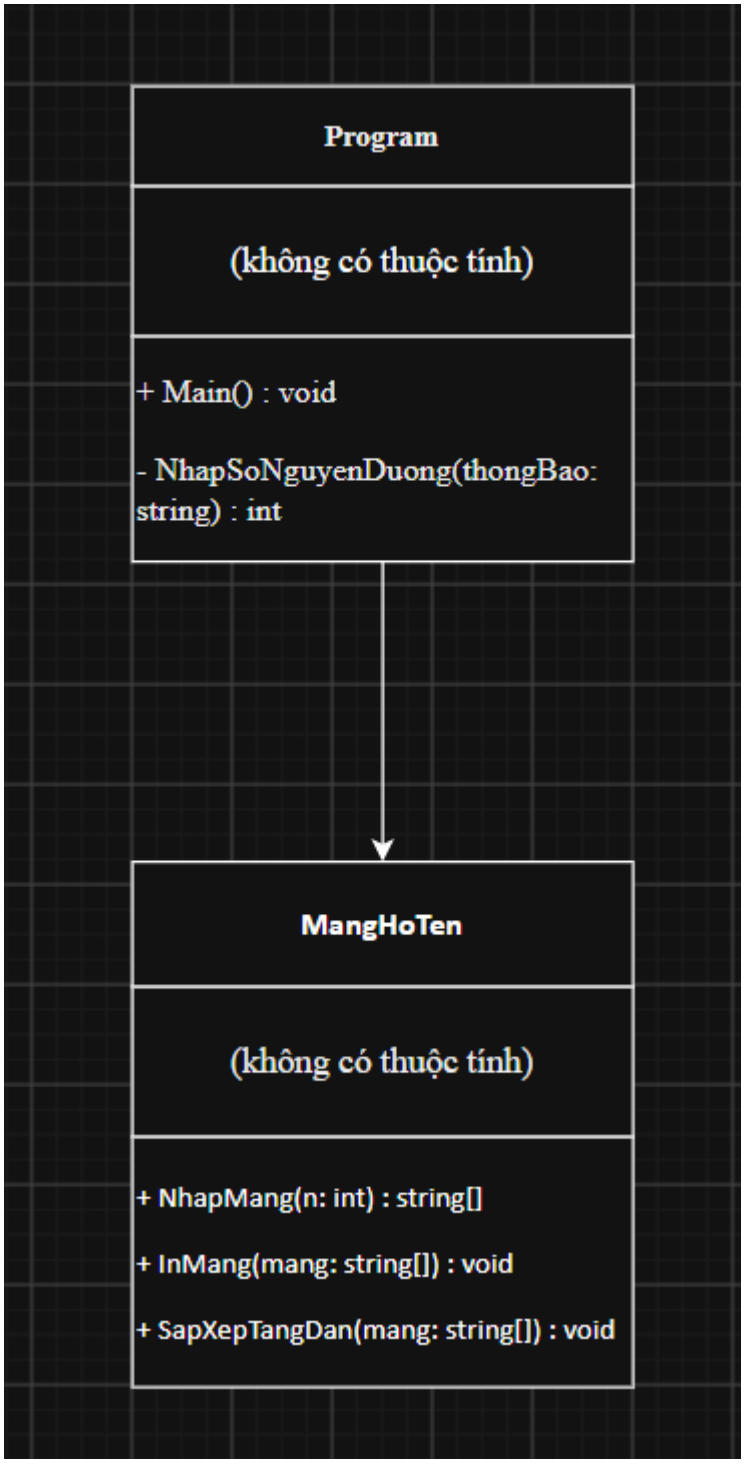
chưa sắp xếp, rồi hoán đổi vào đầu. So sánh bằng string.Compare(..., StringComparison.OrdinalIgnoreCase) để không phân biệt chữ hoa/thường.

- Bước 5: In lại mảng sau khi sắp xếp để đối chiếu kết quả trước/sau.

Điểm cần lưu ý: mảng string là kiểu tham chiếu, nên khi truyền mang vào SapXepTangDan(), hàm thao tác trực tiếp trên mảng gốc (không cần return hay out) — sắp xếp xong, biến mang ở Main đã tự thay đổi theo. Đây là điểm khác biệt so với các bài trước dùng kiểu int (tham trị).

### 3. Sơ đồ lớp (Class Diagram)

Chương trình gồm 2 lớp: Program lo nhập số người và điều phối chương trình; MangHoTen chứa toàn bộ các phương thức nghiệp vụ thao tác trên mảng họ tên.




*Quan hệ giữa hai lớp: Program sử dụng (dependency) MangHoTen — cụ thể là tạo một đối tượng MangHoTen rồi gọi các phương thức nhập/in/sắp xếp của đối tượng đó để thực hiện chương trình.*

**4. Code chương trình**

**File: MangHoTen.cs**

```
using System;

namespace Bai16_SapXepHoTen
{
    public class MangHoTen
    {
        // Nhap mang ho ten gom n nguoi
        // Ho ten co the chua khoang trang nen khong dung TryParse so, chi kiem
tra chuoi rong/trang
        public string[] NhapMang(int n)
        {
            string[] mang = new string[n];
            for (int i = 0; i < n; i++)
            {
                while (true)
                {
                    Console.Write($"Nhap ho ten nguoi thu {i + 1}: ");
                    string? dong = Console.ReadLine(); // string? tranh canh bao
CS8600
                    if (!string.IsNullOrWhiteSpace(dong))
                    {
                        mang[i] = dong.Trim(); // Trim de bo khoang trang thua
dau/cuoi
                        break;
                    }
                    Console.WriteLine("Loi: ho ten khong duoc de trong!");
                }
            }
            return mang;
        }
```

```
// In mang ra man hinh, moi ten 1 dong, co danh so thu tu cho de nhin
public void InMang(string[] mang)
{
    if (mang.Length == 0)
    {
        Console.WriteLine("(Mang rong)");
        return;
    }
    for (int i = 0; i < mang.Length; i++)
    {
        Console.WriteLine($"{i + 1}. {mang[i]}");
    }
}

// Sap xep tang dan theo thu tu tu dien (a-z) bang thuat toan Selection Sort
// Tu viet thuat toan thay vi dung Array.Sort de the hien ro cach sap xep,
// dung string.Compare (khong phan biet hoa/thuong) de so sanh 2 ten
public void SapXepTangDan(string[] mang)
{
    for (int i = 0; i < mang.Length - 1; i++)
    {
        int viTriNhoNhat = i;
        for (int j = i + 1; j < mang.Length; j++)
        {
            if (string.Compare(mang[j], mang[viTriNhoNhat],
                StringComparison.OrdinalIgnoreCase) < 0)
            {
                viTriNhoNhat = j;
            }
        }
        if (viTriNhoNhat != i)
```

```
            {
                // Hoan doi 2 phan tu bang bien tam
                string tam = mang[i];
                mang[i] = mang[viTriNhoNhat];
                mang[viTriNhoNhat] = tam;
            }
        }
    }
}
```

**File: Program.cs**

```
using System;

namespace Bai16_SapXepHoTen
{
    class Program
    {
        static void Main()
        {
            int n = NhapSoNguyenDuong("Nhap so nguoi: ");
            MangHoTen tools = new MangHoTen(); // phai new doi tuong vi la
instance method

            string[] mang = tools.NhapMang(n);

            Console.WriteLine("\nDanh sach truoc khi sap xep:");
            tools.InMang(mang);
```

```
            tools.SapXepTangDan(mang); // sap xep truc tiep tren mang (khong tao
mang moi)

            Console.WriteLine("\nDanh sach sau khi sap xep tang dan:");
            tools.InMang(mang);
        }

        // Nhap so nguoi, bat buoc phai > 0
        static int NhapSoNguyenDuong(string thongBao)
        {
            while (true)
            {
                Console.Write(thongBao);
                string? dong = Console.ReadLine();
                if (int.TryParse(dong, out int giaTri) && giaTri > 0)
                    return giaTri;
                Console.WriteLine("Loi: vui long nhap so nguyen duong!");
            }
        }
    }
}
```

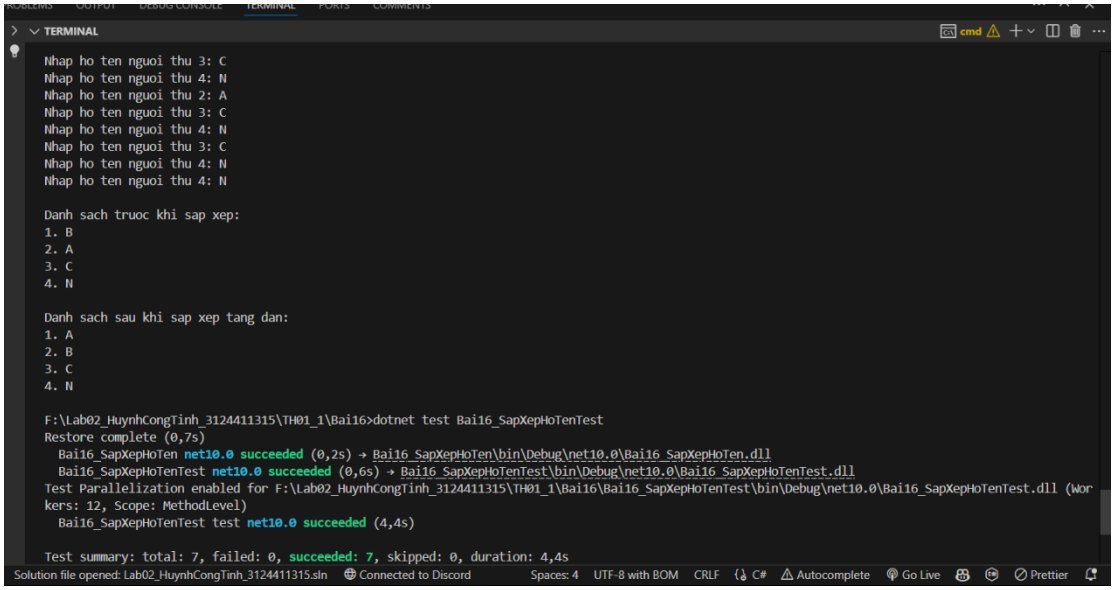
**4. Test case (Input / Output)**

Bộ 7 unit test (MSTest) phủ các trường hợp: mảng bình thường, mảng đã sắp xếp sẵn, mảng ngược thứ tự, mảng 1 phần tử, mảng rỗng, mảng có tên trùng nhau, và trường hợp tên viết hoa/thường xen kẽ.

| STT | Input | Output mong đợi | Ghi chú |
|---|---|---|---|
| 1 | {"Tran Van B", "Nguyen Van A", "Le Thi C"} | {"Le Thi C", "Nguyen Van A", "Tran Van B"} | Trường hợp mảng bình thường |

| STT | Input | Output mong đợi | Ghi chú |
|---|---|---|---|
| 2 | {"An", "Binh", "Chi"} | {"An", "Binh", "Chi"} | Mảng đã sắp xếp sẵn, kết quả không đổi |
| 3 | {"Chi", "Binh", "An"} | {"An", "Binh", "Chi"} | Mảng ngược thứ tự, phải sắp xếp lại đúng |
| 4 | {"MotMinh"} | {"MotMinh"} | Trường hợp biên: mảng 1 phần tử, không cần sắp xếp |
| 5 | {} (mảng rỗng) | {} (length = 0), không lỗi | Trường hợp biên: mảng rỗng không được gây lỗi khi sắp xếp |
| 6 | {"Binh", "An", "An"} | {"An", "An", "Binh"} | Trường hợp biên: tên trùng nhau, không mất phần tử |
| 7 | {"binh", "An", "chi"} | {"An", "binh", "chi"} | Kiểm chứng không phân biệt chữ hoa/thường (OrdinalIgnoreCase) |

**Kết quả sau khi thực hiện:**




## * Mảng, ArrayList

**Bài 17:** Viết các phương thức thành viên sau:

- Sinh ngẫu nhiên mảng A[nxm] trong đoạn [10, 100] (n,m nhập từ bàn phím)
- In mảng ra màn hình
- Trả về hai mảng: mảng các số chẵn và mảng các số lẻ

--- Hết ---

### 1. Ý tưởng giải bài toán

Tách riêng một lớp MangHaiChieu chứa các phương thức thao tác trên mảng 2 chiều int[,] (sinh ngẫu nhiên, in, lọc chẵn/lẻ). Đây đều là các phương thức thể hiện (instance method), phải tạo đối tượng bằng new MangHaiChieu() trước khi gọi.

- Bước 1: Trong Main, nhập số hàng n và số cột m (dùng vòng lặp while(true) + TryParse, bắt lỗi và cho nhập lại nếu <= 0 hoặc sai định dạng).
- Bước 2: Tạo đối tượng MangHaiChieu, gọi SinhNgauNhien(n, m) để sinh mảng 2 chiều với giá trị mỗi phần tử trong đoạn [10, 100] bằng Random.Next(10, 101) (cận trên +1 vì Next loại trừ cận trên). Đối tượng Random được tạo 1 lần duy nhất trong constructor của lớp — tránh lỗi

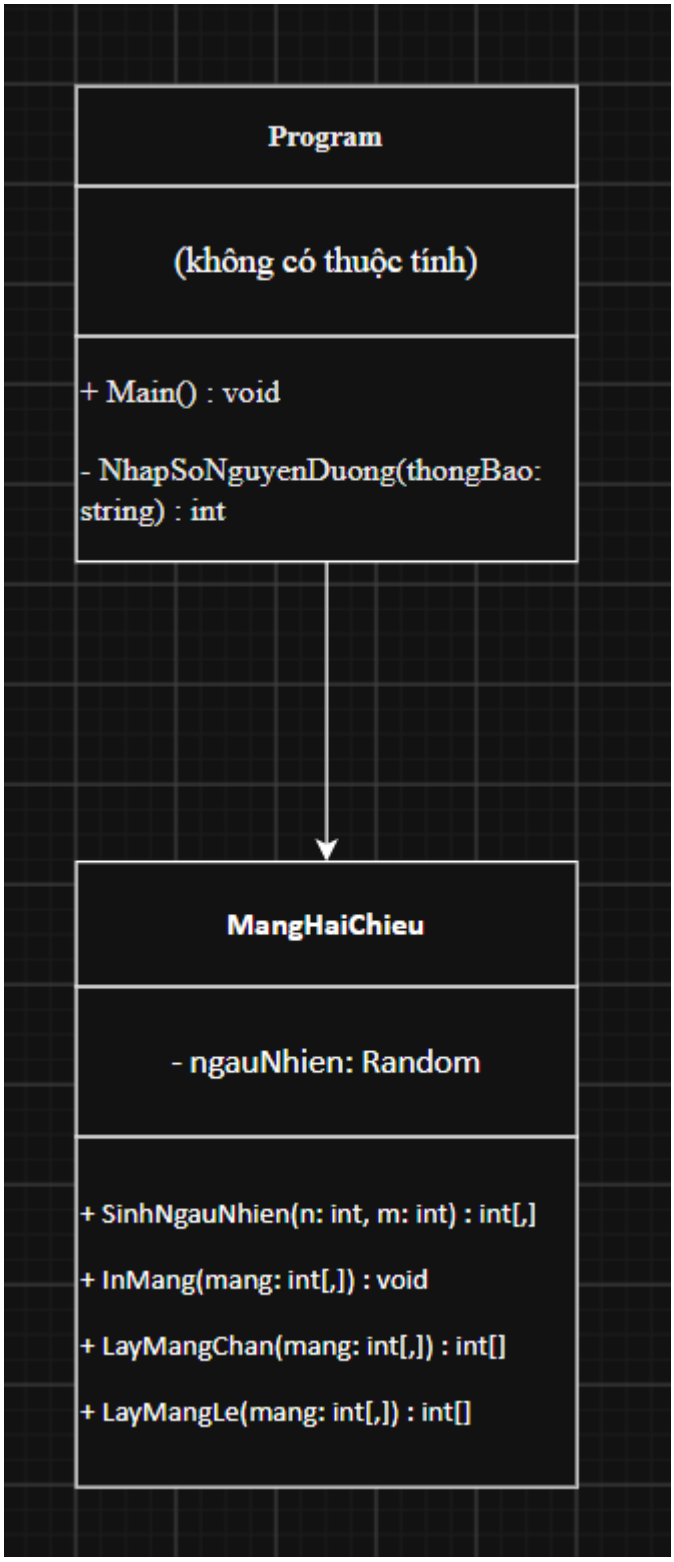
hay gặp là new Random() liên tục trong vòng lặp cho ra các số giống hệt nhau do cùng seed thời gian.

- Bước 3: Gọi InMang() để hiển thị mảng theo dạng bảng — mỗi hàng 1 dòng, các phần tử cách nhau bằng tab.
- Bước 4: Gọi LayMangChan() và LayMangLe() — duyệt qua toàn bộ 2 chiều của mảng bằng vòng lặp lồng nhau, kiểm tra phần dư khi chia 2, gom kết quả vào List<int> rồi trả về mảng 1 chiều bằng ToArray().

Điểm cần lưu ý: dùng int[,] (mảng 2 chiều thật sự / rectangular array) thay vì mảng jagged (int[][]) vì kích thước n, m đã cố định ngay từ đầu và không có hàng nào có độ dài khác nhau. GetLength(0) trả về số hàng, GetLength(1) trả về số cột.

**2. Sơ đồ lớp (Class Diagram)**

Chương trình gồm 2 lớp: Program lo nhập n, m và điều phối chương trình; MangHaiChieu chứa toàn bộ các phương thức nghiệp vụ thao tác trên mảng 2 chiều.




*Quan hệ giữa hai lớp: Program sử dụng (dependency) MangHaiChieu — cụ thể là tạo một đối tượng MangHaiChieu rồi gọi các phương thức sinh ngẫu nhiên/in/lọc chẵn/lọc lẻ của đối tượng đó để thực hiện chương trình.*

**3. Code chương trình**

**File: MangHaiChieu.cs**

```
using System;
using System.Collections.Generic;

namespace Bai17_MangHaiChieu
{
    public class MangHaiChieu
    {
        // Dung 1 Random duy nhat cho ca lop, tao 1 lan trong constructor
        // (tranh loi hay gap: new Random() lien tuc trong vong lap se cho ket qua
        // giong nhau do cung seed thoi gian)
        private readonly Random ngauNhien;

        public MangHaiChieu()
        {
            ngauNhien = new Random();
        }

        // Sinh ngau nhien mang A[n x m], gia tri trong doan [10, 100]
        // Dung int[,] (mang 2 chieu that su) vi n, m co kich thuoc co dinh tu dau
        public int[,] SinhNgauNhien(int n, int m)
        {
            int[,] mang = new int[n, m];
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < m; j++)
                {
                    // Random.Next(min, max) can tren bi loai tru, nen +1 de 100 cung
xuat hien duoc
                    mang[i, j] = ngauNhien.Next(10, 101);
                }
```

```
        }
        return mang;
    }

    // In mang ra man hinh, moi hang 1 dong, cac phan tu cach nhau bang tab cho thang cot
    public void InMang(int[,] mang)
    {
        int n = mang.GetLength(0);
        int m = mang.GetLength(1);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                Console.Write(mang[i, j] + "\t");
            }
            Console.WriteLine();
        }
    }

    // Duyet toan bo mang 2 chieu, gom cac phan tu chan vao mang 1 chieu
    public int[] LayMangChan(int[,] mang)
    {
        List<int> ketQua = new List<int>();
        int n = mang.GetLength(0);
        int m = mang.GetLength(1);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                if (mang[i, j] % 2 == 0) ketQua.Add(mang[i, j]);
```

```
            }
        }
        return ketQua.ToArray();
    }

    // Duyet toan bo mang 2 chieu, gom cac phan tu le vao mang 1 chieu
    public int[] LayMangLe(int[,] mang)
    {
        List<int> ketQua = new List<int>();
        int n = mang.GetLength(0);
        int m = mang.GetLength(1);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                if (mang[i, j] % 2 != 0) ketQua.Add(mang[i, j]);
            }
        }
        return ketQua.ToArray();
    }
  }
}
```

**File: Program.cs**

```
using System;

namespace Bai17_MangHaiChieu
{
  class Program
  {
```

```
    static void Main()
    {
        int n = NhapSoNguyenDuong("Nhap so hang n: ");
        int m = NhapSoNguyenDuong("Nhap so cot m: ");

        MangHaiChieu tools = new MangHaiChieu(); // phai new doi tuong vi
la instance method

        int[,] mang = tools.SinhNgauNhien(n, m);

        Console.WriteLine("\nMang A vua sinh ngau nhien:");
        tools.InMang(mang);

        int[] mangChan = tools.LayMangChan(mang);
        int[] mangLe = tools.LayMangLe(mang);

        Console.WriteLine("\nCac so chan trong mang:");
        Console.WriteLine(mangChan.Length > 0 ? string.Join(" ", mangChan)
: "(Khong co)");

        Console.WriteLine("\nCac so le trong mang:");
        Console.WriteLine(mangLe.Length > 0 ? string.Join(" ", mangLe) :
"(Khong co)");
    }

    // Nhap so nguyen duong, lap lai neu nhap sai dinh dang hoac <= 0
    static int NhapSoNguyenDuong(string thongBao)
    {
        while (true)
        {
            Console.Write(thongBao);
```

```
            string? dong = Console.ReadLine();
            if (int.TryParse(dong, out int giaTri) && giaTri > 0)
                return giaTri;
            Console.WriteLine("Loi: vui long nhap so nguyen duong!");
        }
    }
  }
}
```

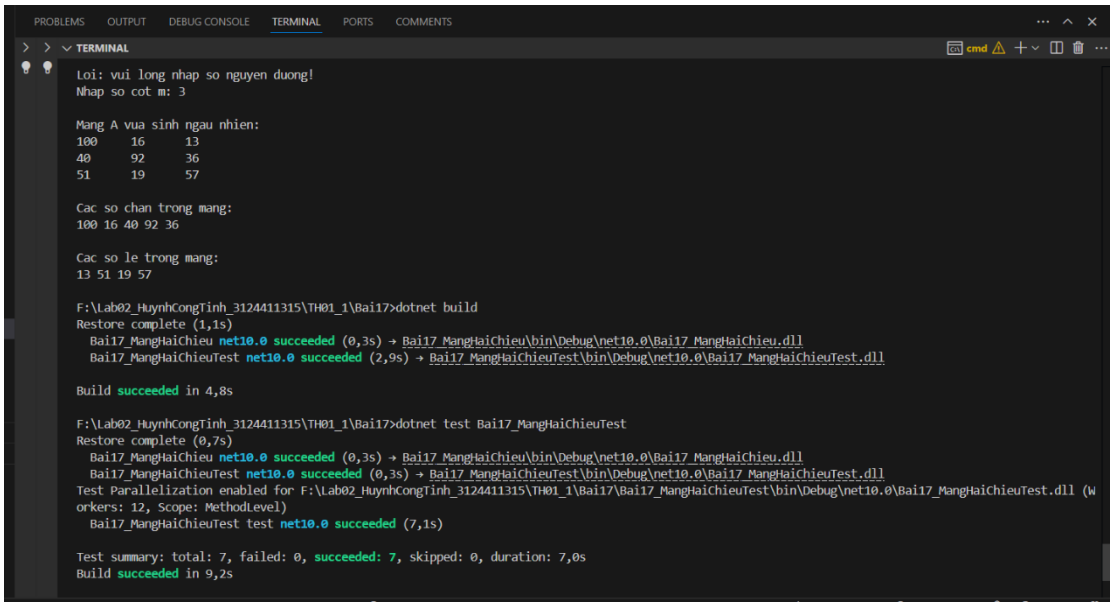
**4. Test case (Input / Output)**

Bộ 7 unit test (MSTest) phủ các trường hợp: mảng 2 chiều bình thường, mảng không có số chẵn, mảng không có số lẻ, kiểm chứng tổng số phần tử không bị mất/trùng, trường hợp mảng 1x1, và kiểm tra SinhNgauNhien luôn trả về đúng kích thước n×m với giá trị nằm trong đoạn [10, 100].

| STT | Input | Output mong đợi | Ghi chú |
|---|---|---|---|
| 1 | {{1,2,3},{4,5,6}} | Chẵn: {2,4,6} | Lọc số chẵn trên mảng 2 chiều bình thường |
| 2 | {{1,2,3},{4,5,6}} | Lẻ: {1,3,5} | Lọc số lẻ trên mảng 2 chiều bình thường |
| 3 | {{1,3},{5,7}} | Chẵn: {} (length = 0) | Trường hợp biên: mảng toàn số lẻ, không có số chẵn nào |
| 4 | {{2,4},{6,8}} | Lẻ: {} (length = 0) | Trường hợp biên: mảng toàn số chẵn, không có số lẻ nào |

| STT | Input | Output mong đợi | Ghi chú |
|---|---|---|---|
| 5 | mảng 3×4 (12 phần tử, giá trị 1..12) | chan.Length + le.Length = 12 | Kiểm chứng không mất/trùng phần tử khi tách chẵn-lẻ |
| 6 | {{8}} (1×1) | Chẵn: {8} | Trường hợp biên: mảng chỉ có 1 phần tử |
| 7 | n=5, m=4 | Kích thước đúng 5×4, mọi phần tử □ [10,100] | Kiểm tra SinhNgauNhien đúng kích thước và đúng khoảng giá trị |

Kết quả sau khi thực hiện:



# CHỦ ĐỀ BUỔI 01_2: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

## I. MỤC TIÊU THỰC HÀNH

- Tạo Solution và Class Library bằng dotnet CLI.
- Viết thư viện C# và gọi thư viện từ project khác thông qua ProjectReference.
- Hiểu lỗi namespace/assembly reference và cách khắc phục.
- Biết xử lý trường hợp một project có nhiều hàm Main().
- Tạo project xUnit và viết unit test cho hàm giải phương trình bậc hai.
- Chạy build, run và test bằng dotnet CLI trong VS Code/Terminal.

## II. NỘI DUNG THỰC HIỆN

### Bài 1: Tạo cấu trúc dự án

Mục tiêu: tạo cấu trúc thư mục dự án thực hành gồm 3 project con nằm trong cùng một Solution:

```
NNLTCSharpC6/
├── NNLTCSharpC6.sln
├── MyLib/
│   ├── MyLib.csproj
│   └── LibBaiTap.cs
├── Buoi01Prj/
│   ├── Buoi01Prj.csproj
│   └── GiaiPTBac2.cs
└── MyLib.Tests/
    ├── MyLib.Tests.csproj
    └── LibBaiTapTests.cs
```

**Bước 1 — Tạo Solution**

```
mkdir NNLTCSharpC6
cd NNLTCSharpC6
dotnet new sln -n NNLTCSharpC6
```

**Bước 2 — Tạo Class Library MyLib**

```
dotnet new classlib -n MyLib
dotnet sln NNLTCSharpC6.sln add MyLib/MyLib.csproj
dotnet build MyLib/MyLib.csproj
```

**Bài 2: Xây dựng thư viện giải phương trình bậc hai**

Mục tiêu: xây dựng class library gồm namespace, public class, static method và tham số truyền theo kiểu ref.

**File MyLib/LibBaiTap.cs**

```
namespace MyLib;

public class LibBaiTap
{
    /// <summary>
    /// Giai phuong trinh bac 2
    /// </summary>
    /// <returns>{-1, 0, 1, 2}</returns>
    public static int GiaiPTBac2(
        double a, double b, double c,
        ref double x1, ref double x2)
    {
```

```
int sn;
if (a == 0)
{
    if (b == 0)
    {
        if (c == 0) sn = -1;
        else sn = 0;
    }
    else
    {
        sn = 1;
        x1 = -c / b;
    }
}
else
{
    double delta = b * b - 4 * a * c;
    if (delta < 0) sn = 0;
    else if (delta == 0)
    {
        sn = 1;
        x1 = -b / (2 * a);
    }
    else
    {
        sn = 2;
        x1 = (-b - Math.Sqrt(delta)) / (2 * a);
        x2 = (-b + Math.Sqrt(delta)) / (2 * a);
        if (x1 > x2)
        {
            double tmp = x1;
```

```
                x1 = x2;
                x2 = tmp;
            }
        }
    }
    return sn;
  }
}
```

Ý nghĩa giá trị trả về:

| Giá trị | Ý nghĩa |
|---|---|
| -1 | Vô số nghiệm (a = 0, b = 0, c = 0) |
| 0 | Vô nghiệm (a = 0, b = 0, c ≠ 0 hoặc delta < 0) |
| 1 | Một nghiệm (phương trình bậc nhất, hoặc delta = 0) |
| 2 | Hai nghiệm phân biệt (delta > 0), luôn được sắp x1 ≤ x2 |

### Bài 3: Tạo project Console để chạy thử thư viện

#### Tạo project và thêm ProjectReference

```
dotnet new console -n Buoi01Prj
dotnet sln NNLTCSharpC6.sln add Buoi01Prj/Buoi01Prj.csproj
dotnet add Buoi01Prj/Buoi01Prj.csproj reference MyLib/MyLib.csproj
dotnet list Buoi01Prj/Buoi01Prj.csproj reference
```

Nội dung tương ứng được VS thêm vào file Buoi01Prj.csproj:

```
<ItemGroup>
```

```
  <ProjectReference Include="..\MyLib\MyLib.csproj" />
</ItemGroup>
```

**Gọi hàm từ Class Library — file Buoi01Prj/GiaiPTBac2.cs**

```
using MyLib;

namespace Buoi01Prj;

public class GiaiPTBac2
{
    public static void Main(string[] args)
    {
        double x1 = 0, x2 = 0;
        int sn = LibBaiTap.GiaiPTBac2(1, -3, 2, ref x1, ref x2);

        Console.WriteLine($"So nghiem: {sn}");
        Console.WriteLine($"x1 = {x1}");
        Console.WriteLine($"x2 = {x2}");
    }
}
```

**Chạy chương trình**

```
dotnet run --project Buoi01Prj
```

Kết quả nhận được:

```
So nghiem: 2
x1 = 1
x2 = 2
```

### Bài 4: Một project có nhiều hàm Main()

Mục tiêu: hiểu khái niệm entry point và cách chỉ định StartupObject khi project có nhiều hơn một hàm Main().

#### Tạo hai hàm Main() và quan sát lỗi

```
namespace Buoi01Prj;

public class Program1
{
    public static void Main(string[] args)
    {
        Console.WriteLine("Main 1");
    }
}

public class Program2
{
    public static void Main(string[] args)
    {
        Console.WriteLine("Main 2");
    }
}
```

#### Khắc phục bằng StartupObject trong file .csproj

```
<PropertyGroup>
  <OutputType>Exe</OutputType>
  <TargetFramework>net8.0</TargetFramework>
  <StartupObject>Buoi01Prj.Program2</StartupObject>
</PropertyGroup>
```

#### Cách tổ chức nhiều bài thực hành trong cùng một project (khuyến nghị)

```
namespace Buoi01Prj;

class Program
{
    static void Main(string[] args)
    {
        Bai1.Run();
        // Bai2.Run();
    }
}

class Bai1
{
    public static void Run()
    {
        Console.WriteLine("Chay bai 1");
    }
}
```

**Khắc phục lỗi không nhận namespace MyLib**

Đây là lỗi cố tình tạo ra theo yêu cầu bài: đổi using MyLib; thành using MyLib.Buoi01; (một namespace không tồn tại).

```
error CS0234: The type or namespace name 'Buoi01' does not exist in the namespace
'MyLib' (are you missing an assembly reference?)
```

- Nếu thư viện khai báo namespace MyLib; thì project sử dụng phải viết using MyLib;, không tự thêm hậu tố (ví dụ MyLib.Buoi01) nếu namespace con đó không tồn tại trong thư viện.

- Class trong thư viện phải được khai báo public nếu project khác cần truy cập.
- Kiểm tra ProjectReference đã được thêm đúng hay chưa bằng lệnh: dotnet list Buoi01Prj/Buoi01Prj.csproj reference.
- Nếu chưa có MyLib trong danh sách reference, thêm lại bằng dotnet add ... reference rồi chạy dotnet clean và dotnet build.
- Có thể gọi trực tiếp bằng tên đầy đủ (fully-qualified name) mà không cần using: int sn = MyLib.LibBaiTap.GiaiPTBac2(1, -3, 2, ref x1, ref x2);

**Bài 5: Unit Test với xUnit**

**Tạo project MyLib.Tests**

```
dotnet new xunit -n MyLib.Tests
dotnet sln NNLTCSharpC6.sln add MyLib.Tests/MyLib.Tests.csproj
dotnet add MyLib.Tests/MyLib.Tests.csproj reference MyLib/MyLib.csproj
```

**Viết unit test — file MyLib.Tests/LibBaiTapTests.cs**

```
using MyLib;
using Xunit;

namespace MyLib.Tests;

public class LibBaiTapTests
{
    [Fact]
    public void GiaiPTBac2_A0_B0_C0_VoSoNghiem()
    {
        double x1 = 0, x2 = 0;
        int result = LibBaiTap.GiaiPTBac2(0, 0, 0, ref x1, ref x2);
        Assert.Equal(-1, result);
    }
```

```
[Fact]
public void GiaiPTBac2_A0_B0_CKhac0_VoNghiem()
{
    double x1 = 0, x2 = 0;
    int result = LibBaiTap.GiaiPTBac2(0, 0, 5, ref x1, ref x2);
    Assert.Equal(0, result);
}

[Fact]
public void GiaiPTBac2_PhuongTrinhBac1_CoMotNghiem()
{
    double x1 = 0, x2 = 0;
    int result = LibBaiTap.GiaiPTBac2(0, 2, -4, ref x1, ref x2);
    Assert.Equal(1, result);
    Assert.Equal(2, x1);
}

[Fact]
public void GiaiPTBac2_DeltaNhoHon0_VoNghiem()
{
    double x1 = 0, x2 = 0;
    int result = LibBaiTap.GiaiPTBac2(1, 0, 1, ref x1, ref x2);
    Assert.Equal(0, result);
}

[Fact]
public void GiaiPTBac2_DeltaBang0_CoNghiemKep()
{
    double x1 = 0, x2 = 0;
    int result = LibBaiTap.GiaiPTBac2(1, -2, 1, ref x1, ref x2);
```

```
        Assert.Equal(1, result);
        Assert.Equal(1, x1);
    }

    [Fact]
    public void GiaiPTBac2_DeltaLonHon0_CoHaiNghiem()
    {
        double x1 = 0, x2 = 0;
        int result = LibBaiTap.GiaiPTBac2(1, -3, 2, ref x1, ref x2);
        Assert.Equal(2, result);
        Assert.Equal(1, x1);
        Assert.Equal(2, x2);
    }

    [Fact]
    public void GiaiPTBac2_HaiNghiem_LuonDuocSapXepTangDan()
    {
        double x1 = 0, x2 = 0;
        int result = LibBaiTap.GiaiPTBac2(-1, 3, -2, ref x1, ref x2);
        Assert.Equal(2, result);
        Assert.Equal(1, x1);
        Assert.Equal(2, x2);
        Assert.True(x1 <= x2);
    }
}
```

**Chạy Unit Test**

```
dotnet test
# Hoặc chỉ chạy project test:
dotnet test MyLib.Tests/MyLib.Tests.csproj
```

Bảng các trường hợp kiểm thử tối thiểu (test case) tương ứng với 7 [Fact] ở trên:

| STT | a | b | c | Kết quả mong đợi | Mục đích |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | -1 | Vô số nghiệm |
| 2 | 0 | 0 | 5 | 0 | PT suy biến vô nghiệm |
| 3 | 0 | 2 | -4 | 1; x1 = 2 | Phương trình bậc nhất |
| 4 | 1 | 0 | 1 | 0 | Delta < 0 |
| 5 | 1 | -2 | 1 | 1; x1 = 1 | Delta = 0 |
| 6 | 1 | -3 | 2 | 2; x1 = 1, x2 = 2 | Delta > 0 |
| 7 | -1 | 3 | -2 | 2; x1 = 1, x2 = 2 | Kiểm tra hoán đổi nghiệm |

## III. NÂNG CẤP HÀM SO SÁNH SỐ THỰC BẰNG EPSILON

Trong bài thực hành cơ bản, việc so sánh a == 0 và delta == 0 trên kiểu double có thể chấp nhận được với dữ liệu nhập tay. Tuy nhiên trong chương trình thực tế, số thực có sai số biểu diễn (floating-point representation error), nên cần so sánh với một epsilon nhỏ thay vì so sánh tuyệt đối.

```
namespace MyLib;

public class LibBaiTap
{
    private const double EPS = 1e-9;

    /// <summary>
```

```
/// Giai phuong trinh bac 2 (ban nang cap, so sanh double bang EPS)
/// </summary>
/// <returns>{-1, 0, 1, 2}</returns>
public static int GiaiPTBac2(
    double a, double b, double c,
    ref double x1, ref double x2)
{
    int sn;
    if (Math.Abs(a) < EPS)
    {
        if (Math.Abs(b) < EPS)
        {
            if (Math.Abs(c) < EPS) sn = -1;
            else sn = 0;
        }
        else
        {
            sn = 1;
            x1 = -c / b;
        }
    }
    else
    {
        double delta = b * b - 4 * a * c;
        if (delta < -EPS) sn = 0;
        else if (Math.Abs(delta) < EPS)
        {
            sn = 1;
            x1 = -b / (2 * a);
        }
        else
```

```
            {
                sn = 2;
                x1 = (-b - Math.Sqrt(delta)) / (2 * a);
                x2 = (-b + Math.Sqrt(delta)) / (2 * a);
                if (x1 > x2)
                {
                    double tmp = x1;
                    x1 = x2;
                    x2 = tmp;
                }
            }
        }
        return sn;
    }
}
```

**Unit test bổ sung cho trường hợp sai số làm tròn**

```
[Fact]
public void
GiaiPTBac2_DeltaXapXi0_DoSaiSoLamTron_VanTraVeNghiemKep()
{
    // a, b, c được chọn để b*b - 4*a*c cho ra một số cực nhỏ khác 0
    // do sai số biểu diễn dấu phẩy động, thay vì đúng bằng 0 tuyệt đối.
    double a = 1.0, b = -2.0, c = 1.0 + 1e-16;
    double x1 = 0, x2 = 0;

    int result = LibBaiTap.GiaiPTBac2(a, b, c, ref x1, ref x2);

    Assert.Equal(1, result);
    Assert.Equal(1.0, x1, 6); // so khop den 6 chu so thap phan
```

```
}
```

## IV. QUY TRÌNH KIỂM TRA TOÀN BỘ BÀI

1. Ở thư mục gốc Solution, chạy dotnet build để build toàn bộ 3 project cùng lúc.
2. Chạy dotnet run --project Buoi01Prj để kiểm tra chương trình chạy thử (UAT).
3. Chạy dotnet test để chạy toàn bộ các unit test xUnit trong MyLib.Tests.
4. Nếu lỗi namespace: kiểm tra lại khai báo namespace, từ khoá public trên class/method, và ProjectReference giữa các project.
5. Nếu lỗi nhiều entry point (CS0017): cấu hình lại StartupObject trong .csproj, hoặc gom các bài về một hàm Main() duy nhất.

## V. TRẢ LỜI CÂU HỎI ÔN TẬP

### 1. Class Library khác Console Application ở điểm nào?

Class Library (dotnet new classlib) biên dịch ra file .dll, không có entry point (Main) và không thể tự chạy độc lập — nó chỉ chứa các lớp, hàm dùng để cung cấp chức năng cho project khác tham chiếu tới thông qua ProjectReference. Console Application (dotnet new console) biên dịch ra file .exe (OutputType là Exe), bắt buộc phải có một hàm Main() làm điểm khởi chạy và có thể chạy trực tiếp bằng dotnet run.

### 2. ProjectReference khác với using như thế nào?

ProjectReference là khai báo ở cấp độ project/biên dịch (đặt trong file .csproj), cho biết project hiện tại phụ thuộc vào assembly (.dll) của project khác — nếu thiếu, trình biên dịch sẽ không tìm thấy assembly chứa kiểu dữ liệu cần dùng dù mã nguồn có đúng đến đâu. using là khai báo ở cấp độ mã nguồn (đặt trong file .cs), chỉ có tác dụng rút gọn cách gọi tên kiểu (không phải viết tên đầy đủ

MyLib.LibBaiTap mà chỉ cần LibBaiTap) — using không tự tạo ra tham chiếu assembly.

**3. Tại sao có using MyLib nhưng vẫn có thể bị lỗi nếu chưa ProjectReference?**

Vì using chỉ là một chỉ dẫn cú pháp giúp trình biên dịch biết tìm kiếm kiểu dữ liệu trong namespace nào khi gọi tên rút gọn, chứ bản thân nó không nạp assembly chứa namespace đó vào project. Nếu project chưa có ProjectReference tới MyLib.csproj thì trình biên dịch không hề "biết" đến sự tồn tại của assembly MyLib, nên dù có using MyLib; vẫn báo lỗi dạng CS0246 (không tìm thấy kiểu hoặc namespace) vì namespace đó chưa từng được nạp vào ngữ cảnh biên dịch.

**4. Từ khóa public có ý nghĩa gì khi class nằm trong thư viện được project khác sử dụng?**

public cho phép class (và các thành viên bên trong nó) được truy cập từ bên ngoài assembly chứa nó. Một class hoặc method trong Class Library nếu không khai báo public (mặc định top-level class là internal) thì chỉ có thể được sử dụng trong nội bộ assembly MyLib, project khác dù đã ProjectReference và using đúng vẫn không gọi được, trình biên dịch sẽ báo lỗi CS0122 ("is inaccessible due to its protection level").

**5. Khi một project có nhiều Main(), StartupObject dùng để làm gì?**

Khi một Console Application có nhiều hơn một hàm Main() hợp lệ (nằm trong các class khác nhau), trình biên dịch không thể tự xác định đâu là điểm khởi chạy chương trình và sẽ báo lỗi CS0017. Thẻ <StartupObject> trong file .csproj dùng để chỉ định tường minh class nào (ghi đầy đủ Namespace.TenClass) chứa hàm Main() sẽ được dùng làm entry point khi build/run, giúp giải quyết tình trạng nhiều entry point mà không cần xoá bớt code.

**6. [Fact] trong xUnit có ý nghĩa gì?**

[Fact] là một attribute của xUnit đánh dấu một phương thức là một unit test đơn, không nhận tham số và luôn đúng với đúng một bộ điều kiện cố định

(khác với [Theory] dùng cho test có tham số hoá chạy nhiều bộ dữ liệu qua [InlineData]). Khi chạy dotnet test, xUnit sẽ tự động tìm tất cả các method được gắn [Fact], thực thi từng method và báo cáo Pass/Fail dựa trên các câu lệnh Assert bên trong.

**7. Test nào kiểm tra nhánh x1 > x2 trong hàm GiaiPTBac2?**

Test GiaiPTBac2_HaiNghiem_LuonDuocSapXepTangDan (bộ dữ liệu a = -1, b = 3, c = -2) là test kiểm tra nhánh này. Vì hệ số a âm, công thức nghiệm thô ban đầu cho ra x1 lớn hơn x2, buộc khối lệnh if (x1 > x2) { hoán đổi tmp } phải được thực thi để đưa kết quả về đúng thứ tự x1 ≤ x2 như test đã Assert.True(x1 <= x2).

**8. Tại sao không nên phụ thuộc hoàn toàn vào phép so sánh delta == 0 với double?**

Kiểu double lưu trữ số thực theo chuẩn dấu phẩy động (IEEE 754) nên không thể biểu diễn chính xác tuyệt đối mọi giá trị thập phân — các phép tính trung gian như b*b - 4*a*c có thể tích luỹ sai số làm tròn rất nhỏ (ví dụ 1e-16) khiến một delta về lý thuyết bằng 0 lại được tính ra khác 0 (hoặc ngược lại). Nếu so sánh tuyệt đối delta == 0, chương trình có thể nhầm một nghiệm kép thành hai nghiệm phân biệt rất gần nhau, hoặc nhầm thành vô nghiệm. Giải pháp an toàn hơn là so sánh Math.Abs(delta) < EPS với một ngưỡng epsilon nhỏ phù hợp với độ lớn dữ liệu.

## VI. KẾT LUẬN

Qua bài thực hành, đã nắm được quy trình tạo Solution nhiều project bằng dotnet CLI, cách tách phần logic nghiệp vụ ra Class Library và tái sử dụng thông qua ProjectReference, cách xử lý các lỗi thường gặp liên quan đến namespace/assembly reference và nhiều entry point, cũng như cách viết và chạy unit test với xUnit cho hàm giải phương trình bậc hai. Đặc biệt, việc so sánh số thực double cần sử dụng epsilon thay vì so sánh tuyệt đối để tránh sai số do biểu diễn dấu phẩy động, đây là một kinh nghiệm quan trọng khi lập trình các bài toán tính toán số học trong thực tế.